



ARISTOTLE
UNIVERSITY
OF THESSALONIKI

Forecasting renewable energy generation using deep learning

Dimosthenis Elmas

A thesis submitted to the
Faculty of Sciences
of the School of Informatics
of the Aristotle University of Thessaloniki
in partial fulfillment of the requirements for the

Master's degree in Artificial Intelligence

Supervising professor: Nikolaidis Nikolaos

* This is an unofficial translation (made by the author) of the original text submitted in the Greek language. The official version can be found in the Aristotle University's library website: <https://ikee.lib.auth.gr/record/356904?ln=en>

September 2023

Table of Contents

Introduction	6
Thanks	7
Part A: Basic Concepts.....	9
1. A brief overview of key technologies: From RNNs to Transformers	9
1.1 RNNs.....	9
1.1.1 Introduction	9
1.1.2 The problem with Vanishing gradients	11
1.1.3 Other issues	14
1.2 Transformers	14
1.2.1 Introduction	14
1.2.2 Sequence to sequence	14
1.2.3 The general idea.....	16
1.2.4 Advantages.....	17
1.2.5 The Layers of the Transformer	18
1.2.5.1 Embedding	18
1.2.5.2 Positional encoding.....	19
1.2.5.3 Attention	20
1.2.5.4 Post layer normalization (Add & Norm)	25
1.2.5.5 Feed forward layer	25
1.2.5.6 Multi head attention	25
1.2.5.7 Masking.....	26
1.3 A note about the Transformer's training.....	26
Part B: Introduction to the subject of this thesis.	28
2. Forecasting the energy production of Renewable Energy Sources parks	28
2.1 General overview	28
3. Previous work on the subject.....	29
4. The datasets	31
4.1 The construction of the data loaders	33

5. Introductory elements on the proposed methods	34
Part C: First proposed method	35
6. Basic overview of the first proposed method (spatio-temporal attention, Informer, sliding windows)	35
7. Spatio-temporal attention.....	36
8. The problem of attention layer's complexity	37
9. Variations for the attention mechanism	38
10. Informer	39
11. Detailed description of the proposed methods.....	41
11.1 Introduction	41
11.2 Implementation topics	42
11.2.1 The encoder input.....	42
11.2.2 Embeddings και Positional encoding	44
11.2.2.1 Temporal embedding	45
11.2.2.2 Positional encoding	46
11.2.2.3 Token encoding	48
11.2.2.4 Farm index embedding	48
11.2.2.5 Feature embedding.....	48
11.2.3 Analysis of embeddings	48
11.2.4 The mask in the decoder's self-attention.....	49
11.2.5 The feed forward layer in the decoder.....	54
11.2.6 Training on vanilla Transformer and on the Informer	54
11.2.7 The decoder's predictions on Informer.....	55
11.3 Two stage training using sliding windows	56
12. Comparison with other methods	58
12.1 Temporal fusion transformer	58
Introduction	58
12.2 The TFT architecture	59
13. Evaluation of the experimental results	60
13.1 The network parameters.....	60
13.2 Tables of scores	62
14. Conclusions	64
15. Comments on the problem of overfitting	72
Part D: Second proposed method.....	73

16. Diffusion.....	73
16.1 Introduction	73
16.2 Adding the noise	73
16.3 Sinusoidal embedding.....	75
16.4 Reverse diffusion.....	76
17. Using diffusion to construct synthetic data for time series	77
17.1 CSDI: Conditional Score-based Diffusion Model for Probabilistic Time Series Imputation	77
17.2 two stage spatiotemporal attention	78
17.3 Other details about the CSDI network	79
17.3.1 Incorporation of the Variable Selection Layer from the TFT architecture	81
17.4 Results of electricity production forecasts from wind farms using the CSDI network.....	83
17.5 Conclusions.	86
18. Aggregate results for all methods	87
Part E - Other thoughts - Suggestions for future study.....	88
19. Suggestions for future study	88
19.1 Comments on synthetic data construction.....	88
19.2 Overview of available techniques for producing synthetic data.....	88
19.2.1 GANs.....	88
19.2.1.1 The problems of the GANs.....	89
19.2.1.2 Wasserstein GAN with Gradient Penalty (WGAN-GP).....	89
19.2.2 Real NVP.....	90
19.2.3 Diffusion	92
19.3. Other suggestions and ideas for the architecture of a noise reduction network.	92
19.3.1. Other ideas-suggestions: Training with sliding windows	93
19.3.2. Sinusoidal encoding for time series data	94
20. Epilogue.....	94
Bibliography - references	97
Appendix	101
Tables Index.....	105
Equations Index.....	106
Code snippets in PyTorch (for the first proposed method).....	106

Introduction

In this thesis we will study, among other things, the problem of forecasting time series concerning the production of electricity from renewable energy parks. The multivariate time series we used concern a set of wind farms which, due to their geographical proximity, may exhibit correlations in terms of the electricity production. Such time series partly depend on future interpretive variables or covariates, (that is, variables that refer to future moments such as weather forecasts, etc.), and partly on the so-called spatio-temporal correlations (as mentioned in the relevant literature) (J Grigsby 2021).

We will focus our effort on the use of neural networks. The neural network architecture that will mainly concern us is that of the Transformer and its variants. In the following chapters we are going to explain all the terms in more detail.

The paradigm we will be using concerns the forecasting of electricity production from wind farms. Essentially, we are interested in investigating ways of building machine learning models that can learn rich correlations between past and future time points. We are also interested in studying the use of the modern technique which is called 'diffusion' (Ho 2020). We will explain more about the diffusion-based models later in this text.

As we will see, our study will bring to the surface several problems related to the complexity of the underlying algorithmic mechanisms and, in particular, to the so-called Attention, but also the (possible) need for sufficiently large data sets or, the possibility of enriching them with "artificial" data which can be produced in various ways, either simple ones such as e.g. by adding noise, or more sophisticated ones such as e.g. using GANs, Diffusion techniques, etc.

As mentioned above, in this work we have chosen to focus our attention on the family of neural networks based on the so-called Transformer architecture. This is done for a variety of reasons. Firstly, the networks based on this architecture, and especially on the attention mechanism, have been proven in practice to be a very powerful tool and one of the vehicles that have led in recent years, to an unprecedented development in many areas, mainly in natural language processing, automatic translation, etc.

As for the problem of natural language translation and natural language processing specifically, the Attention mechanism and the Transformer architectures as well as their variants, have led to top results and have essentially opened new ways to work with such problems.

Also, these networks have the ability to learn complex correlations between the data and in fact, under certain conditions, they may even reveal to us correlations that humans have not realized existed, for example, through the study of the so-called 'Attention Scores' which we will be describing later in this text.

Of course, the main reason that made us turn our attention specifically to these architectures, is the fact that they have essentially opened new roads, which, rightly, have directed a significant part of the academic research into the study and exploitation of such networks.

In the following paragraphs we will start with an introduction to the architectures based on the Attention mechanism and in particular to the Transformer networks. This will help the reader both to better

understand the underlying mechanisms, and to identify the limitations and problems that these types of networks face.

Naturally, each tool has its advantages and disadvantages. We will examine them in more detail below. We will also look at some of the ways that have been proposed to deal with such issues. As we will see, our demand for the production of rich spatio-temporal associations, will lead us to search for variations of the Attention mechanism, such as e.g. the ‘probSparse’ attention which is used by a variant of the Transformer called the Informer (Zhou 2021) (J Grigsby 2021). This, as we will see, is necessary to deal with the problem of the quadratic complexity of the Attention mechanism, at least in the vanilla (basic) version of it.

We will also study the use of the Diffusion technique based on the interesting work by: (Yusuke Tashiro 2021). In this method the network learns to process a time series to which some amount of noise has been intentionally added. As we will see, training the network to predict the added noise presents many interesting possibilities. This technique has been used extensively for image processing (Foster 2023) and it seems that it can be generalized for time series with satisfactory results.

Finally, let us note at this point that our purpose in this work is not to build a product, but rather to explore some of the tools available to us for forecasting multivariate time series using deep learning techniques. It is our hope that this paper give the reader a useful overview and analysis of some of the most important methods and techniques that can be used, as well as, perhaps, some more general thoughts and ideas for building a time series forecasting system.

Finally, we would like to emphasize that we have made an effort to accurately record all the items of our research and have carefully checked the entire text. All sources have been recorded and any images we have relied upon have been checked in terms of their copyright permissions etc. The reader is kindly asked to bear in mind that there is always the possibility that some typographical or other type of error may have occurred.

Thanks

I would like to thank my supervisor, prof. Nikolaos Nikolaidis, as well as Mr. Haralambos Symeonidis, (Ph.D.), for their support. I also want to thank my brother for the many hours of discussions we had on various technical issues, and of course my parents.

Overview of the content of this thesis

At this point it would be useful to give a general overview of this thesis:

Firstly, for the better clarification of the concepts and for the convenience of the reader, we will provide a brief introduction to the basic architecture of the Neural Networks that we will be using, i.e. the Transformers. We will see the reasons why the contribution of this architecture is important and its

advantages in relation to the Recurrent Neural Networks, (RNNs), whose problems were the starting point for the development of the Attention mechanism and the Transformer-based networks.

Next, we will give a general description of the problem we are interested in studying. This is the problem of forecasting the energy production of Renewable Energy Parks. After giving a brief introduction to the problem in question, we will refer to some of the ways of dealing with it that have been proposed in the scientific literature.

Then follows the description of our attempt to deal with the problem we are studying. We will examine the basic idea of the Spatio-Temporal Attention as proposed by (J Grigsby 2021), on which we have partially relied upon, but also the integration of information regarding future weather forecasts to our neural network. As we will see in the following chapters, we can modify the Transformer's Decoder Layer so that, with the use of a suitable Mask and a suitable Encoding, we could potentially integrate the information about future weather forecasts into the training loop of the network. We will also describe the problems that arise from the use of the so-called vanilla (i.e. basic, as it is usually called), version of the Transformer architecture. As we will see, the quadratic complexity of the basic Attention layer will lead us to the use of variants of it such as the 'probSparse' Attention (Zhou 2021) of the Informer architecture, as is also proposed in the work of (J Grigsby 2021). We will also consider the case of training the network in two stages, using 'Sliding Windows' (the 'sliding' referring to the temporal direction). Next, we will be discussing the results we got from our experiments, but also the problems of the method we studied, such as the problem of overtraining and overfitting.

We will also consider an alternative method for dealing with the problem of forecasting the energy production from Renewable energy parks (wind power parks) based on the so-called Diffusion technique and more specifically on the important work of (Tashiro 2021).

Finally, we will close this thesis with our conclusions and suggestions for future research on the subject, making special reference to the Diffusion technique regarding the possible construction of synthetic (artificially derived) timeseries data.

Part A: Basic Concepts

1. A brief overview of key technologies: From RNNs to Transformers

At this point it was considered appropriate to offer a brief and basic introduction to the central topics concerning the technologies we will be using. This is done to make it easier for the reader (in case he/she is not familiar with the network architectures called Transformers, the mechanism of the so-called Attention, etc.) and to get a better picture of the underlying mechanisms, their possibilities and also their problems. Thus, we hope that the presentation of our experiments in the following paragraphs will be facilitated by contributing to the clarification of the more detailed presentation that will follow.

1.1 RNNs

1.1.1 Introduction

Before the advent of the Transformer architecture, neural network researchers had at their disposal the so-called Recurrent Neural Networks (RNNs). Architectures based on RNNs can efficiently solve many interesting problems and are often a valid and appropriate choice. However, RNNs also have some limitations, which were part of the reason behind the development of newer architectures based on the Attention mechanism, the Transformers, etc. We will refer to this issue in more detail below.

RNN networks process sequences of tokens, (as each individual element of the sequence is usually called) after their possible embedding (a type of codification which will be discussed later). Typically, these networks attempt to capture the knowledge that can be harnessed from examining previous tokens in the sequence in some condensed, summary form h_t where t is the temporal dimension. Thus, by leveraging this knowledge, they can provide predictions that better respond to the sequential nature of the data.

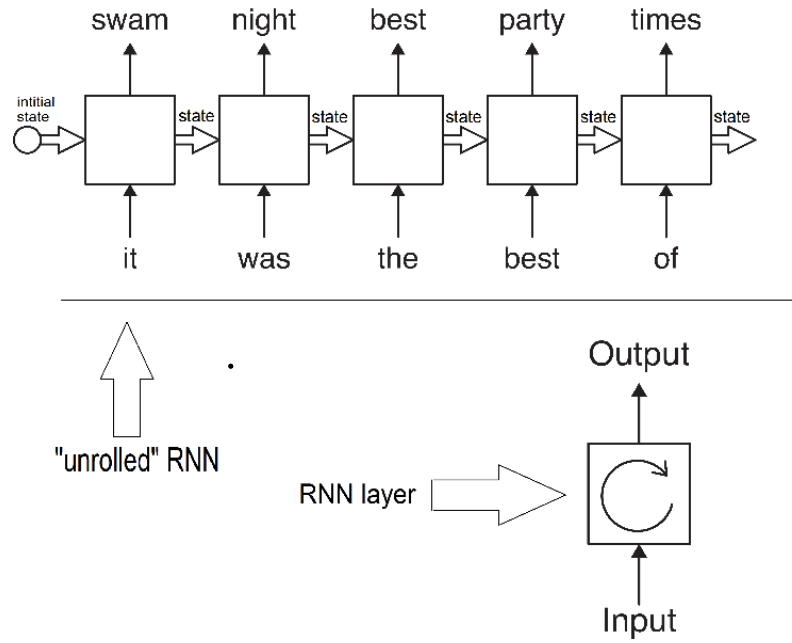
It is clear that data in the form of a sequence inherently contains the concept of 'next' and 'previous'. Therefore, in such sequences there is some temporal correlation or (equivalently) some positional correlation. When each token is associated with some time-related information, (e.g. encoded or embedded using a timestamp), we are talking about a temporal correlation. Or it could be that the correlation may concern the position of the token within the sequence, like when we process e.g. sequences of words (sentences) in a natural language.

When processing data in the form of sequences, it makes sense to try to somehow capture the knowledge from processing previous tokens. When reading some text in a natural language for example, we can predict, to some extent, the next word because it often depends not only on the previous one, but also on the general context (conceptual, semantic, grammatical, syntactic, etc.) in which it is placed.

On the other hand, if we change the order of words in a text or even in a sentence, its content can become completely incomprehensible. Thus, the information depends both on the relationship between the tokens (e.g. words in a sentence) and on their position in the sequence.

RNN networks define the concept of 'state' to encode the information that has resulted from the sequential processing of the data.

Picture 1 presents the basic idea behind RNNs. (We relied on the images provided by: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)



Picture 1. Unrolled RNN. Picture based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)

Essentially the image describes the following idea, expressed symbolically:

$$p(x_t | x_{t-1}, \dots, x_1) \approx p(x_t | x_{t-1}, h_t) \quad (\text{Zhang 2023})$$

Statement 1. The definition of the RNN.

Where h_t is the 'state' or 'hidden state' as it is often called. It is essentially a latent variable which, theoretically, is calculated as:

$$h_t = f(x_t, h_{t-1})$$

Statement 2. Hidden state (RNN) (Zhang 2023)

Usually, for the best visual presentation, we use the so-called "unrolled" form of the RNN network. In this way we represent the successive inputs and outputs of an RNN cell at successive time points. As you can see, the idea is for the network to sequentially process all the words (or tokens) of the sequence one by one. At each step, the network produces 2 vectors or tensors (note that, technically, the word 'tensor' as

used in the context of a neural network building framework like PyTorch etc., is not equivalent to the mathematical definition of a tensor): the state, and the output. The state is a vector that we hope captures all the information and knowledge that the network can derive from the previous inputs it processed. This knowledge, which captures what the network ‘saw’ at previous moments in time, will be used to produce the prediction for the next word or token (the ‘output’ as shown in the image).

At this point we need to make a clarification: The network, as seen in its unrolled form, is just a visualization for the best presentation of the concepts. In reality, there is only one RNN network and not five of them, as shown in the picture. That is, the image represents the network at different moments in time. Thus, the concept of the sequential processing of data by the network is better understood. This means that, for the network to produce the prediction for the 3rd word for example, it must have successively processed the first and then the second word or token etc.

1.1.2 The problem with Vanishing gradients

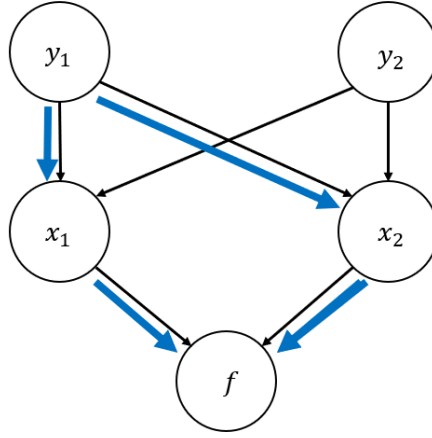
RNNs have some issues that we need to take into account, especially when processing relatively longer sequences. A key problem of RNNs is the so-called vanishing gradients problem. Let us see in more detail what this means and its implications:

RNN networks, for their training, are based on an algorithm known as backpropagation through time (BPTT). The backpropagation process is the basic mechanism by which neural networks with multiple hidden layers are trained regardless of their architecture. The propagation of the error during training (backpropagation) through the BPTT algorithm can end up in quite large computational graphs, effectively making RNNs a deep architecture with many layers which are distributed in time. (Note: speaking in terms of PyTorch or TensorFlow programming, these computational graphs are stored in data structures called gradient tapes (see: (TensorFlow n.d.))). In other words, these layers arise, not from the "static" structure of an individual RNN cell, but from the way the output of one moment in time is fed to the input of the next. You can find a detailed explanation about these algorithms in the important book (Zhang 2023).

All networks that have a multitude of hidden layers, and especially the RNNs, whose unrolled form reveals this latent multi-layered nature, can exhibit the problem called vanishing gradients. When calculating the gradient through the backpropagation process, values for the gradient of the loss function which tend to zero may appear. This can have a devastating effect on the learning process for the network, as learning will slow down or even stop altogether. Even if the problem occurs in an ‘internal’ layer of the network, a zero gradient will result in the ‘paralysis’ (stopping the learning process essentially) to all of the previous layers which are connected to it.

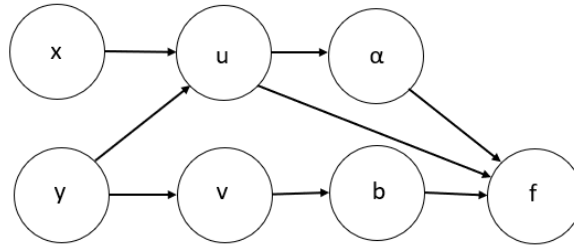
The backpropagation process is based on the chain rule. In the most general formulation this rule is

written as: $\frac{df}{dy} = \frac{df}{dx} \frac{dx}{dy}$



$$\frac{\partial f}{\partial y_1} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial y_1} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial y_1}$$

Picture 2 The chain rule. Based on: (Tamura 2023)



$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \frac{\partial a}{\partial u} \frac{\partial u}{\partial y} + \frac{\partial f}{\partial u} \frac{\partial u}{\partial y} + \frac{\partial f}{\partial b} \frac{\partial b}{\partial v} \frac{\partial v}{\partial y}$$

Picture 3 The chain rule. A more complex example. Source: (Zhang 2023)

As can be seen from **Picture 3**, the products of the gradients of the function can be extended by the addition of multiple layers.

This problem will not be solved even if one resorts to the logarithmic space to take advantage of the exponent's ability to express small values. (Ian Goodfellow 2016). As mentioned in (Zhang 2023), the eigenvalues of the weight matrices of each layer may vary greatly during training, and the product of all the gradients may be too small or too large. More specifically, in an RNN network, during the process of training there will be paths in the computational graph of the backpropagation algorithm in which the exponentiation of a matrix $\mathbf{W}$ to a certain power depending on the number of steps in the sequence will be needed. (Ian Goodfellow 2016)

The eigen-decomposition of $\mathbf{W}$ will be $\mathbf{W} = \mathbf{V} \text{diag}(\lambda) \mathbf{V}^{-1}$ from which it arises that:

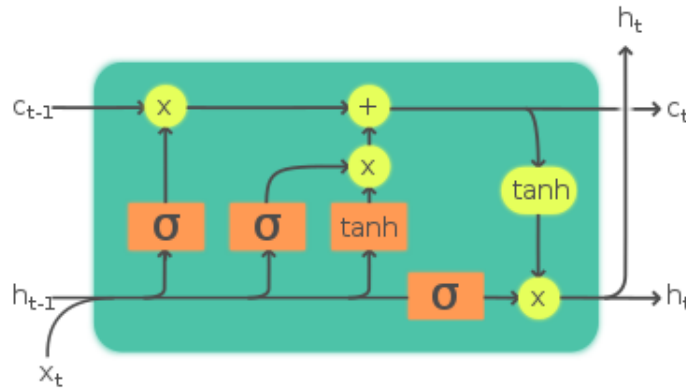
$$W^t = V \text{diag}(\lambda)^t V^{-1}$$

Statement 3. The problem with using very deep networks.

(please See: (Ian Goodfellow 2016) and (Zhang 2023) from where this paragraph is mainly derived.)

The problem is created by the term $(\lambda)^t$ as any eigenvalue that is sufficiently below 1 in absolute value will "vanish", or if it is sufficiently above 1 it will "explode" (the exploding gradients problem), resulting in either the stopping of learning or a very unstable training process since the weights of the W matrix will take very small (close to zero) values or very large affecting the stability of the training and essentially canceling the balancing action of the optimizer. (derived from: (Ian Goodfellow 2016))

Usually, to deal with this problem, we resort to the use of some kind of the so-called skip connections or residual connections, as these were proposed in the ResNet architecture. The problem of vanishing gradients for RNNs with relatively longer inputs can be partially addressed by using the LSTM and GRU networks, which are a more elaborate form of RNNs.



Picture 4 An LSTM cell. Source of this image (Chevalier 2023)

The figure shows the architecture of an LSTM.

Generally speaking, the exact explanation of why an LSTM network addresses (to some extent) the problem of the vanishing gradients pertains to a more extensive discussion and for this reason we will be referring the reader to the important book: (Zhang 2023) where they will find a more detailed explanation. You may also refer to the webpage: (Fleuret n.d.) where you can find an interesting series of slides (presentations) on the issue of RNN networks etc. Here, we will simply mention the following: The problem of the vanishing gradients is related to the repeated use of non-linearities, i.e. activation-functions. Of course, the usage of activation functions is necessary in any deep neural network (in the appropriate layers, depending on the specific architecture), both in order to introduce nonlinearities to the network's trainable layers and also to achieve the required depth in the deep neural network. Note that without the usage of activation functions, any deep neural network collapses to essentially a single neuron. This is the reason non-linearities (activation functions) are necessary.

The basic idea behind the LSTM and its related architectures is to use a mechanism of bypassing long chains of repeated activation functions (sometimes called: pass-through). Thus, here we are just going to note that the derivatives of the loss function with respect to the cell state will not vanish in these

architectures. For more information the reader is kindly requested to refer to the relevant literature e.g.: (Zhang 2023), (Fleuret n.d.) etc.

1.1.3 Other issues

RNN networks are an important class of networks with many interesting applications, but they have two main problems. As for the first one, we have already mentioned it. It is the problem of vanishing and exploding gradients. At this point it would be useful to mention one more issue to consider when designing networks using RNN-based architectures. RNNs will need to condense all the knowledge that can be gained from processing the previous inputs of a sequence into a hidden state (or, in the case of LSTMs into a 'hidden state' and a 'memory cell'). This architecture will certainly exert some "pressure" on the network, in the sense that it will not be easy to find an encoding for all this knowledge that captures the complex interrelationships between previous moments in time. Note that this encoding needs to be compressed into just one or two tensors (as these are defined in PyTorch, TensorFlow etc.) (Glassner, Deep Learning: A Visual Approach 2021). That is, there is a limit to what can be stored in the network 'state' tensor.

Finally, another problem is that RNNs need to process their input data one by one. Therefore, it seems likely that there is room for improvement, as with some other neural network architecture (Transformers, Attention layers etc.) we will be able to take advantage of the possibilities offered by parallel processing on modern graphics cards, Tensor Processing Units (TPUs), etc.

1.2 Transformers

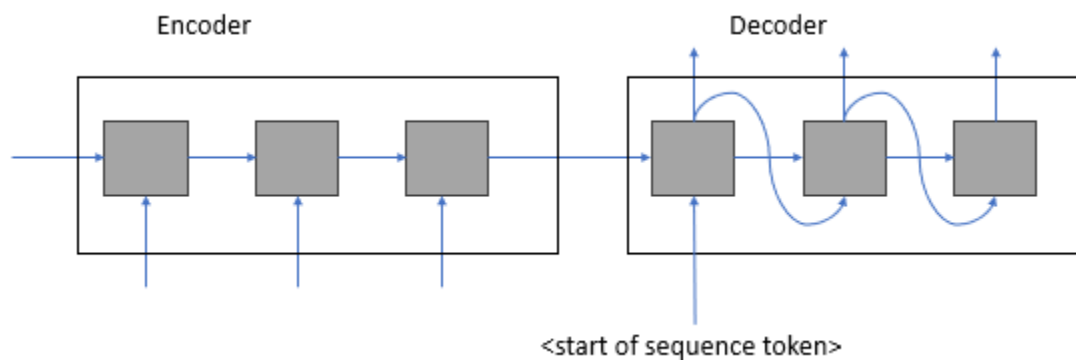
1.2.1 Introduction

Networks based on the transformer architecture were proposed in 2017 in the important work of (Vaswani 2017). In the original publication this architecture was used to build networks that do automatic translation from one natural language to another. Regarding the problem of natural language processing, these networks have displayed top performance and are essentially the main tool for natural language processing at the moment. As we will see next, these networks work in the logic of the so-called 'sequence to sequence'. That is, they transform one sequence into another sequence. The new transformation or encoding will allow the subsequent stages of the network to more accurately produce the desired outcome.

1.2.2 Sequence to sequence

The 'Transformer' as a deep learning architecture, was proposed as an alternative tool for working with problems otherwise approached with the so-called 'sequence to sequence' architectures. Essentially, networks based on the 'sequence to sequence' architectures are similar in logic to auto encoders, but they involve sequences and not just single elements.

In Picture 5 we see the basic logic for a ‘sequence to sequence’ architecture using RNN networks. Let us examine the picture in the context of natural language translation:



Picture 5. Sequence 2 sequence using an RNN. Based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)

We have 2 distinct parts: The encoder, on the left, and the decoder, on the right. The idea is to use the encoder of the network to find an encoding for the input sequence, (e.g. for the problem of natural language translation, the encoder’s input sequence would be the sentence in the original language) which is calculated in such a way that it captures all the knowledge about the interrelationships between the tokens and the structure of the entire sequence, in order to facilitate the decoder to produce its prediction (e.g. the translation of the sentence into the target language).

In the image’s example the encoder consists of an RNN network. Taking as an example the problem of natural language translation, the encoder takes as its input the words of the original sentence and produces the so-called ‘context’ vector. Essentially, it is the hidden state that the network has calculated. As we can see from the image, the forecasts of the specific network (encoder) are essentially of no concern to us (note that there are no arrows pointing upwards, ‘exiting’ the encoder’s RNN cell in the image above). Essentially, we are only interested in encoding the knowledge resulting from the sequential processing of the input tokens. Thus, the encoder will produce, (in the form of the context vector), an encoding of the entire original sentence that will (hopefully) condense the knowledge about its structure and semantics.

The context vector is then fed to the decoder layer. The decoder, equipped with this knowledge derived from the output of the encoder, will then begin the translation process. It will initially take as input the context vector along with the special purpose token [START] so it knows to produce the first word of the translation (in the context of natural language translation). In the next steps, the input of the decoder will be the context vector (hidden state, memory cell, etc.) that has been derived from the previous moment in time, and also the previous prediction of the decoder.

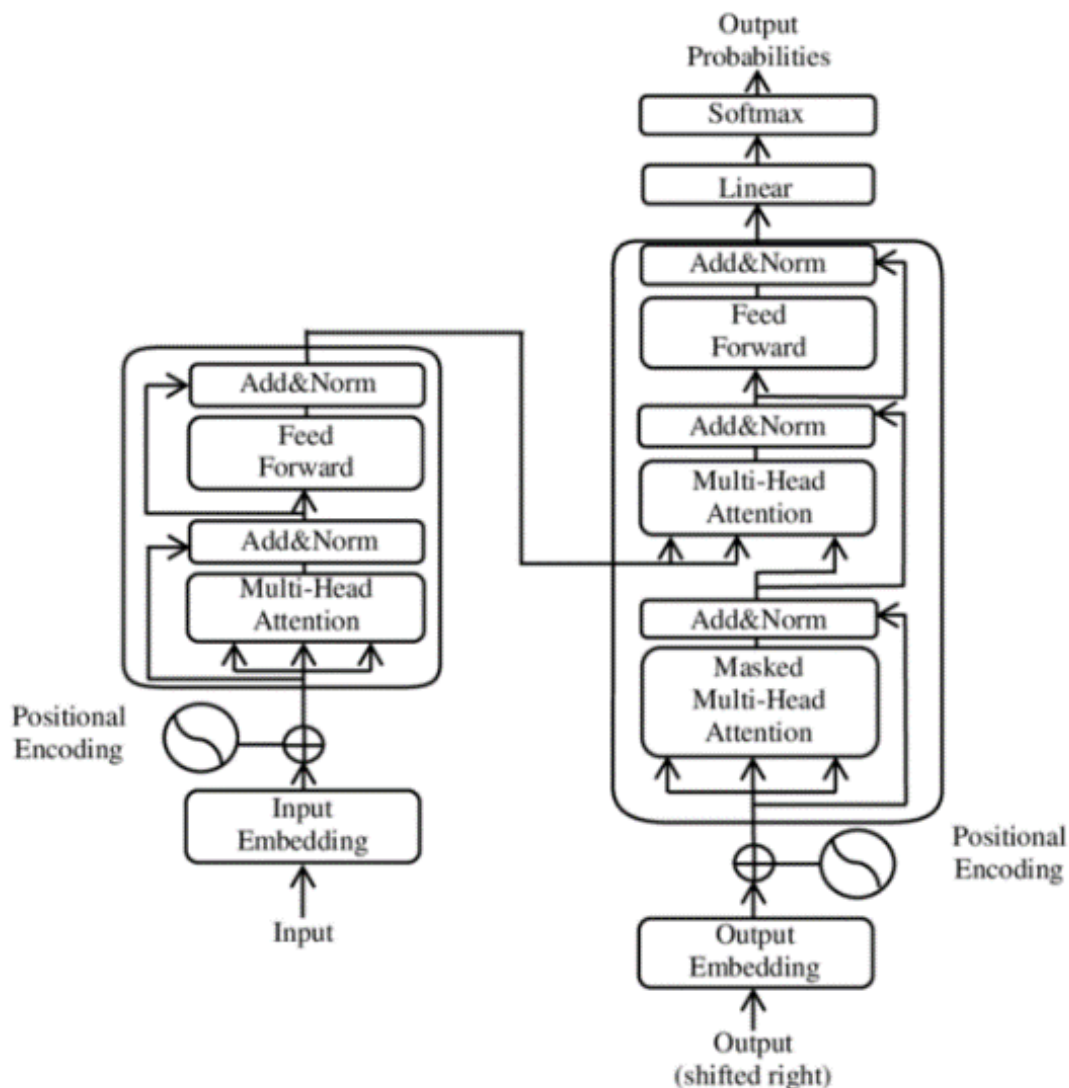
In general, the idea of encoding and decoding is fundamental to many deep neural network architectures. An encoding layer is a tool that allows the network to transform its input data into whatever form it finds most appropriate for the later stages of processing (layers) during training. Especially regarding Transformers, as we will see later in the text, this process is done in layers, often in a

similar fashion as in CNN networks for example, which use successive encodings (so-called feature maps) of increasing sophistication, from the general to the specific.

There are several variations of 'sequence to sequence' architectures but the basic idea is shown in the image above.

1.2.3 The general idea

Let's now move on to the architecture of Transformers. The next picture shows the basic structure of a Transformer. It is a more complex architecture, but the basic idea of the encoder and the decoder is maintained, as well as the 'sequence to sequence' logic.

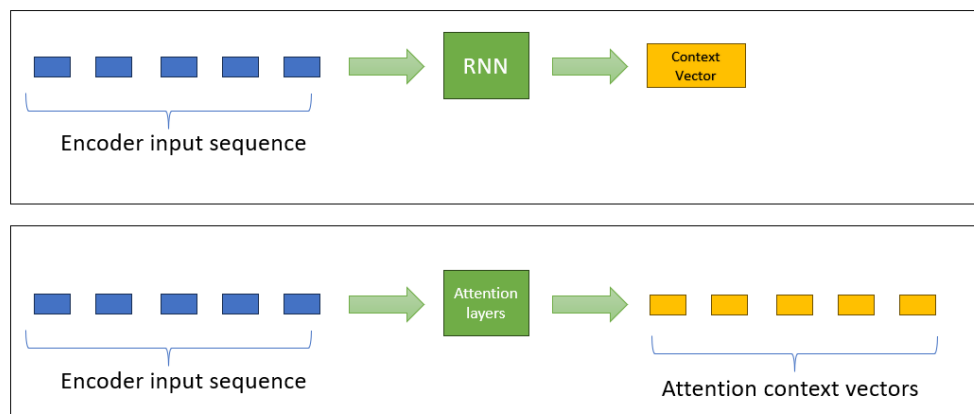


Picture 6 Transformer. The basic architecture. Source: (Jia 2023)

As we will see next, the encoder (which is shown in the left part of the image), takes the original sequence of the input and passes it through a first general stage of processing called embedding which

works at the level of individual words (it does not take into account the intercorrelations between the words). This stage is shown in the image by the items flagged as 'input embedding' and 'positional encoding'. Often, the input embedding is a pre-trained layer if there is such a possibility as e.g. in the case of the processing of a text. We will study these layers in more detail later. At this point we are interested in examining the general idea.

In the next stage we have a second form of information processing based on the so-called 'Attention' mechanism. In this case the tokens are processed by taking into account the intercorrelations between them. Thus, the knowledge from examining the sequence has now been captured not just in a single context vector, as when we using RNNs, but in the very encoding of each of the individual words of the sequence. This means that the attention mechanism will construct a new sequence with, possibly, the same length as the original one, but whose encoding of the individual tokens will be such as to incorporate all the knowledge resulting from the sequential examination of all the tokens, and the correlations between them.



Picture 7 The basic difference between an RNN and an Attention layer

1.2.4 Advantages

The core of the Transformer and all of its related variants is the so-called Attention mechanism. This mechanism has managed to effectively deal with the problem arising from compressing all the knowledge from the input sequence into a single context vector. This problem, inherent in architectures based on RNNs, can result in information loss as the size of the input sequence increases. (Uday Kamath 2022).

In summary, the main advantages that Transformers have over RNN-based architectures are that:

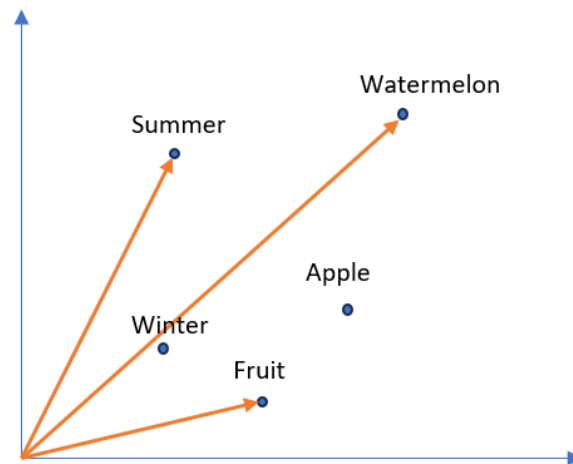
- they can construct richer encodings, with less loss of information
- they can be trained on larger data sets
- they are less sensitive to the problem of vanishing and exploding gradients

- they are able to make better use of the parallel computing capabilities offered by modern machines.

1.2.5 The Layers of the Transformer

1.2.5.1 Embedding

The first processing stage in a transformer network is the embedding of the encoder input tokens. This is an information processing stage that attempts to place the tokens of the sequence in a multidimensional space, the geometry of which is such that concepts having some kind of relevance are placed in geometric proximity within the multidimensional space defined by the embedding. An interesting consequence of embeddings is that we are often able to add or subtract embedded vectors, resulting in a point in the geometric space defined by the embedding function, that expresses a new, more complex concept defined by the union or intersection of the concepts we added or subtracted. See the next image for an example:



Picture 8 Embedding addition (Example)

Often, the embedding layer is pre-trained. For example, in the case where we are processing sequences of words in a natural language, we can use a ready-made and pre-trained layer such as GloVE, word2vec, fastText, etc. Here we have to emphasize that the embedding layer does not consider the interrelationships between tokens. Essentially, it only considers individual words and not their position within the sequence. For this reason, we are able to use pre-trained such layers. Besides, this processing serves as a translation of the tokens from their original form (set of characters of the alphabet (words) e.g.) to vectors, (tensors etc.) which the neural networks can process.

It should be noted here that the issue of constructing an embedding layer is complex and there are many variations of the algorithms. Especially for applications requiring clustering, many interesting solutions have been proposed, some of which define the concept of proximity very different ways. For example, the emphasis may be placed not on Euclidean distances but on the so-called geodesic distances, etc. We

close this side-note by pointing out that the interested reader can refer to the relevant literature for a more detailed analysis.

1.2.5.2 Positional encoding

The next stage of processing after the embedding layer is the positional encoding. As mentioned above, Transformers process the entire input sequence at once, unlike RNNs which examine the tokens one by one in the correct order. This means that without appropriate measures, the positional information of the input will be lost. That is, the network will lose the concept of ‘previous’ and ‘next’. This problem is addressed by the so-called positional encoding. The goal of positional encoding is to somehow incorporate into the existing encoding of a token, the information about the position of the token within the sequence.

Positional encoding (typically) attempts to add to the initial embedding of the token a new vector of equal dimensions that uniquely captures the position of the token within the sequence. The positional encoding does not depend on anything other than the position of the token within the sequence. Thus, this particular layer (positional encoding layer) is not a part of the trainable network (no gradients are stored in the gradient tapes etc.).

More specifically, positional encoding is a vector which is (typically) calculated using the following formulas: (Rothman 2022):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Statement 4. The equations for the positional encoding of the vanilla Transformer.

Where d_{model} is the dimensionality of the embedding vector, pos refers to the position of the token within the sequence, and i refers to one of the d_{model} values of the embedding.

As can be seen from the above formulas, the even dimensions out of the total d_{model} ones are calculated using the formula with the sine while the odd ones with the formula that includes the cosine.

Especially for positional encoding, many methods have been proposed to reduce the information loss after the addition of the token embedding and the positional encoding. In the book: (Rothman 2022) for example, we have found a reference for adding the value

$$token_embedding * \text{math.sqrt}(d_{model})$$

(where d_{model} is the embedding size) in the embedding of each token before adding the positional encoding vector. Regarding the detailed analysis behind the calculation of the positional encoding vector described by the above formulas, we will refer the reader to the relevant literature, e.g. (Zhang 2023). The important thing to remember at this point, is that the positional encoding presents a mechanism for incorporating positional information into an embedding and this mechanism is deterministic. Also, using an appropriate implementation, positional encoding does not depend on the length of the sequence.

This means that the relative distances between tokens in sequences of different lengths will be exactly equal. This allows Transformers to process sequences of different lengths, using an appropriate design. It is also proven that relative distances arise as linear transformations of some absolute position. Positional encoding can therefore capture both the absolute positional information and the relative one. More specifically, the positional encoding of the position $i + \delta$ is shown to arise as a linear transformation of the positional encoding of the position i . You can find the proof for this in the important book: (Zhang 2023).

1.2.5.3 Attention

We now come to the most important mechanism of the Transformer, the so-called Attention. The use of the Attention mechanism is not limited to Transformers. It can be used as a standalone layer to enhance architectures based on RNN, CNN, GAN networks, etc. This mechanism is able to capture the relationships that exist between the tokens of the sequence and to incorporate this knowledge into the encoding of each token itself. Unlike the RNN architecture where all this knowledge had to be condensed into a hidden state, with the Attention mechanism each token will incorporate all its interrelationships with other tokens in the sequence into its own encoding. Therefore, Attention essentially creates a new sequence of the same length as the original (we will examine variations of this case later). This new sequence will be passed to the Decoder which is trained to be able to utilize this new encoding to produce the corresponding predictions.

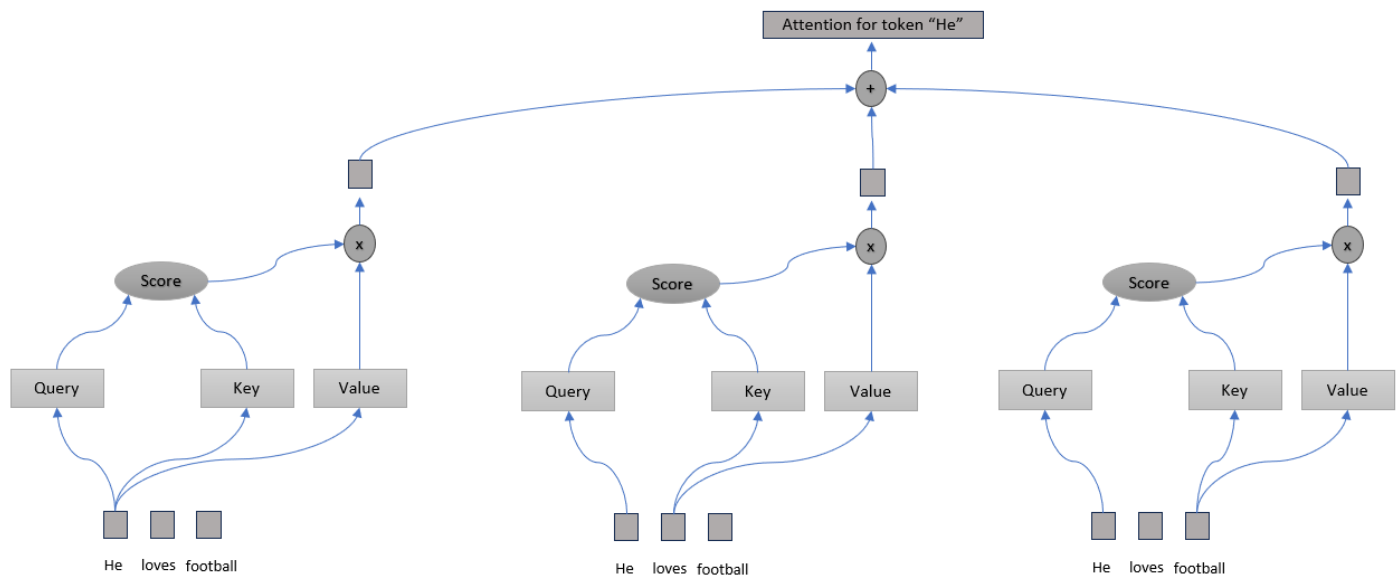
Attention is often described using the concepts of 'query', 'key', and 'value'. These concepts come from the study of database systems. These are 3 networks - subsystems that undertake to carry out the calculations needed in the Attention layer. The 'query' and 'key' networks (layers) are responsible for assessing the degree of correlation or relevance of different tokens. This is achieved through the 'score' which is calculated as a function of the output of the 'query' and 'key' networks. The 'value' network, using the calculated 'score' takes on the task of determining the 'quantity' (in a way) of information from the token associated with the 'key' (the output of the 'key' network), which will be passed to the encoding of the token associated with the 'query'.

As another example for the intuitive understanding of attention, we mention the book by: (Uday Kamath 2022) where there is a very good analogy that borrows elements from psychology to give an intuitive presentation. An Attention layer is based on the concept of focus and observation, as the name of it implies. The attention mechanism attempts, in a way, to incorporate into the encoding of each token, only what is important and useful (as defined by the loss function), removing what is unnecessary and adding or reinforcing what is important. In the book by (Uday Kamath 2022), there is a discussion based on the work of psychologist William James, who studied the phenomenon of focus and, of course, attention.

Specifically, the concepts of volitional cue, nonvolitional cue, and sensory input are defined. Consider using different colors to underline important words or content within a notebook. The concept we want to focus on is the volitional cue. This means that it is a signal that is under our control and we can choose it every time. It has arisen from our volition. The different colored marks in the notebook will correspond to the so-called non-volitional cues. And finally, sensory input can be mapped to the words themselves

or the content which has been highlighted with different marks and colors. The Attention mechanism will take over to produce an output that will mainly include information from sensory inputs related to the non-volitional cues that interest us most. So, in this analogy the query corresponds to the volitional cue, the key to the nonvolitional cue and the value to the sensory input.

The concepts of 'query', 'key' and 'value' are derived, as we said, from the study of database management systems. For example, when we want to send a query to the database, the DBMS will examine the relationship that the query has with the search keys in the database. In the context of a DBMS, 'value' refers to the stored data associated with the key related to our query.



Picture 9. The calculation of the self-attention. Image based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)

In Picture 9 we can see the arrangement we described.

As shown in the example image, we want to find the similarity between the word “he” and “loves”. As we can see, the query and key networks will produce a score. The score is a number between zero and one. We achieve this by placing a SoftMax layer after calculating the score. This layer guarantees that the sum of all the outputs of the neurons will sum to one. The score is then multiplied by the output of the ‘value’ network.

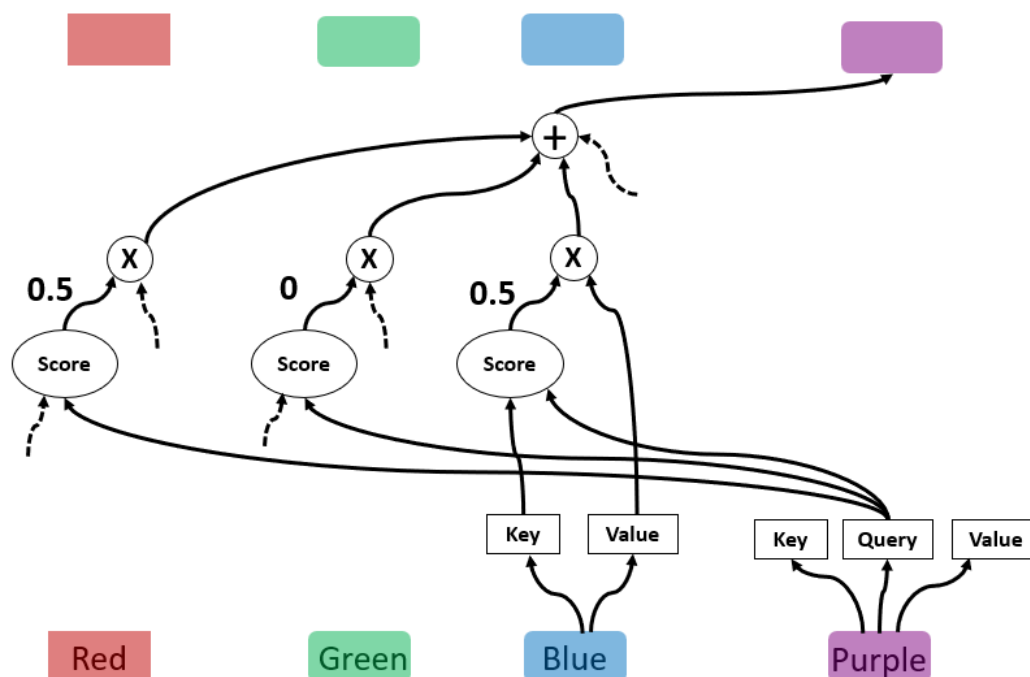
This type of calculation will be done for all the words (tokens of a sequence). That is, we will find the relationship of the word “he” (query) with the word “loves” (key), the relationship of the word “loves” with the word “football”, etc. This relationship may be at a semantic, grammatical, syntactic, or other level depending on the ‘Attention Head’, as we will see below.

The sum of all these sub-results (value times score as shown in the image) constitutes (after some other processing such as normalization, etc., which at this stage we omitted for simplicity) the output of the attention layer for the specific query.

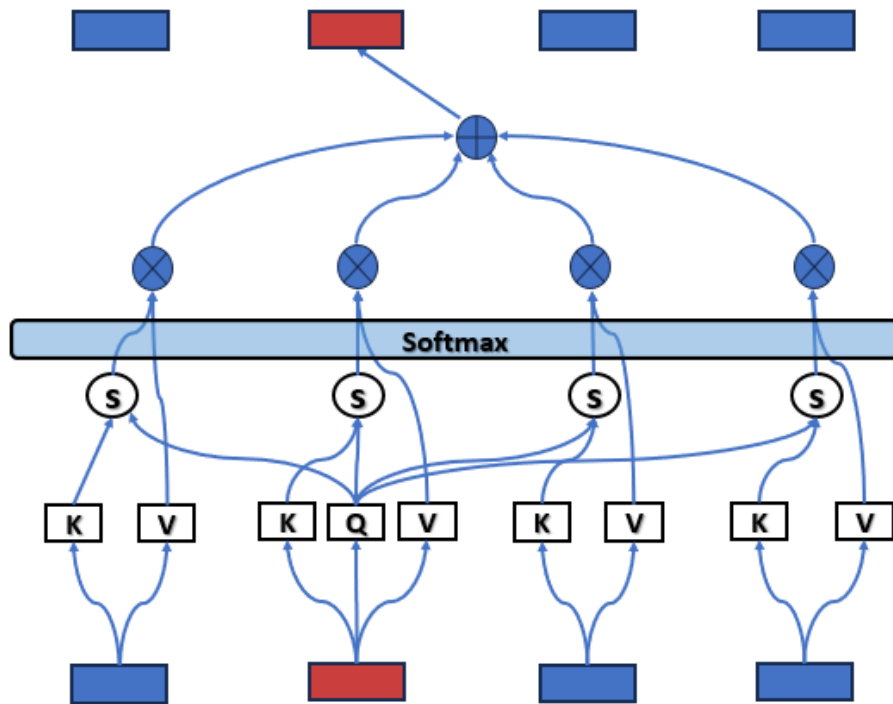
Let's look at another example:

In the book by (Glassner, Deep Learning: A Visual Approach 2021) there is a very interesting idea for explaining how Attention works intuitively. The author uses the analogy of mixing colors to create a new shade. We will therefore rely on this example to provide another insight into how the Attention mechanism works.

Suppose we have the names of various colors and we want to generate their codes in hexadecimal format. E.g. for the blue color it is #0000ff etc. If we use an attention layer we will have something like what is shown in Picture 10 (example).



Picture 10 An intuitive basic understanding for the Attention Scores



Picture 11 Self-attention. (Based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023))

As we know, the color purple is made up of 50% blue and 50% red. In our simplistic example the corresponding scores were calculated as 0.5 and 0.5. This number will determine the contribution of each token (red and blue in the case of the image) to the final encoding result for the word "purple". The description we gave above, assumes that the queries keys and values will all be derived from the same sequence. This is called Self-Attention and it is shown in Picture 11: For a specific query, we calculate all the scores, which have been derived as a function of the attention query and the respective attention key. Then, the attention output for that particular query is calculated as some kind of weighted average of the attention values (output of the 'value' layer of the network). This process continues for all queries until a new sequence of equal length to the original is produced.

At this point, we have to make a reference to the distinction between self and cross attention. In self-attention, the queries and the keys are derived from the same sequence. This means that attention will produce an encoding that associates tokens with other tokens from the same sequence including their "self". In contrast, in the so-called cross attention, the queries come from one sequence, while the keys and values come from another sequence possibly of a different nature than the first one (e.g. having sentences in two different languages). Cross attention will therefore find similarities between elements of 2 different sequences. As we will see later, cross attention is used in the decoder of the Transformer architecture.

At this point we will give a more formal and precise description of attention based on the book by: (Uday Kamath 2022).

According to the previous intuitive discussion, the Attention layer's basic mechanism groups pairs of 'keys' and 'values' $(k_1, v_1), \dots (k_n, v_n)$ where n is the size of the sequence. That is, a specific key is related to a specific value. The score calculated by attention is essentially a measure of the similarity between query and key.

$$a_i = a(\mathbf{q}, \mathbf{k}_i)$$

Statement 5. Attention score

One way to calculate the degree of similarity is to use the inner product of the vectors. Therefore:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q} \cdot \mathbf{k}$$

Statement 6. Attention score as inner product

A problem that has been highlighted with the above calculation is that for vectors with large dimensions the inner product can take on very large values. This will create issues during the training of the network. To address the problem, we typically calculate the index of similarity (a) using this function:

$$a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}$$

Statement 7. Attention Scores (before the application of the SoftMax layer)

Where d_k is the dimensionality of the keys.

We have to note that there are many ways to calculate a . The most common is the one above. The interested reader can refer to (Minh-Thang Luong 2015) for an alternative case.

We then use the SoftMax function so that the sum of all the scores always amount to unity.

$$\mathbf{scores} = \text{softmax}(\mathbf{a})$$

Finally, the scores are multiplied by the values:

$$\mathbf{o} = \sum_{i=1}^n b_i \mathbf{v}_i$$

In a more compact description using matrices, we have:

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Statement 8. Calculating Attention. Attention can be thought of as a kind of weighted average.

1.2.5.4 Post layer normalization (Add & Norm)

We have already mentioned the problem of information loss in networks that apply multiple levels of processing to their input (deep networks). Ideally, we would like each layer to add some additional knowledge - information to the encoding it produces and not to lose the information that has resulted from the processing carried out in previous layers. As we have already noted in the paragraph discussing the LSTM networks, we have to create some sort of a mechanism of bypassing long chains of repeated activation functions (pass-through). For these purposes we usually use the so-called residual connections. The basic idea comes from the Taylor series. In this series we have a, possibly infinite sum that approximates the value of a function. The more terms added, the more accurate the approximation.

Looking at the Transformer's architecture in Picture 6 we find that residual connections appear after each processing stage. This is done not only to reduce information loss as the network produces increasingly complex encodings of information, but also to avoid the problem of vanishing and exploding gradients, contributing to the stabilization of the training process. In general, this is a relatively simple but very effective mechanism that is widely used by modern architectures.

Also notice the existence of a layer named "add and norm" as shown in Picture 6. This is a normalization step before or after the addition of the tensors. In general, the location of the normalization layer inside the network's architecture is a matter of negotiation and different solutions have been proposed for its exact location (Glassner, Deep Learning: A Visual Approach 2021). Typically, the results of the sub-layer's output are added to its input and then the resulting output is normalized. The important thing is that residual connections are usually followed by an add norm (addition + normalization) or norm add stage. In conclusion, the normalization layer is important for reducing the risk of information loss and to better stabilize the training process.

1.2.5.5 Feed forward layer

After the attention layer and the add & norm layer, the so-called feed forward layer follows. We will not provide a detailed discussion about this specific layer and will refer the reader to the literature for more information. We would just like to mention that this layer usually consists of two 1×1 convolutions whose purpose is to reduce redundancy in the information gathered by the processing layers, making it easier for the network to focus on what is most important (Glassner, Deep Learning: A Visual Approach 2021).

1.2.5.6 Multi head attention

Usually, each Attention layer is divided into the so-called Heads. This means that the process described above and illustrated in the images we provided will be carried out for each of the Attention Heads. After all the Attention Heads are calculated, the final result of the Attention layer will be the concatenation of all the Heads.

For example, in the basic version of the Transformer we have the following: The encoding size is 512. There are also 8 Heads, each of which will produce a sequence of size $(n, (512/8))$. Each of the 8 Heads

will therefore contribute 64 values to the final Attention result. Then, the concatenation of all the Heads will be performed and the result will be a sequence of which each element will have a size of 512. The purpose of attention heads is to allow the network to produce associations of a different nature in each head. For example, one Head may focus on semantic connections, another may focus on syntax, etc. if we process, for example, a text in a natural language. In a way, the Attention Heads could be likened to the filters we have in a Convolutional layer.

1.2.5.7 Masking

Now let's turn our attention to the decoder. As we see in the image with the basic architecture of the Transformer, in the decoder layer we have a self-attention stage followed by a cross-attention stage where the queries come from the decoder's input sequence and the keys and values from the encoder's output sequence. The question that arises is how to train the decoder when its input already contains the answers to whatever it is supposed to be predicting.

If we give it the outcome we want it to predict, the network will simply learn to return exactly what it got as input. Consider, for example, the case of the problem of translating a sentence from English to Greek. The input to the encoder will be the sentence in English. The decoder will receive the translation of the sentence into Greek as its input, but in such a way that it cannot "see" future words, that is, words that it has not yet predicted.

This means that in order to produce the prediction for the n^{th} word of the expected output, the decoder will only have access to keys and values derived from tokens that are positioned before the n^{th} token in the output sequence. The keys, in combination with the query, will produce the scores according to what we saw above. The scores associated with later keys (i.e. keys that relate to subsequent positions after position n) will all be zeroed out of the decoder's mask. Therefore, if a score becomes zero, the effect of the corresponding value on the query is canceled. In practice this is done by placing the $-\text{inf}$ value before the SoftMax stage in the score's calculation. If the value $-\text{inf}$ passes through the SoftMax layer we will naturally get zero as a result.

Therefore, at the input stage of the decoder, a mask is applied which hides the words or tokens that have not yet been predicted by the decoder. In this way, the decoder learns to predict the next element of the sequence based on the encoder's encoding and the predictions made so far.

Finally, note that the first element of the decoder's input should be a special token (usually denoted by $\langle \text{SOS} \rangle$ (start of sequence)) which signals the beginning of the sequence.

1.3 A note about the Transformer's training

In the previous paragraph we described the process by which the Transformer (in its most basic formulation) calculates its predictions. As we saw, in the decoder layer we will need some mask which will hide the elements that have not yet been predicted. We have two options to consider:

In the first case, the training is done in exactly the same way as the inference from the trained network. That is, we first input the special token $\langle \text{SOS} \rangle$ to the network, then $\langle \text{SOS} \rangle$, pred_1 where pred_1 is the previous prediction, etc. Therefore, n forward passes will be needed.

The second option is to pass the entire decoder's input sequence directly to the decoder network with the first element being <SOS>. That is: [<SOS>, token_1, token_2, ... token_n-1], where n is the number of predictions we want to obtain, and token_1, 2 etc. are the actual values (not the network's predictions in previous forward passes). If we want to train the network to predict, say, the m^{th} token of the predictions' sequence, we use a mask to hide tokens $m - 1$ to $n-1$ (The last token is always hidden). Again, note that the tokens preceding the position we want to predict are derived directly from the real dataset values not the previous predictions. This way we will have only one forward pass for each batch.

In practice, the second option is usually used to train the network, and this is done for the following reasons: The first reason has to do with the fact that, in the first case, the network error will tend to accumulate in the later predictions. Also, in the first option, many more forward passes will be needed for each batch. Finally, the computational graph required for the back propagation process will be particularly extensive, leading to a very large memory footprint.

There are of course variations of this process. Therefore, not all networks based on the transformer architecture are trained and nor do they predict in the same way. In the case of the Informer architecture, for example, which we will study next (Zhou 2021), we can avoid multiple forward passes for obtaining the predictions (inference), using a trick that will be described next. Here we will simply mention that the basic idea is to use a small preamble at the input of the decoder to start the prediction process.

Part B: Introduction to the subject of this thesis.

2. Forecasting the energy production of Renewable Energy Sources parks

Following the introduction to the basic concepts which we will be using, we come to this point in the discussion which concerns more specifically the main subject of this work, which is the prediction of the amount of energy produced by RES parks (e.g. wind farms, etc.)

2.1 General overview

Renewable energy production (RES) is undoubtedly an important factor in the energy mix of most developed economies. According to the Eurostat, in 2021 the energy produced from renewable energy sources accounted for 21.8% of the total consumption in the European Union. (Eurostat 2023). The changes brought about in recent years by the liberalization of the energy market, with the institutionalization of various energy exchanges and with the related financial derivatives, have brought to the fore the need for better forecasts of both energy production and consumption.

Unlike other goods, energy cannot be stored long-term for sale at a later time. With existing technology, storing large amounts of energy can only be done in limited, and relatively short, time periods. This fact makes it necessary to seek methods for predicting the production and consumption since, ideally, we would like all the energy produced to be able to be sold immediately or almost immediately since the possibilities for storing the energy are limited for various technical and economic reasons.

Furthermore, accurate forecasting of energy production and consumption is of great importance in designing an investment portfolio and minimizing investment risk. In addition, such forecasts are also important for the effective integration of RES into the electricity grid. For example, in the work by (J. Schmidt 2013) where an econometric model for portfolio selection under the influence of different tariffs is presented, the researchers highlight the importance of forecasts for choosing the degree of portfolio diversification.

In this thesis we will study the problem of predicting the electricity production of wind farms. The dataset we will use concerns the energy production from ten wind farms which, due to their geographical proximity, may influence each other in terms of their energy output.

We are mainly interested in studying the possibilities offered by neural network architectures based on transformers and the attention mechanism for producing rich spatio-temporal associations. We will also focus our attention on techniques based on the so-called Diffusion. (Ho 2020). Therefore, we are primarily interested, having the GEFCOM 2014 dataset (which will be described below) as our main vehicle, to study and experiment with the possibilities offered by these architectures.

3. Previous work on the subject

Several different strategies have been proposed for predicting electricity production from RES parks and for related problems. It is clear that forecasting energy production from RES parks is a fairly specific problem and therefore, to somewhat broaden the scope of the discussion, we will also examine works that study related problems such as the study of time series related to road traffic, etc. The techniques are essentially very similar, although, in practice each problem presents separate and particular problems, depending on the form of the available data.

The options available to us to address such problems are wide-ranging: For example, from the use of purely statistical methods, to the use of neural networks or even a combination of techniques. In recent years, as expected, interest has begun to turn to techniques using purely neural networks or even combinatorial techniques that utilize both neural network architectures and statistical techniques.

The work by Jiaan et al. (Jiaan Zhang 2021) for example, adopts a hybrid method for forecasting energy production from wind farms. In other words, it is based on both statistical indicators and the training of a neural network. More specifically, for each moment in time a matrix called "spatiotemporal correlation characteristic matrix" is constructed. This table is constructed using an arrangement of various statistical correlation indices which are weighted using the Shapley method. Then a network with convolutional layers is used whose output (the feature maps) will be fed to an LSTM network.

A problem with this method is the selection of the so-called "standard wind farm", as defined in this publication, in order to calculate the correlation matrix mentioned above. This is done to reduce the amount of computation and is a reasonable choice, but in any case, it may reduce the information available to the network. Also, the specific model proposed in this publication forecasts the energy output of all the parks in the cluster as a whole and not individually. Finally, the convolutional layers, especially if no attention mechanism is used, can detect relatively local (and not wider) spatio-temporal correlations, which will be determined by the size of the filters, the stride, etc. However, perhaps this relative locality of correlations is a reasonable assumption for the datasets used. According to the researchers, the method showed satisfactory results.

In a paper by (Huijuan Wua 2022) an attempt is made to tackle the problem of wind speed prediction. No attempt is made here to predict the energy production of renewable energy parks, but the subject matter is relevant. The authors of the paper used the ensemble empirical mode decomposition (EEMD) method to decompose the signal (wind speed) into intrinsic mode functions (IMFs). They then feed the new time series, now larger than the original, into a Transformer network. This is an interesting idea showing good results, as the researchers claim. In this paper, the wind speed at only a single location is considered and therefore any knowledge that would arise from cross-correlations between different locations is not used.

Another approach to the problem of forecasting energy production from renewables, with the pure use of neural networks is presented in the work by: (Lei Wang 2022). In this work we have an approach based on the use of multiple Transformer networks in a parallel arrangement. Each of these Transformers processes a sequence that includes only one of the features (columns) of the time series (e.g. temperature, wind speed, etc.). Each Transformer will therefore specialize in processing a particular aspect of the time series. Then, the outputs of all these "parallel" Transformers are fed to a fully

connected layer named Feature Fusion Layer which undertakes the combination of the information captured by all the Transformers from the previous layer (feature extractors).

The idea of this parallel arrangement of Transformers is interesting. As a simple thought about this architecture, we will mention that, although the usage of an arrangement of multiple 'parallel' Transformers reduces some of the problems associated with the computational complexity of finding rich spatio-temporal correlations, it may limit the propagation of some of the spatiotemporal correlations that appear between different features of the time series since this task will be undertaken, not by the attention mechanism, but by some fully connected layer. This is something that may need to be investigated.

Fabio Lima et al. (Fabio Lima 2022) proposed a model for forecasting the energy production from wind farms which is based on 3 stages of processing. Initially, the K-Means algorithm is used for the clustering of the wind farms. Then the proposed model will process each group of wind farms separately. In the second stage of processing, the time series data for each of the clusters detected by K-Means will be processed by self-attention layers. In the third stage of the model, the output of the attention layer is fed to a sequence-to-sequence network using an RNN (Recurrent Neural Network).

Using a clustering algorithm is a logical proposition. One comment we could make concerns the use of the K-Means algorithm. This particular algorithm will face problems in case the groups are anisotropically distributed. There are of course many algorithms that can be used alternatively such as e.g. the spectral clustering algorithm which addresses the anisotropy issue to some extent. However, in general the clustering problem is quite a delicate one and the choice of appropriate algorithms should be carefully evaluated. Regarding the use of the self-attention layer, the structure of the input data here is particularly important. The attention mechanism, in its basic version at least, can calculate scores for temporal correlations only. However, it might be possible that we are also interested in spatial correlations. Later in this thesis, we will consider some of the ways we can achieve this.

In the paper by: (Inkit Padhi 2021) the researchers studied the problem of predicting fraudulent transactions and the problem of predicting several air pollution indices. Both of these cases show significant relevance to the prediction of energy production, etc. as the format of the datasets is similar. Notably, we have tabular data to work with, since for each moment in time we have multiple columns.

The researchers used the BERT and GPT architectures to address the problems of classification, regression, and synthetic data generation. In particular, a network that the researchers call Field Transformer was used to create an embedding for each line of the dataset. This particular layer takes on the task of finding local correlations between the columns. The Field Transformer will find local correlations concerning the columns of a single transaction (intra-transaction relationships), regarding e.g. the dataset for the study of the fraud detection problem. There are problems where finding correlations between features related to different points in time may be useful. However, the proposed network is of particular interest as BERT is an architecture that can be trained to solve a variety of different problems such as the problem that in language learning is sometimes called the cloze task as well as the problem of classification. Regarding the issue of synthetic data generation, the researchers suggest using the GPT architecture. To generate synthetic data for the dataset which is derived from credit or debit card transaction logs, the researchers train a separate GPT network for each user. Networks based on the GPT architecture are very powerful tools, but care must be taken as its training can present difficulties due to the large size of the network.

In the problem of road traffic forecasting the work by: (Chenhan Zhang 2019) presents a technique inspired by considering the time series of the speeds recorded by motion sensors as a graph. The authors used the attention mechanism to construct an adjacency matrix that will help the later layer of LSTM networks.

Regarding the interpretation of the data in graph format, it should be noted that it has also been proposed by: (Vijay Prakash Dwivedi 2020), a variant of the Transformer architecture which, among other things, modifies the positional encoding mechanism so that its calculation is now done using techniques applied in algorithms such as the spectral clustering algorithm, which is based on the construction of the so-called 'graph Laplacian' matrix. This is a logical solution. Note that when calculating the distances between the elements of a graph we may have to resort to specialized ways of calculating the distance between observations as the graph space may follow its own geometry which may not be Euclidean. You can see more details in the original paper as well as in the book by: (Uday Kamath 2022).

An interesting idea, which we will discuss in more detail in the following paragraphs, concerns the use of the Informer network (a variant of the Transformer architecture) for calculating rich spatio-temporal correlations in multivariate time series. In the paper by: Grigsby et al. (J Grigsby 2021) an architecture is proposed that the researchers called Spacetimeformer. Essentially, this architecture is more about the manipulation of the form of the network's input. The underlying mechanisms remain basically the same. We will later discuss more details about the method proposed in this publication. Here we will simply give a general picture.

The researchers' basic idea is to use networks based on the attention mechanism to produce rich correlations in multivariate time series. The goal is to produce attention scores that not only concern the correlation of one entire observation (all features-columns) with another entire observation with a different timestamp, but also a column (feature in the dataset) of one observation with another column of another observation with different timestamp. In this way, the production of rich correlations is achieved that are not limited to just one axis (the timestamps axis).

A 'side effect' of such a modification is the fact that the number of attention scores will potentially be vastly expanded. To address this problem, the researchers resorted to using a variant of the Transformer called the Informer. (Zhou 2021). In the Informer architecture, the complexity of the attention layer is logarithmic instead of quadratic, allowing for the processing of larger sequences. Note that, besides the Informer's architecture, many variations of the Transformer and the attention mechanism have been proposed that address, to some extent, the issue of the complexity of calculating the attention matrix. In the following paragraphs we will provide more details about this.

4. The datasets

For our experiments we will rely on the dataset provided in: GEFCOM 2014 (T. Hong 2016) and also in: AEMO (Australian Electricity Market Operator) (J. Dowell n.d.).

The GEFCOM dataset was presented by Tao Hong in the context of the creation of a competition to predict various tasks related to the production of electricity from different sources. In the GEFCOM 2014 dataset various tasks are defined, each of which has its own data set. Regarding our own experimentation, we will deal with the data concerning wind farms. To achieve the richest possible dataset, we have combined data from different tasks (related to wind farms) as defined in GEFCOM 2014. Therefore, we will not only rely on the data of a specific task, but will combine data from multiple different tasks. As a result, we have a dataset regarding the electricity production for each of ten wind farms, which are assumed to be in some kind of geographical proximity, which means that there may be correlations between the farms in terms of the model predictions for the energy output of each farm.

Let's now look more specifically at the features (columns) of the dataset. For each of the ten parks we have the following information:

- Timestamp: The time and date for the specific observation.
- TARGETVAR: The electric energy output of a specific wind farm (one of 10). The target that we want to predict.
- U10: In meteorology, wind is usually broken down into two components which, by convention, are usually called u and v. We can calculate the wind's speed using the L2 norm: $s = \sqrt{u^2 + v^2}$. Here, U10 refers to the u component of the wind measured at an altitude of 10 meters.
- V10: It refers to the v component of the wind measured at an altitude of 10 meters.
- U100: Similarly to the above description, but for a different altitude.
- V100: Similarly to the above description, but for a different altitude.

It should be noted that in the original dataset there is also another column, ZONEID. This column takes a value of 1 – 10 depending on the park. Each park corresponds to a ZONEID. For our experiments we will skip this column and instead the information about the ID of each park will be derived from the positional encoding, as we will see below.

Therefore, our dataset will have a total of 50 features plus the timestamp: (targetvar, u10, v10, u100, v100) times ten parks plus the timestamps for each park. The timestamp column, as we will see next, will not be passed to the features of the network input since it will be integrated into the embedding of each token.

Regarding the completeness of the dataset, we have to mention that there were some missing values. In these cases, we filled the corresponding fields with zeros.

The total number of observations corresponds to 17538 hours. Observations extend from 1/1/2012 at 1:00 a.m. to 12/31/2013 at 6:00 p.m.

As far as the AEMO dataset is concerned, we have data available for the years 2012-2013 which concern energy production forecasts as well as weather forecasts provided to us at five-minute intervals. The dataset concerns wind farms located in south-eastern Australia. We have at our disposal 22 farms and 9 different weather forecasts. From this dataset, we kept 10 of the total 22 parks for our experiments* and 4 of the 9 weather forecast items (wind_direction_10,100m and wind_speed_10,100m). This was done to

* the 10 parks selected are: CAPTL_WF, CATHROCK, CULLRGWF, LKBONNY1, MTMILLAR, STARHLWF, WAUBRAWF, WOOLNTH1, WPWF, YAMBUKWF

somewhat reduce the memory footprint of the network and also to achieve a schema for our data similar to the GEFCOM dataset. It is also noted that for our experiments we considered hourly observations only, by taking the average of the 5-minute observations for each hour. This way, we ended up with a total of 17540 hours (observations) for the AEMO dataset.

As we will see in alter paragraphs, for the type of architectures we are interested in studying, i.e. Transformers, etc., such a dataset may, in some cases, not be large enough. This is a problem that can cause several side effects with the main one being overfitting. When working with data collected from wind farms it is difficult to find significantly more extensive datasets. We will discuss this issue in more detail in the following paragraphs where we give a more detailed description of our experiments.

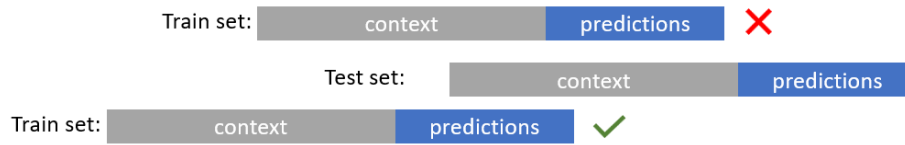
As we'll see next, there are some decisions we need to make about how we handle our data. Specifically, a question arises about the size of each token (number of features) in the sequence. For each point in time, we have 50 columns in the dataset, 5 features for each of the ten parks. One option is to construct each token in the sequence to have 50 features, placing all the columns of the dataset for a single timestamp in one token (5 times ten features for each token). Another option is to divide these groups of 50 columns for each timestamp into smaller sub-units to achieve richer associations in the attention layer. We will be discussing this issue in more detail in the following paragraphs.

4.1 The construction of the data loaders

At this point we should make a brief comment regarding the construction of the training batches from our dataset. Let's consider the GEFCOM dataset (the situation is similar for AEMO). We usually want to take a random selection of the training samples. For our experiments we are going to give access to the network to a window of 18 hours into the past and expect a prediction of 4 hours into the future, predicting only our target (power output for each of the 10 wind parks, we do not predict the meteorological data). In the case of the time series we are studying, this means that we have to choose a random number between 1 and $17538 - (18 + 4)$ and return the corresponding slice of the time series which starts from the random index we got. We have to sample 2 random subsets one for the training and one for the testing of our models. Let's assume, for example, that we will be having 800 sequences (of size $18 + 4$ tokens) for the training set. Thus, we will be having $800 / 16 = 50$ batches if each training batch is of size 16.

But there is a problem we have to address. For our discussion we will be using the word 'context' for the window of past data (meteorological predictions plus target outputs) given to the network in order for it to predict the future target data (power outputs for the 10 wind parks). It is important that we exclude from the training set all sequences that overlap with sequences from the test set in the 'context' region (the context has a length of 18 tokens in our case). This is necessary because, otherwise, during its training, our network will have partial access to sequences which are supposed to be inaccessible (these sequences are partially included in the test set). If the overlap occurs between a) the 'context' region for the test set and b) the predictions region for the training data, then there is no problem since the network will be called upon to predict the overlapping elements during its training. The should, however,

be no overlap between a) the ‘context’ region of the training sample and b) any region of a test set sample.



Picture 12 The construction of the training samples

Consider following the above sampling method while also randomly selecting the index for the first hour of the ‘context’ region for each training sample. This method will dramatically reduce the size of the training set, and we have to keep in mind that the dataset is already quite small. For example, if we want to have just 800 sample sequences for the test set, then the training set will only have an estimated 2597 sample sequences available to it, out of 17538 hours of total hourly observations. Because the dataset is already quite small, (about 2 years of hourly observations), we will be using another sampling method for our experiments (i.e. method for selecting the index of the first hour of the context region in the training set samples). In general, the sampling method is an open issue and is subject to improvements depending on the data we have available.

To make the most of the dataset, in the sense of having a large number of training examples, we will take the n consecutive hours for the training set and the remaining ones for the validation and test sets. This way, overlaps will be as minimal as possible since the hours will be consecutive. The disadvantage of this method is that the control data will not have exactly the same distribution as the training data, i.e. there will be no exact stratification.

5. Introductory elements on the proposed methods

Regarding the proposed methodology for predicting the energy output of the wind farms, we will work on 2 main axes: One axis of our experimentation is related to the use of the Informer architecture and the so-called spatio-temporal attention (J Grigsby 2021) (Zhou 2021). The other case that we will examine concerns the use of techniques based on the so-called diffusion and we will rely on the interesting work by (Tashiro 2021). We will therefore successively examine these two strategies, starting with the case of the techniques based on the spatio-temporal attention and the Informer architecture.

Part C: First proposed method

6. Basic overview of the first proposed method (spatio-temporal attention, Informer, sliding windows)

The first method we will examine concerns the use of a technique named 'spatio-temporal attention' (J Grigsby 2021) while also utilizing the Informer's architecture (Zhou 2021). We have already made a brief reference to these techniques and to the respective publications. In this part of the thesis, we will provide a more detailed introduction to the background of the architectures of concern in order to clarify the terms that we will be using as much as possible, and then we will describe our own experimentations in detail.

It might be useful to start by giving a brief description of the basic idea behind the so-called 'spatio-temporal' correlations:

In the classic case where we have to process the words in a sentence, the attention layer (at least in its vanilla (basic) version) of a Transformer network will calculate a score for each query and key pair. Typically, the attention layer score is calculated using some variation of the calculation of the inner product of two vectors. Because the inner product, in rough terms, is related to the concept of distance and similarity between vectors, this means that the correlation or similarity between pairs of words in a sentence (for example) will be reflected in the attention scores. In the example of processing the sentences in a natural language, the sequence of words in a sentence, as far as the attention layer is concerned, can be considered a one-dimensional sequence. That is, the sequence has length n on the positional axis, and 1 on the 'spatial' axis, since for each of the n positions of the sequence we have only one word.

Now consider the case where, instead of words in a sentence, we have a multivariate time series where for each point in time we have not only one but many 'columns' (or features) in the dataset. In this case, we will be having a time axis (determined by the timestamp of each observation), and a 'spatial' axis (determined by the column (feature) of a row (timestamp) of the dataset).

In the paper by: Grigsby et al. a modification of the input to the encoder and decoder of a Transformer network is proposed, so that the attention scores that will be produced concern not only the temporal but also the 'spatial' axis (that is, the feature axis). The researchers' main idea was to modify the way the transformer layers are fed their input. That is, instead of calculating an embedding for each of the observations (tokens) of the time series, and therefore resulting in a token for each timestamp, we now have m tokens for each timestamp where m is the number of features (columns) for each observation of the time series.

In this thesis, we were faced, among other things, with the following issue: In the case of studying time series concerning the forecast of energy production from wind farms, in addition to the target, which is the energy production, we also have at our disposal a weather forecast time window. These weather forecasts concern future times points for which we want to predict the energy production. Therefore, we

must, somehow, give the Transformer (or the Informer, which is a variant of it) this knowledge regarding weather forecasts for the wind speeds, etc., which concerns future points in time.

One way to incorporate this knowledge for the future is to appropriately modify the input of the network's decoder layer so that the cross-attention layer can now calculate scores that relate the future weather forecasts to past observations (either the target or the weather forecasts regarding past timestamps) of the time series. In later paragraphs, we will be discussing in more detail about the modifications that will be needed to the network to achieve this.

We will also discuss how the total size of the dataset has some influence to the overall design. We will also be discussing about the problem of over-fitting, but also to some possible solutions (ideas) for it.

7. Spatio-temporal attention

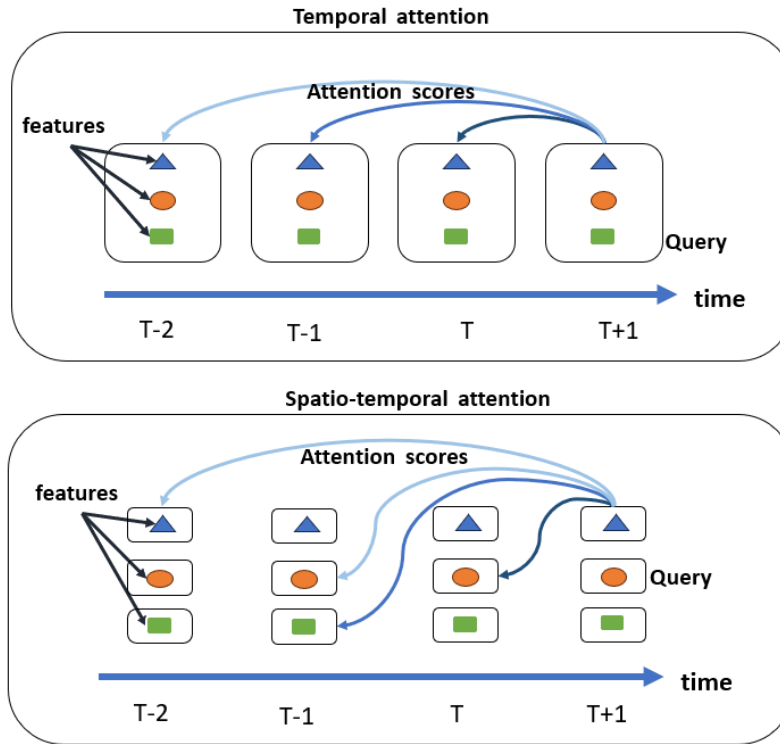
We have already given the definition and basic idea of spatio-temporal correlations in a previous paragraph. Here we will be discussing more specifically at the method followed in the work by (J Grigsby 2021).

An attention layer will produce a score for each query and key pair. It is noted that in the time series that we are interested in, we have multiple features (columns in the dataset) for a single timestamp. In the dataset that we will be using for our experiments, we have 10 parks, for each of which we have records for 5 features (energy production plus 4 weather forecasts for each timestamp). Therefore, there are 50 features for each timestamp of our sequence. In the simplest case, we can pass these groups of 50 features through an embedding layer and produce a single token of specific dimensions for each timestamp. For example, in the simplest case of a vanilla transformer for processing natural language sentences, each word is converted into an embedding of dimension $d = 512$. We could therefore convert these 50 features into an embedding of size e.g. 512. The attention layer in this case, will have to calculate $n \times n$ scores, where n is the number of timestamps (for the case of self-attention). Therefore, attention scores with this type of input will only take temporal associations and similarities into account (thus, will ignore all the direct intra-feature correlations). Thus, the intra-feature (spatial) correlations (e.g. feature n from timestamp a with feature m from timestamp b) are going to be, somehow, indirectly considered, if at all.

Furthermore, for the datasets we are examining, we must somehow incorporate the knowledge about weather forecasts that concern future timestamps. If we choose to pass the entire dataset lines (50 features for each timestamp (dataset line)) through an embedding layer, we will need to modify the network to accept 2 sequences as input. One will concern embeddings from entire rows of the dataset, and the other will concern only the weather forecast data. In such a case where we have 2 sequences as input, a cross-attention layer or some other mechanism will be needed to combine the information from the 2 different sequences. This is necessary because in this case we cannot apply a mask to the decoder's self-attention to hide future energy productions (targets) and simultaneously make available all future weather forecasts, since these two elements will be embedded within the same token.

Previously, we mentioned the option where the entire rows of the dataset (50 features) pass through an embedding layer. This may not be sufficient in some cases. It may be necessary, for example, to find similarities (attention scores) between the 12th (out of 50) feature of one observation and the 8th

feature of another observation at a different time. The case described in the previous paragraph will not record such similarities, as it works only in the context of temporal and not spatial correlations. This kind of richer similarity relationships between features (columns), and not only between entire rows of the dataset is called spatio-temporal attention (see: Grigsby et al.). Picture 13 visualizes the above description:



Picture 13 Spatio-temporal attention. Based on: (J Grigsby 2021)

Therefore, the researchers propose expanding the sequence given to the input of the encoder and decoder so that each token is derived from the embedding of a single feature and not from embedding all the features of the observation (dataset row) in a single token. This will also raise other issues such as the handling of the positional encoding, for example, which now needs to be modified appropriately. We will see more details about this later.

8. The problem of attention layer's complexity

In essence, Grigsby et al. propose extending the sequence so that each of the features of the dataset is encoded as a separate token of the encoder and decoder input. In this case, the total number of scores that the encoder's and decoder's attention layers will have to calculate will amount to $(\text{number of timestamps} \cdot \text{number of features})^2$. In the case of the simplest mechanism using vanilla attention, the scores will be equal to the square of the number of tokens (assuming each token corresponds to a single, unique timestamp).

It is clear that such an approach will produce richer scores in the attention layer but will also create other important problems. Training such a network will result in very extended gradient tapes and the complexity of the attention mechanism will start to become a burden. It will not be easy from a computational point of view to train a network to produce such a large number of attention scores.

One of the major problems with vanilla attention (the basic version) is the quadratic complexity of calculating the scores. Consequently, if our problem requires the processing of very long sequences, the network will need increased computational resources to be trained. Several solutions have been proposed to address this problem. Below we list some of the variations of the Transformer architecture that allow the processing of larger sequences by effectively reducing the complexity of the attention layer.

9. Variations for the attention mechanism

Beltagy et al. have proposed (Iz Beltagy 2020) a variant of the Transformer architecture called Longformer. The idea is to make the attention scores table sparser by making some assumptions about its structure. For example, we may want the attention layer to only emphasize positions that are close to the query in question. In this case we have a window that will define the number of positions before and after the query for which the scores will be calculated. Thus, the structure of the table of attention scores will look something like what is shown in Picture 14 (Uday Kamath 2022):

•	•					
•	•	•				
	•	•	•			
		•	•	•		
			•	•	•	
				•	•	•
					•	•

Picture 14 Longformer with a window of size $w=1$

That is, the calculation of only those scores related to keys which are positioned within a certain distance from the query is selected. This method is called sliding windows. Of course, many variations of this pattern can be made depending on the structure of the problem we are working on. This is a good idea, but it requires some, more or less, arbitrary assumptions to be made.

In the book by (Uday Kamath 2022) other variants of the Transformer can also be found. We would like to mention the cases of the Reformer (Nikita Kitaev 2020), Performer (K. CHROMANSKI 2021) and Big Bird (M. ZAHEER 2020).

As for the Reformer, it is a complex and very interesting new method. The basic idea is to hash the sequence of queries and keys into buckets using the Locality-sensitive hashing (LSE) method. The full

attention is only calculated within the context of a bucket and not across the entire sequence. Buckets are built so that they contain items that are similar. Thus, the hash function will 'route' the query to the bucket that contains the most important keys. In addition to this improvement, the researchers propose other modifications to the basic transformer architecture involving a more efficient utilization of the memory through the use of the so-called Reversible residual layers. This is indeed a very interesting publication. You can refer to the original publication and related references for more details.

10. Informer

The architecture of the Informer will be studied a bit more extensively because of its importance for our experimentation with the PyTorch code, etc. Note that in the paper by: (J Grigsby 2021), on which we partly relied, the researchers use the Informer architecture (Zhou 2021) to address the computational complexity of the application of spatio-temporal attention layers using the vanilla form of the attention mechanism. The use of this specific architecture was deemed appropriate as the Informer architecture can be used for small to large sequences, while according to (Zhou 2021) other architectures, such as the Reformer, are optimized for very large sequences, up to 1 million elements. However, the use of alternative architectures, including the Reformer, which seems to have significant advantages, is an open issue that can be considered when dealing with long sequences.

The use of a variant of Transformer that uses a computationally faster mechanism for the calculation of the attention scores, arises from the need to process large time series. These larger timeseries have resulted from the expansion of the original time series in the manner that will be described in the next paragraph. Here we will simply mention that each token of the sequence will correspond to a feature. That is, instead of having, for example, a sequence of the type: t_1, t_2, t_3 etc. we are going to have: $f_{1_1}, f_{2_1}, f_{3_1}, f_{1_2}, f_{2_2}, f_{3_2}, f_{1_3}, f_{2_3}, f_{3_3}$ etc.

The researchers who proposed the Informer architecture attempted to address three problems:

- The quadratic complexity of vanilla attention
- The memory bottleneck in Transformers with many layers
- The speed reduction in the prediction stage, which is caused by the mask mechanism in the decoder layer

Regarding the first two problems, their cause is clear and we have introduced a relevant discussion in previous chapters. As for the problem of reduced speed for calculating predictions, it is sufficient to consider the masking mechanism used by the decoder layer of a Transformer. In a typical implementation (we are talking about the inference not the training stage), we have an iterative loop where, at each step we assign the decoder to predict the next element of the sequence based on its own predictions for the preceding tokens. This iterative loop will run slower, as it will likely require the use of the CPU which is lagging behind in speed, due to the fact that the GPU and other specialized machines are specifically designed to perform massive matrix computations in batches.

More specifically, regarding Informer, the basic idea is based on an alternative definition of the attention scores given in a paper by: (Yao-Hung Hubert Tsai 2019). This paper defines attention in the light of the concept of a kernel function. Yao-Hung et al. define Attention as follows:

$$\text{Attention}(x_q; M(x_q, S_{x_k})) = \sum \frac{k(x_q, x_k)}{\sum k(x_q, x'_k)} v(x_k) = \mathbb{E}_{p(x_k | x_q)}[v(x_k)] \text{ όπου: } x_k, x'_k \in M(x_q, S_{x_k}),$$

$$p(x_k | x_q) = \frac{k(x_q, x_k)}{\sum_{x'_k \in M(x_q, S_{x_k})} k(x_q, x'_k)},$$

και $k(x_q, x_k) = \exp(x_q x_k^T / \sqrt{d})$ (Asymmetric exponential kernel)

Statement 9 Alternative definition of Attention using the concept of asymmetric exponential kernel (Yao-Hung Hubert Tsai 2019).

Where: d is the dimensionality of the keys vector (Yao-Hung Hubert Tsai 2019) (see. Definition 1 in the original paper).

As for $M(.,.)$ it represents the mask that we may have in the decoder's cross attention layer. S_{x_k} is the set of keys. Also, v is a function representing the 'value' layer. And lastly, x_q, x_k are the query and value tokens of the sequence respectively.

Zhou et al. (Zhou 2021) then proceed to define a metric to evaluate the importance of each query. Their goal is to construct a sparse matrix $\bar{Q}$ which will, theoretically, include only the Top- u most important queries. The attention scores will be calculated only for the queries defined in this table.

The proposed metric for evaluating the significance of a query token is called sparsity measurement. Recall that, in the paper by (Yao-Hung Hubert Tsai 2019) the definition of a conditional probability arose from the alternative (but theoretically equivalent) definition of the Attention: Specifically we have: $p(k_j | q_i)$ where k_j, q_i is a key and query respectively. As observed by Zhou et al. : The dominant inner products (query-key pairs) encourage the query probability distribution to move away from the uniform distribution. (Zhou 2021). This formulation reminds us somewhat of the concept of entropy which is used e.g. in decision trees etc. Based on this formulation, researchers use the Kullback-Leibler method (KL divergence) to define a metric that will evaluate the importance of queries.: Specifically, if $p(k_j | q_i)$ is closely resembles the uniform distribution $1/L_K$ then the calculation of attention is trivial and the query is not expected to contribute significantly to the network's predictions. Therefore, the metric proposed by the researchers is based on the use of KL-divergence to assess how close this distribution is to uniform. After some improvements in the mathematical formulation, which are related to the numerical stability of the calculations, the researchers arrived at:

$$\bar{M}(q_i, K) = \max_j \left\{ \frac{q_i k_j^T}{\sqrt{d}} \right\} - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}}$$

Statement 9 Sparsity measurement

(Zhou 2021)

Where: L_K is the length of the keys' sequence and K represents the set of the keys.

According to Zhou et al. (Zhou 2021), it is sufficient to use a random subset of the keys, no larger than $U = L_K \ln L_Q$ to calculate the estimator $\bar{M}(\mathbf{q}_i, \mathbf{K})$ which will give us an estimation of how important the query $\mathbf{q}_i$ is. Then, the attention scores are calculated as usual, using the formula: $(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})$, but now using the table $\bar{\mathbf{Q}}$ instead of $\mathbf{Q}$, which will include only the top- u queries, i.e. the u most important ones based on the metric $\bar{M}$. Regarding u , (in top- u) it is defined by a parameter c which the researchers name the sampling factor, and which can be specified by the user. Thus, u is defined as: $u = c \cdot \ln L_Q$. Therefore, this architecture implies a complexity of the order of $\mathcal{O}(L \ln L)$, a clear improvement compared to the $\mathcal{O}(L^2)$ of vanilla attention.

This variation of attention that we described above is called by Zhou et al. (Zhou 2021) ProbSparse attention. In addition to the ProbSparse attention mechanism, Informer uses 1D convolutional layers, after the ProbSparse attention output, reducing the length of the sequence with each convolutional layer. How much the length of the sequence will be reduced is also something that can be dialed as a control parameter by the user.

At the level of the decoder, however, such convolutional layers are not used. More specifically, in the decoder layer, the researchers use the ProbSparse attention only for the self-attention layer. In the cross-attention layer, the full attention is used in its vanilla version. This makes sense, as after several successive passes of the sequence through the encoder layers, the convolutional layers will have significantly reduced the length of the sequence. Therefore, to avoid a significant loss of information, it has been determined to be beneficial to use the basic version of attention (full attention) for the cross-attention layer of the decoder.

Still, there are several additional issues we have to discuss, concerning the Informer's architecture mainly related to the way the predictions are created using a specialized mask in the decoder. We will study this issue in more detail later. Here we are interested in giving a basic understanding of how Informer works.

11. Detailed description of the proposed methods

11.1 Introduction

In a previous paragraph we gave an overview of the main axis of our experiments. In this part of the thesis, we will attempt to build a model based on the Spacetimeformer and Informer architectures utilizing the so-called spatio-temporal attention (J Grigsby 2021).

In addition, we will consider an alternative option based on the use of Diffusion models (Ho 2020) (Tashiro 2021).

For our experiments we have to somehow modify these architectures to be able to incorporate the weather forecasting information for future timestamps.

Regarding the predictions, note that we have a number of wind farms (10 wind farms) and we want to predict the hourly energy output for every single one.

11.2 Implementation topics

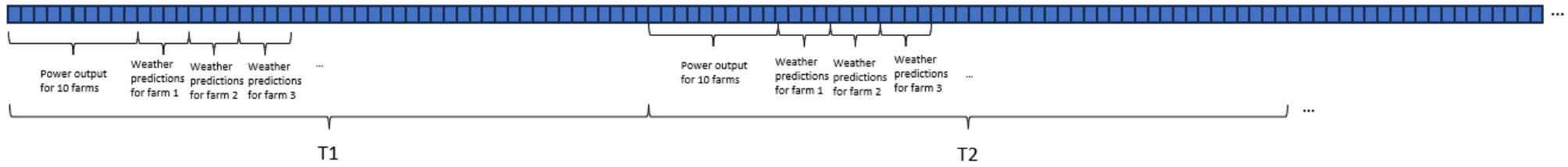
For the part of our experiments regarding the spatio-temporal attention etc., we relied on the software repository provided by the researchers who developed Informer (zhouhaoyi 2020). The basic idea we want to test concerns the modification of the architecture proposed in (J Grigsby 2021) in order to adapt it to the problem of forecasting the energy production of a cluster of wind farms. In this case we will also have at our disposal wind forecasts concerning future timestamps. So, we need to incorporate this knowledge into our model.

As we mentioned in a previous paragraph, a network based on the Transformer architecture will take two inputs (during training): one input to the encoder and one input to the decoder. Typically, the input to the encoder only takes into account the part of the sequence concerning the past, while the input to the decoder only concerns future moments in time i.e. related to anything we want the network to predict. It is therefore possible to place the weather forecasts for the target timestamps in the sequence that the decoder receives at its input.

11.2.1 The encoder input

The basic idea behind Spatio-temporal attention stems from the work by: (J Grigsby 2021). More specifically, we extend the input sequence by reconsidering the definition of a single token. Instead of encoding an entire line of the dataset (all features for a single timestamp) into a single token, we now encode each individual feature (column). Therefore, our sequence will now take the following form (regarding the Informer's encoder input sequence): (see Picture 15):

Picture 15 The encoder's input



Therefore we will have a sequence of the form:

$$\begin{aligned}
& TARGETVAR_{farm_1}, TARGETVAR_{farm_2} \dots, TARGETVAR_{farm_{10}}, \\
& U10_{farm_1}, V10_{farm_1}, U100_{farm_1}, V100_{farm_1}, \\
& U10_{farm_2}, V10_{farm_2}, U100_{farm_2}, V100_{farm_2}, U10_{farm_3}, \dots, V100_{farm_{10}}, \\
& TARGETVAR_{farm_1}, TARGETVAR_{farm_2}, \dots
\end{aligned}$$

etc.

In this way, the attention queries and keys will be drawn from a sequence that includes every single piece of information we have at our disposal. Thus, the attention layer will, hopefully, be considering not only the temporal relationships but also the so-called spatial relationships which are to be found between different columns (features) and, potentially, different rows (timestamps) of the dataset.

For the specific datasets we used, each row of the dataset will contribute a total of 50 tokens in the sequence. Therefore, for each timestamp of the dataset we will have 50 tokens at the input of the encoder. This sequence will give us a set of attention scores richer (theoretically) than if we only had temporal similarities and correlations.

In the vanilla version of attention, we generally assume that sequences will be of a relatively small to moderate length. In the classic example of processing sequences of words in a natural language, the only positional information that is meaningful simply concerns the position of the token within the sequence. In the case we are examining, however, we have a large sequence which is essentially organized into subunits.

One issue that arises from the organization we described above is that we will now need to modify the positional encoding as well as the embedding layer in order to incorporate this multi-level structure.

Therefore, for every 50 tokens of the sequence there should be an encoding that incorporates the information for a specific (the same for each of the 50 tokens) timestamp. In addition, we will also need to encode the position (index) of the token within the long (extended) sequence, just like in the vanilla version of positional encoding. The difference now is that a position is considered within the context of a single timestamp. Thus, the position (index) now concerns not the n th token but the n th timestamp (the index increases every 50th token).

In addition, we also have to somehow encode into each single token the index of the specific feature the token is related to (5 features for each timestamp). Furthermore, each token must be associated through an appropriate embedding with the wind farm to which it refers.

It is clear that our sequence will have a multi-level organization. To implement the embeddings and encodings for this multi-level organization, we will rely the code by: (zhouhaoyi 2020).

11.2.2 Embeddings και Positional encoding

Let's examine more specifically how we can achieve this multi-level organization in the embeddings and encodings of our sequence. Also, note that there are many ways to deal with the problem we described

in the above paragraphs. Here, we will examine a few different ideas but note that we can choose among many different methods, each of which is to be evaluated experimentally.

11.2.2.1 Temporal embedding

We will start with the temporal organization of our sequence. As we mentioned above, from the point of view of the temporal organization of the sequence, we essentially have two axes. The first axis concerns timestamps which include information about the month, day and time, and the second axis concerns the absolute position of the timestamp within the sequence.

Regarding timestamps, in the repository by (zhouhaoyi 2020) we find an interesting idea that utilizes the class `Embeddings` provided by PyTorch (PyTorch, PyTorch Documentation 2023). This class can be considered as a kind of look-up table. Essentially, the class `Embeddings` maps an index to a specific tensor. For example:

```
embedding = nn.Embedding(10, 8)
embedding(torch.tensor([2]))
'''
output:
tensor([[ 0.7150, -0.5313,  1.3762,  0.0237, -1.2320, -1.1563, -0.8640, -0.4942]],
      grad_fn=<EmbeddingBackward0>)
'''
```

As we see in the example above, `nn.Embeddings` converts the numbers 0 – 9 into 8-dimensional tensors. It is noted that the embeddings will be part of the trainable network (there can also be deterministic, non-trainable, variations). Therefore, the network will learn the appropriate representation during training. The class looks like a Linear layer. The main difference is that in the case of this class we have the indexes as the inputs and not a one-hot encoding (see: (machinelearningmastery 2023)) as we would have in an implementation using Linear layers.

Zhou et al. (zhouhaoyi 2020) apply the following method to embed the timestamp information into each token of the sequence:

Initially, they separate the dataset's features from the timestamps and then construct a tensor of dimensions (b,t,3), which is based on the timestamps, where b is the batch size, and t is the total number of tokens in the sequence. As for the number 3, it refers to the 3 dimensions implied in each timestamp (month, day, hour).

For the datasets we are studying we will need a somewhat coarse embedding for the timestamps. To explain what we mean, consider the case where instead of data on wind, we had data on daylight, e.g. energy production from photovoltaics etc. In such a case, it matters whether the time of the day is 10:00 or 11:00, for example. Therefore, we need a different embedding for each time of day. However, in the case we are examining, we have data regarding wind which does not require such a detailed embedding regarding the time of day. Therefore, we can divide the 24-hour day into 4-hour intervals and the days of the month into 4 groups (4 weeks). Specifically for the temporal embedding, we have the following PyTorch code:

```

...

self.hour_size = 4
self.day_size = 8
self.month_size = 12

Embed = nn.Embedding

self.hour_embed = Embed(24 // self.hour_size, d_model)
self.day_embed = Embed(31 // self.day_size + 1, d_model)
# + 1 because the month indices range from 1 to 12,
# otherwise we would get an out of range error.
self.month_embed = Embed(self.month_size + 1, d_model)

...

```

One issue is how to combine the tensors: `hour_embed`, `day_embed`, `month_embed` and embed them into the tokens of our sequence. A simple solution, which is derived from (zhouhaoyi 2020) is just adding it all together. Generally, in the most typical (and simplest) scenario, both embeddings and encodings can simply be added to the tokens.

Thus, in the forward pass for the temporal embedding we have:

```

...

hour_x = self.hour_embed(x[:,2])
day_x = self.day_embed(x[:,1])
month_x = self.month_embed(x[:,0])

return hour_x + day_x + month_x

...

```

Finally, all the embeddings and encodings will be added to obtain as a result a token of the sequence that will be fed to the inputs of the encoder and decoder.:

```

class DataEmbedding(nn.Module):

    ...

    def forward(self, x, x_mark):
        x = self.value_embedding(x) + self.position_embedding(x) + self.temporal_embedding(x_mark) +
self.farm_idx_embedding(x) + self.feature_embedding(x)

        return self.dropout(x)

```

In the previous paragraph we described the code corresponding to `temporal_embedding`. Next we will examine `position_embedding`, `farm_idx_embedding`, `value_embedding` and `feature_embedding`.

11.2.2.2 Positional encoding

On the occasion of the introduction of the function: `self.position_embedding(x)` it might be useful to make a brief comment regarding the naming conventions. Since we have found the names positional encoding and positional embedding to often refer to the same concepts we will need to make a distinction. When talking about embedding we usually refer to a learning process and not just some static mathematical calculation of a function. When referring to an encoding, we usually refer to the static calculation of a function that is not affected by backpropagation, etc. That is, this function is not part of the trainable network. We assume that the choice of the word 'embedding' in the part of the code of the repository (zhouhaoyi 2020) we are referring to, might not be the most indicative. However, this is a very small detail that does not affect anything other than this possible linguistic confusion. Probably a better name would be to use `position_encoding`, again noting that this is probably a trivial issue.

Let's now examine the necessary modification to the function `position_embedding`:

As is clear from the code by (zhouhaoyi 2020) the positional encoding has been declared not to contribute to the autograd's records (`pe.require_grad = False`). (see Code snippet 1 from the appendix). That is, it is not part of the trainable network. This is natural and expected.

Also note the usage of a variant of the basic type of calculation for the positional encoding using logarithms which was probably considered more appropriate for longer sequences (see: Zhou et al). However, the basic functionality of the positional encoding does not change.

We already mentioned the problem regarding the usage of the positional encoding for very long sequences. When having, possibly, thousands of tokens it is likely that the effect of positional encoding will 'fade out' too quickly for our purpose. In our experiments, the position (in the context of the positional encoding) of the token within the sequence will be defined as the absolute position of the timestamp with which the token is associated. That is, the position (in the context of the positional encoding) for all tokens within a 50-group (line in the dataset, where one timestamp of the dataset relates to 50 features) will be the same.

Regarding the implementation, you may find the corresponding excerpt from the code in the appendix in: Code snippet 2.

The idea behind positional encoding is that the similarity between distant tokens in terms of the position with which they have been encoded will decrease. Here we slightly modified the implementation to achieve the positional axis definition that is considered more suitable for our problem.

We can use the function `cosine_similarity` by scikit learn (scikit-learn 2023) to measure the cosine similarity between two different (positional) encodings of the same token. The similarity decreases a bit as the positions with which the tokens are encoded move further apart. This will serve us in a scenario like the one we described above.

Of course, positional encoding is more than that. The simple reduction of similarity could be achieved simply by using embedding like the ones we described in previous paragraphs. Positional encoding preserves both absolute and relative positions and can be used in cases where we have to process linear transformations, etc. On the contrary, the temporal embedding that we described above and which is calculated based on the timestamps will only capture the notion of an absolute position (e.g. month x, day y, etc.) and not a relative one (e.g. two days before this, etc.).

11.2.2.3 Token encoding

Regarding the embedding of each token Zhou et al. use 1D convolutions. In case we used a 1x1 sized kernel in the convolutional filters we would end up having a layer with a functionality similar to that of a Dense (Linear) layer. This is a similar implementation to the one we have in the so-called feed forward layer of the Transformer. The important thing is that, after the embedding, the tokens will have `d_model` dimensions. The initial version of the code by Zhou et al. uses a kernel of size 3 for the convolutional layer of the `token_embedding` function. Because each token corresponds to a single feature (out of 50) of our dataset, we chose to use a kernel of size 1x1.

For our experiments we chose `d_model = 128` (embedding size), as this will help us to achieve a smaller footprint of our network in the GPU's memory. It is important to note that the size of the model's token embeddings must be an integer multiple of the number of attention heads. Our network's configuration uses 8 attention heads. Therefore, each head will contribute 16 columns (dimensions) out of the total `d_model = 128`. It should be noted that the number of attention heads was chosen mainly to avoid very large gradient tapes (see: training using sliding windows) which would have been particularly difficult to backpropagate on a typical graphics card with a relatively limited memory. In general, all arguments can be selected via a grid search (exhaustive examination of combinations of hyper-parameters for predefined values), but this would require access to significantly powerful resources (expensive graphics cards, etc.). Our purpose here is not to build a final product but to study various techniques for addressing the problem of energy forecasting in RES parks.

11.2.2.4 Farm index embedding

Completing the presentation of the embedding layer, we also have to mention the so-called 'farm index embedding'. As we have seen, each token is associated with a specific wind farm. Therefore, we have to add another component using the `nn.Embeddings` layer provided by PyTorch to account for the farm index. See: Code snippet 3 from the appendix for more details regarding the implementation.

11.2.2.5 Feature embedding

The last component to the embedding layer is the 'feature embedding'. For each timestamp we have a total of 50 features (5 features for each of the 10 farms), therefore each of these 50 columns will be 'stamped' with their corresponding index (1 – 50). In a similar way as in 'farm index embedding', we also work for the implementation of the 'feature embedding', which will codify the feature's index into our final embedding of the token. Thus, we hope that the network will be able to draw a distinction between different features.

11.2.3 Analysis of embeddings

In the previous paragraphs we described the method we followed to construct the embedding layer of our network. This layer, as we have seen, includes the use of the `nn.Embedding` class provided by

PyTorch which is, essentially, a trainable lookup table, i.e. it maps a number to a tensor of size `d_model`. A positional encoding was also used, which is of course a non-trainable layer, meaning it is a static calculation of a function.

One problem with using the `nn.Embedding` class is that it limits us to using a maximum of n tokens where the number n is passed as an argument to the constructor of the `nn.Embedding` class. Networks based on the Transformer architecture can theoretically accept sequences of any length and are not limited to an input of fixed length n . This results from the design of the Attention layer which calculates the query, keys, and values regardless of the size of the sequence, for which it is indifferent. In the case where we use the class `nn.Embedding`, however, we are essentially limiting the maximum length of the input to the attention layers.

In the specific implementation we described earlier, we will not be having this problem regarding the total length of the time series based on the number of timestamps (time axis), because we used the class `nn.Embedding` to encode elements that are not related to the length of the sequence on the temporal axis, such as the farm index (1 - 10), the feature index (1 - 50) etc. Obviously, these intervals (1 - 50, 1 - 10) will not change for the specific datasets we are studying. However, if they do change, (e.g.: if we want to use our model for a different dataset that has a different number of farms, a different number of total forecast features etc.), we will be having a problem.

A solution to this problem is presented in (Foster 2023). Specifically, the book refers to a tool named 'sinusoidal positional encoding' (you may find an implementation for it in `keras_nlp.layers.SinePositionEncoding`). This is a variant of the positional encoding that supports real numbers and not just integers for encoding positions. You may refer to (Foster 2023) and (Keras 2023) for more details.

We note that in our implementation we have not included the above trick, but we mention it here for the reader's fuller information.

11.2.4 The mask in the decoder's self-attention

The decoder's input in networks based on the Transformer architecture is generally related to the predictions. As we know, the decoder in a Transformer network usually has 2 attention layers. The first layer is self-attention which means that both the queries and the keys will come from the same sequence. However, if the decoder input (during the training) includes predictions, some measures must be taken to ensure that the network does not 'cheat' by having access to data related to future timestamps, that is, data that it is supposed to learn to predict. The means we have to achieve this is to use some kind of mask at the input of the decoder.

Let's consider an example: Suppose the decoder is tasked to calculate the (self) attention for a specific query, q_n . Suppose this query is located at position n within the sequence (temporal axis) and this position concerns an element that is to be predicted. Calculating the attention layer's output for q_n

requires calculating the scores for each pair (q_n, k_i) where k_i is the i_{th} key of the sequence, that is, the key which is derived from the i_{th} token. If index $i > n$ then the attention layer will have access to information it is supposed to predict as all the positions in the sequence after the position n must be considered unknown as they are supposedly referring to future observations. Therefore, it will be necessary that the pairs (q_n, k_i) refer to keys for which $i < n$ applies.

We therefore have to apply a certain masking mechanism to somehow hide from the attention layer all keys (and corresponding values) concerning tokens positioned later in the sequence which have (supposedly) not yet been predicted (we are referring to the training process mainly). Let us now come to the specific case we are considering (wind farms energy outputs etc.). As shown in Picture 15, for each hour of the dataset we will have 50 tokens due to the spatio-temporal attention mechanism, as described above. Therefore, as far as the specific sequence is concerned, it is obvious that if a query is located at position 126 e.g. it makes no sense to hide any of the keys that concern tokens located in positions from 0 to $\left\lceil \frac{126}{50} \right\rceil \cdot 50 - 1$. On the contrary, all subsequent keys should be hidden by the mask.

However, as we saw above, the first 10 tokens out of every 50 concern the energy production for each of the 10 wind farms. This is the prediction target of our network. The next 40 tokens out of every 50 are about weather forecasts (which are not prediction targets). These predictions do not make sense to be hidden from the self-attention layer of the decoder for any moment in time either in the past or in the future. Thus, the mask of the self-attention layer of the decoder will only concern the first 10 elements out of each 50, which concern the prediction target values. We hope that modifying the mask in this way will enable the network to make better predictions by making available to it all the information about both the past and the future in such a way that it can produce rich spatio-temporal correlations between the elements of the sequence.

Let us now examine the specifics of the implementation of such a mask. Examining the file named `attn.py` from the software repository available in: (zhouhaoyi 2020) we find the definition of a class named `ProbAttention`. This class includes the implementation of the so-called probSparse attention as described in a previous paragraph. The basic idea is to use some metric that gives us an estimate for the importance of each query. This will be based on the estimation of the similarity of the distribution for $p(k_j|q_i)$ to the uniform distribution. You may refer to the previous chapter where we discuss the Informer's architecture in more detail as well as study the original paper (zhouhaoyi 2020).

Non-significant queries, i.e. those which are not expected to contribute significantly, if at all, to the predictions, are expected to give distributions for $p(k_j|q_i)$ which are close to uniform. This means that if a query q_i is not important, then all attention scores for it will be approximately equal to $1/L_k$ where L_k is the total number of keys. Therefore, the calculation of the attention layer's output will be trivial. We can just take the average of all the keys.

Let's look more specifically at the code: The class `ProbAttention` includes the methods: `_prob_QK`, `_get_initial_context`, `_update_context`, and `forward`. As for: `forward`, no introduction needed. It is the standard function commonly used in PyTorch and is related to the 'forward pass' in the network's gradient tapes.

`_prob_QK` includes the implementation of the assessment for the importance of the query as described in the previous paragraphs, based on the KL divergence, etc. Essentially, it returns the scores for the top-

u queries based on Informer's sparsity measurement, where u is a number that can be chosen by the user, through the parameter named `factor`. (see: (zhouhaoyi 2020)). For example, for our experiments we have chosen `factor` = 50. The examples used by Zhou et al. in their GitHub repository they use a `factor` of 5. Note that for our specific experiments the decoder input is 400 tokens long. This number could be much higher. The problem with using a `factor` of 5, as in the original repository's (zhouhaoyi 2020) examples, is that Informer's prob Sparse attention layer will produce scores for only 30 queries for a sequence of 400 tokens, since the number of important queries is defined as a parameter by the user based on the formula: $u = c \cdot \ln L_Q$ where c is the 'factor' as it appears in the original Informer's Python code (zhouhaoyi 2020). By choosing `factor`=50 we end up with the top-300 queries which seems (at least intuitively, and in our opinion) more logical for the problem we are going to tackle. Indeed, the exact number u for the top-u queries should, of course be chosen experimentally, if the available resources are adequate. However, here we have a point that could be improved in Informer. Perhaps with some better design it would have been possible to automatically select the correct value for the factor parameter.

The function: `_update_context` essentially calculates attention's output as a kind of weighted average, as we saw in a previous paragraph, by multiplying the scores by the values and then summing. However, the attention scores are calculated only for the important queries (the top-u ones) while for the remaining queries, the average of the values (output of the 'value' network) is simply used, since it is assumed that all scores for the non-important queries will be equal to $1/L_k$. Let's take a look at the respective code snippet:

```
context_in[torch.arange(B)[:, None, None],
           torch.arange(H)[None, :, None],
           index, :] = torch.matmul(attn, V).type_as(context_in)
```

(zhouhaoyi 2020)

Where `B`, `H` refer to the number of batches and the number of attention heads. As for `attn` it contains the scores for the top-u queries and `V` contains the corresponding values (the outputs of the 'value' network). Regarding `index`, it contains the indexes of the queries, that is, their absolute position within the sequence. Essentially, the above command takes as input the tensor: `context_in` which includes the trivial calculation of all queries as the average of the keys, and replaces those positions that correspond to the important queries (indexed by the tensor named: `index`) with the values resulting from the formula: $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ (according to the analysis of the attention mechanism discussed earlier).

As for: `context_in`, it is obtained from the `_get_initial_context` function. Let's see how `context_in` is calculated in the Informer's original source code for the case where we do not have a mask, such as in the encoder layer or in the decoder's cross-attention:

```
V_sum = V.mean(dim=-2)
context = V_sum.unsqueeze(-2).expand(B, H, L_Q, V_sum.shape[-1]).clone()
```

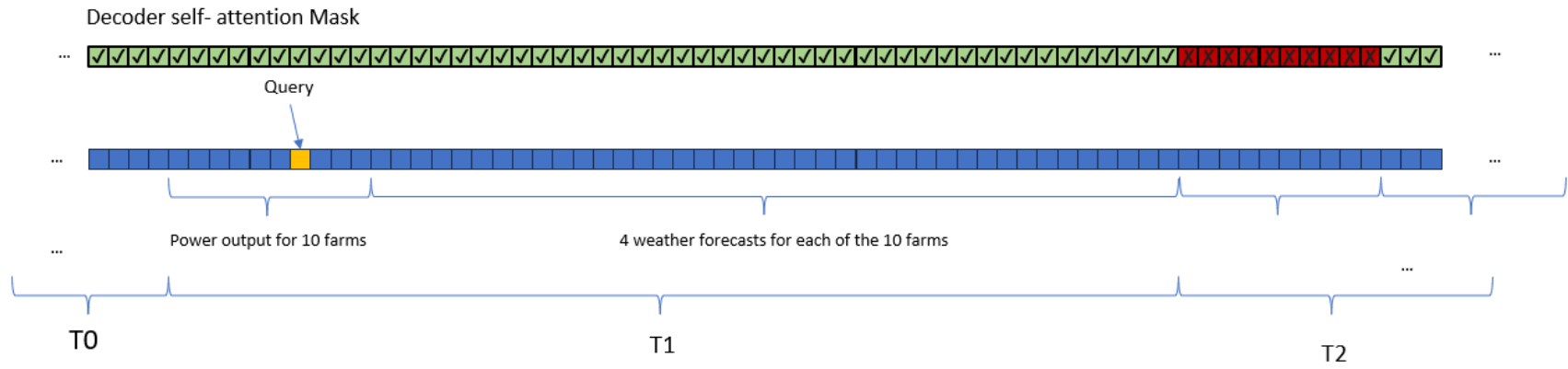
As we see, each (non-significant, according to the sparsity measurement) query is mapped to the average of all keys. Because we don't have a mask in this case all queries will get the same value, as we see in the second line. But what happens if we have to apply a mask?

Zhou et al. use the following calculation in their original source code for the Informer:

```
context = V.cumsum(dim=-2)
```

That is, the cumulative sum is used. This can certainly work, but if the sequence is very long, we will probably have a problem with the accumulation of large numbers. It would be better to use the cumulative average. Moreover, attention itself can be considered as a kind of weighted average. However, in the case we are examining for this thesis, this is not sufficient either: we want the mask to hide all data regarding the future energy production of each farm, which is the target of the model, and to also reveal all weather forecasts for either future or past timestamps. Please refer to: Picture 16:

Picture 16 Decoder self-attention mask



Picture 16 shows the mask for a single query. Thus, `_get_initial_context` needs to be modified at the point of the calculation of the 'attention' for the non-important queries. For these queries `_get_initial_context` will have to calculate the appropriate cumulative average while also taking into account the mask discussed above. Thus, the line:

```
contex = V.cumsum(dim=-2)
```

has to be replaced. Regarding the required modification of `_get_initial_context`, note that we will need to construct an appropriate mask for the attention keys taking into account the spatio-temporal correlations. For the case of the typical, 'vanilla' mask, the implementation is very easy as we can use `torch.tril` (provided by PyTorch) to construct a triangular mask with zeros and `-inf` (this is called an additive mask). In the case we are considering, this is not enough because the mask is not exactly triangular. Specifically, the first $n = n_features$ lines of the mask will all be filled with the same value. This is because the first n queries all refer to the same timestamp, so there is no reason to apply a different mask. That is, the mask depends on the timestamp to which the token is associated and not simply on the absolute position of the token within the sequence. The 'step' for hiding the keys is therefore $n = n_features$ and not 1 as in the case of the vanilla attention 's mask.

We present a way to construct such a mask (see Code snippet 4. The construction of the mask for the self-attention layer of the decoder. from the appendix) using the `torch.einsum()` function provided by PyTorch. Additionally, we have to perform the appropriate actions to handle future weather forecasts which will always be available to all queries when calculating the attention layer's output (see the appendix).

There are still some technical details regarding the changes that will need to be made to the code. In order to not overcrowd the text, we invite you to examine Code snippet 6 from the appendix, for the specifics.

11.2.5 The feed forward layer in the decoder

The decoder layer will output a sequence of dimensions: $(batch_size, n_features * (label_len + pred_len), d_model)$. However, we want the final result to be of this format (dimensions): $(batch_size, pred_len, n_farms, 1)$. To achieve this, we use 2 one-dimensional convolutional layers (`conv1d`), adapting the technique deployed by: (zhouhaoyi 2020) (see Code snippet 5).

It is noted that there are various ways to deal with the decoder's feed forward layer and there may be better methods for its implementation.

Next, we will examine the last point that we will need to modify regarding the implementation of the decoder mask we are interested in. Before we proceed, we should briefly review Informer's training at the decoder level and its differences compared to the vanilla Transformer.

11.2.6 Training on vanilla Transformer and on the Informer

In the typical scenario, the vanilla Transformer will produce the predictions one by one. The mask will hide any position in the sequence that has not yet been predicted by the network. In practice this means that the tokens located at the positions to be hidden could be replaced by zeros etc. or, even better, we could set the respective attention score value at $-\text{inf}$ directly, instead of calculating $\frac{QK^T}{\sqrt{d_k}}$ before the application of the SoftMax layer. Since the SoftMax function returns values that sum to 1, using $-\text{inf}$ in the place of $\frac{QK^T}{\sqrt{d_k}}$ will definitely result in zero attention scores for the query-key pairs we want to exclude. As a result, the keys hidden by the mask will not contribute to the final calculation of attention.

In the vanilla Transformer, we would have to calculate each prediction for each timestamp one by one. The previous prediction would be incorporated into the next input to the decoder, without the specific position of the sequence being hidden by the mask anymore. Any element that has not been predicted will continue to be hidden by the mask.

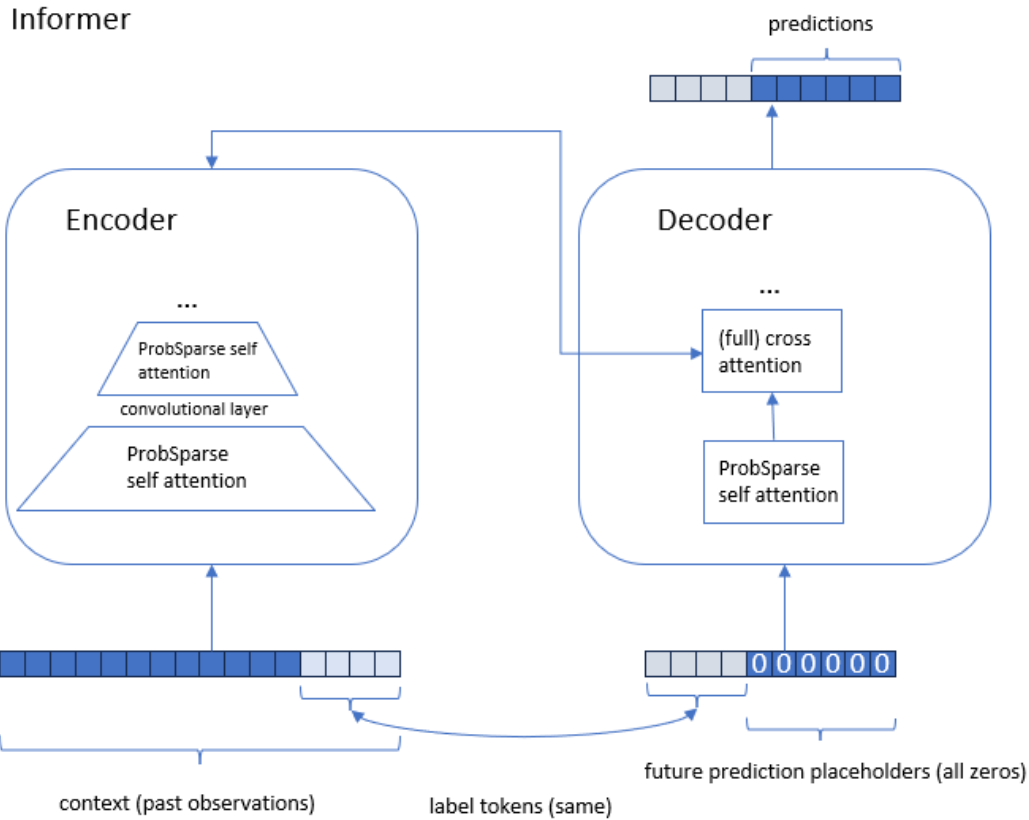
We therefore see that the network will have to go through several stages to produce the prediction for the entire sequence. This may be problematic when processing very long sequences or large data sets. Zhou et al. in the publication for Informer (Zhou 2021) propose a different procedure for constructing the decoder's predictions. We will examine this in the next paragraphs.

11.2.7 The decoder's predictions on Informer

In the case of Informer, the decoder predictions will all be produced in one forward pass. That is, the entire sequence will be predicted at once. This is done through a process that the researchers call 'Generative Inference'.

In the typical vanilla Transformer's version, the decoder input sequence begins with a special token which marks the beginning of the sequence. Conventionally we represent it with <START>. This special-purpose token somehow signals the decoder to produce the prediction for the first token in the sequence.

In Informer we have, in a way, an extension of this method. The idea is to use some kind of preamble in the decoder sequence which makes it easier for the decoder to produce all the predictions at once. This preamble, which researchers call 'label', is essentially a small part of the sequence, preceding the predictions that interest us. This means that a small piece from the end of the encoder input sequence will be present as a preamble at the decoder input. This preamble will be followed by a series of zero values (a placeholder token essentially), since these are the positions in the sequence that we want to predict. You may refer to the Picture 17 for the general architecture of the Informer at a glance.



Picture 17 Informer's architecture

The consequences of the above mechanism, and given the ProbSparse attention, make some changes to the Informer code necessary to enable it to work with the inputs we want. In other words, we want to pass information about future weather forecasts to the decoder input. To achieve this, we will need to modify the Informer's code in (among other places) a portion of the function (`_process_one_batch`) which is tasked to process a batch from the training dataset. You may refer to Code snippet 7 from the appendix.

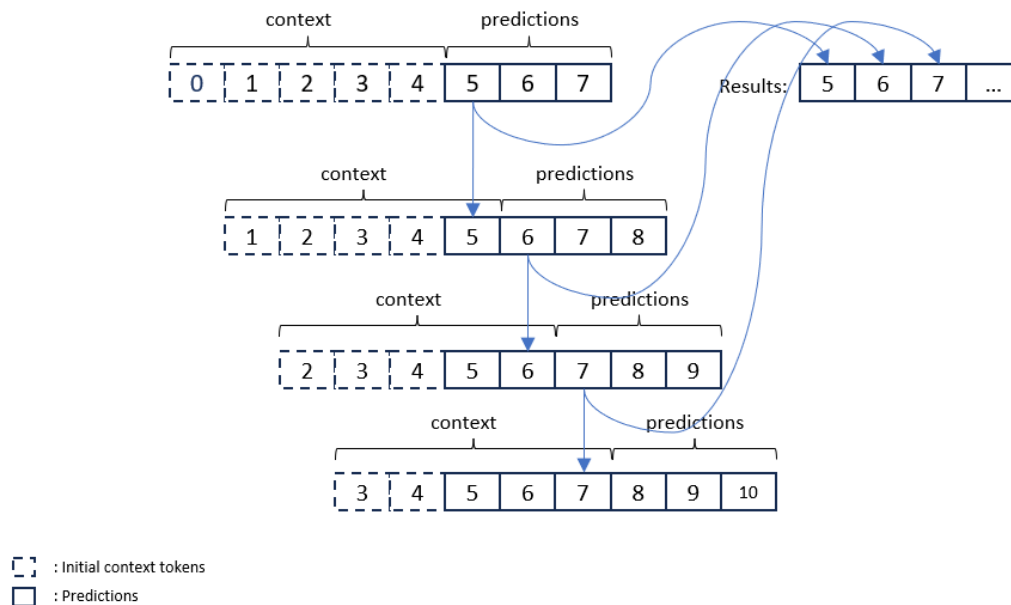
11.3 Two stage training using sliding windows

Our network is configured to produce predictions for only 4 or 12 hours depending on the training parameters. This was done to reduce the size of the network's gradient tapes during training as much as possible because we will be using a method we call the 'sliding windows'. It is the gradient tapes that primarily drive the network's memory footprint.

Regarding the sliding windows: one way to get more hours of forecasting is to use a kind of sliding window. We can place the previous predictions made by the network at the end of the encoder's input.

This way we can get as many predictions as we want. The idea is to put this whole sliding windows process inside the backpropagation graph when training the network. That is, the training will take place in 2 stages:

Predictions using sliding windows



Picture 18 Sliding window

- In the first stage, we train the network to predict the energy production for n_{pred} number of hours in the future (all in one forward pass) using the Informer's 'Generative Inference' method.
- In the second stage we use the model we trained in the first stage, now training using the sliding windows mechanism described in the image above. In a way, therefore, the network is trained in a similar way to how Recurrent Neural Networks (RNNs) are trained.

As we will see below, this idea seems to be quite useful and could possibly give better results than the standard training process.

This idea came from the observation that the network faces the problem of overfitting as we will see below in the discussion related to the results. It is possible that this type of 2-stage training push the model to produce more reliable predictions as the network will have to rely on its own predictions from previous time points to increase the horizon of its future predictions.

As we will see later, this idea seems to help to address the problem of overfitting to some extent. The disadvantage of this training method using sliding windows is that the training time increases considerably, which under certain conditions could be a problem. This is because of the large size of the gradient tapes since, for each 'window', the backpropagation algorithm has to append copies of the

entire network's gradients to an ever-growing gradient tape. However, in our opinion, it is worth attempting such a 2-stage approach to training our network as this strategy seems to be supported to some extent by the results.

12. Comparison with other methods

In this paragraph we will refer to the methodology we followed for the network's evaluation. Specifically, we will compare the performance of our network with 2 methods:

- The 'vanilla' Informer (i.e. without spatio-temporal attention and without a custom mask)
- The Temporal Fusion Transformer (TFT) (Bryan Lim 2021)

Regarding the architecture of the 'vanilla' Informer, we have mentioned it extensively in a previous paragraph. As for the TFT method we are offering a briefing below. It is noted that this specific method (TFT) has demonstrated a fairly good quality in its predictions and was generally considered a suitable benchmark due to the very good performance of this architecture.

12.1 Temporal fusion transformer

Introduction

The Temporal fusion transformer (TFT) architecture was proposed in 2021 (Bryan Lim 2021). The researchers worked on 3 main axes:

- 1) To produce an interpretable, according to the researchers, variation of the attention layer, using a modified version of it in which the weights of the 'values' network are common to all 'heads'.
- 2) The incorporation of 'static' knowledge (the publication uses the term 'static covariates'). In this way, certain pre-calculated parameters of a statistical nature can be incorporated to the network's training.
- 3) The selection of the most important features using a layer called a variable selection layer.

Note: The TFT network implementation we used for the comparisons constructs forecasts for only one of the 10 wind farms. This is mainly related to the implementation at the Variable Selection Layer level. As we will see, VSL will, in a way, compress all the features into one (in the case of the time series we are examining, for example, we have 50 features for each time point). It is therefore logical to choose to use different models, each of which will be specialized in only one of the 10 parks. Therefore, it is essentially an ensemble-type method and this, arguably, dictated by the TFT architecture itself.

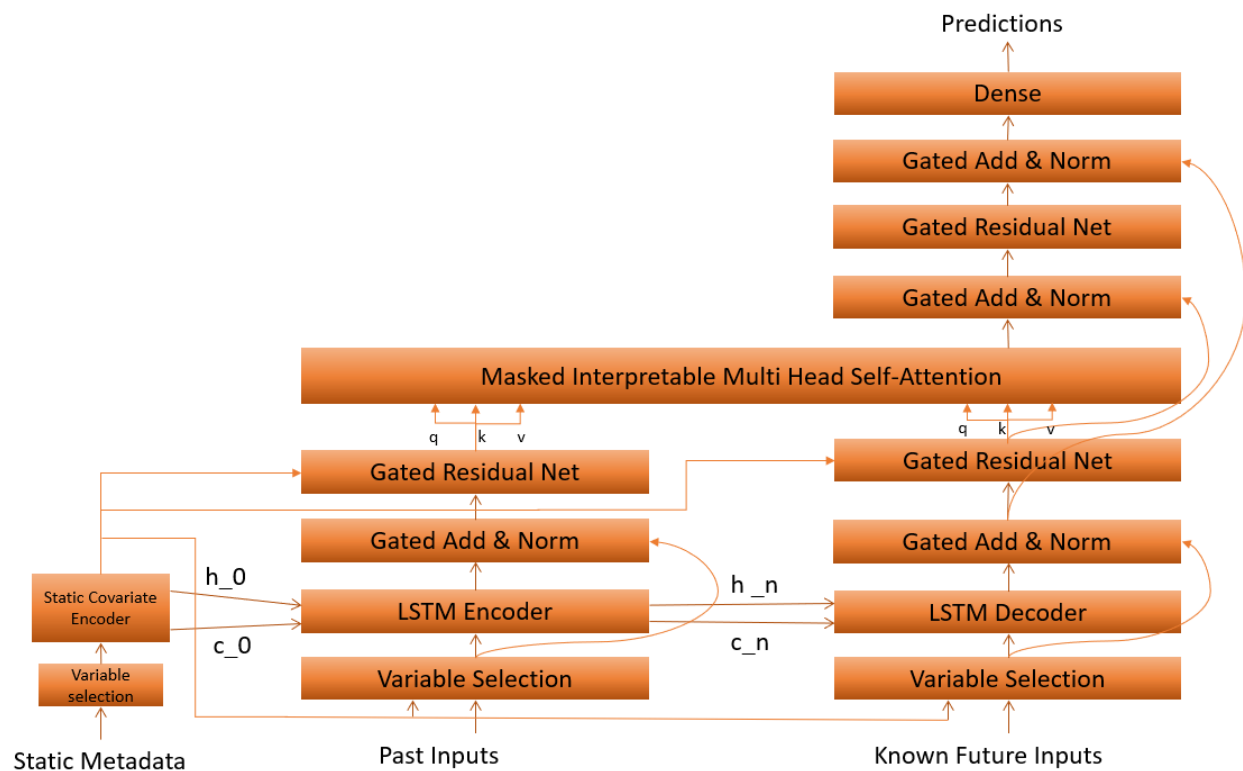
A question that arises is whether we could use similar techniques (ensemble, etc.) for the other methodologies we are examining. The answer is yes, we could do something like that, but this would require significant computing resources especially in cases where we have a large number of parks. In this thesis, the methods that we will be examining predict, with a single model, the energy production

for all of the parks. The use of multiple models, each specializing in only one of the parks, is a subject for future research and may present some advantages, but this needs to be tested experimentally and will require significantly more computational resources.

Therefore, regarding the TFT, we will essentially have 10 models, each of which will produce predictions for a single wind park. Our network produces all the predictions for all the parks, so for comparison we will take the score for the average of the TFT models' prediction scores. It is also noted that the parameters for the TFT network were found after a grid search (exhaustive search of all combinations for selected ranges of hyper-parameter values). The size of the context (past time points) and the number of predictions will be the same for all methods being compared.

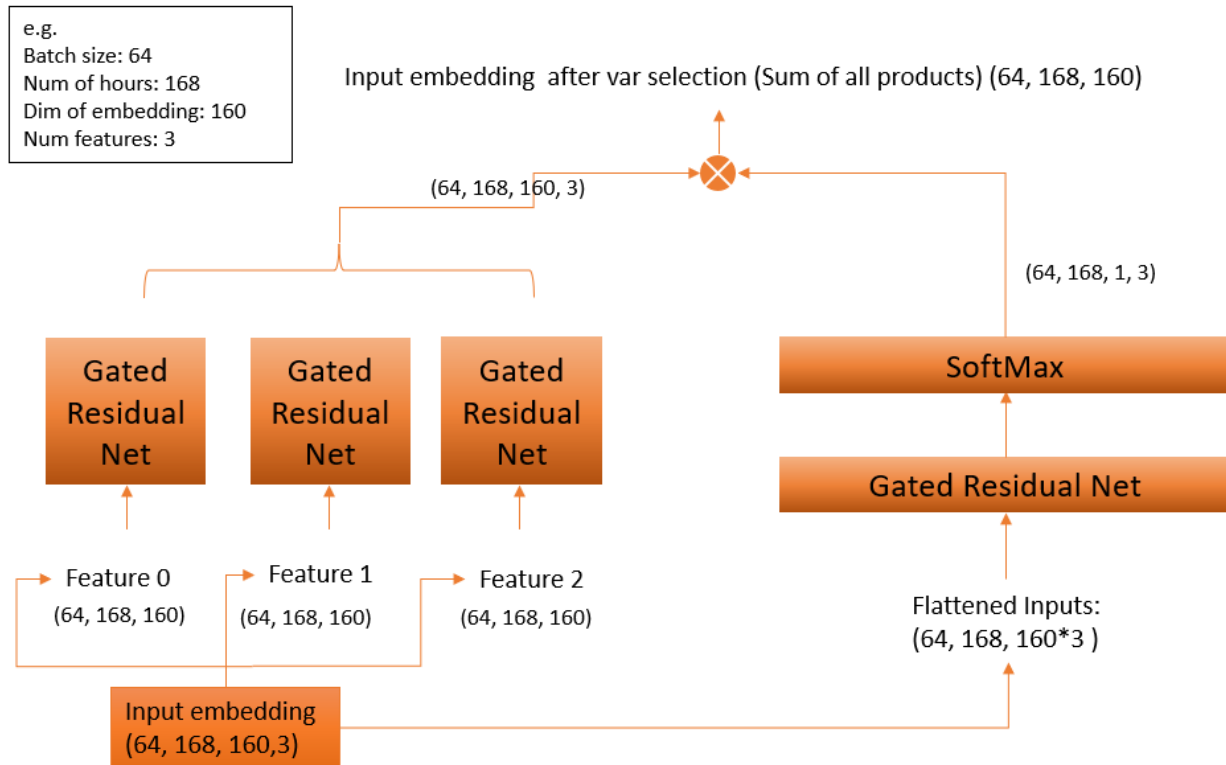
12.2 The TFT architecture

At this point we will examine the TFT network's architecture in more detail. In Picture 19, which is based on the publication by (Bryan Lim 2021), we can see a general overview.



Picture 19. Temporal Fusion Transformer's architecture (Bryan Lim 2021).

Regarding the Variable selection layer, its architecture is presented in Picture 20. In this particular image we have included an example of the dimensions of the input and output tensors (the tensor's 'shapes') for a better understanding of the computation.



Picture 20. Temporal Fusion Transformer (Variable selection layer). Based on: (Bryan Lim 2021).

As we can see, the Variable selection layer will somehow 'compress' the input channels or features into one. The gated residual net is responsible for selecting the elements from each feature which will be included in the final embedding. This final embedding will eventually be pushed to the attention layer.

For a precise description of the gated layers, we will refer to the original publication by (Bryan Lim 2021). Here we will simply mention that this specific mechanism is based on the use of the sigmoid activation function. A similar idea of a gating mechanism can be found in the LSTM architecture.

13. Evaluation of the experimental results

13.1 The network parameters

First, let's look at the network parameters we used for our experiments. Here we should mention that the selection of hyper-parameters (hyperparameter tuning) is a complex issue and we often end up applying the grid search method, which means that we examine all combinations from a set of different values for each parameter of the network and get the combination that corresponds to the best model. However, because this requires a lot of computational power, here we will be using only 2 different setups. However, we believe that this is enough to give us some clues. Our purpose here is not, of

course, to prove anything, we simply want to obtain some indicative information that may give us some ideas for continuing our research.

We therefore tried 2 setups:

- **Setup 1:**

d_model = 128 (the size of the embedding)

batch_size = 16

n_heads = 8

d_layers = 2 (number of decoder layers)

e_layers = 1 (number of encoder layers)

attention: Prob sparse attention in all the layers except in the decoder's cross-attention where we apply the full (vanilla) attention.

factor = 50 (please refer to paragraph: 11.2.4)

context = 18 (18 hours of historical data at the encoder input)

n_preds = 4 (predictions for 4 hours into the future. This is the size of the decoder output)

label_len = 4 (the size of the preamble (label) at the decoder's input)

n_farms = 10 (we have 10 wind farms)

n_weather_forecasts = 4 (4 weather forecast items for each park and for every hour)

dropout = 0.2 (where there is a dropout layer it takes a value of 0.2)

d_ff = 2048 (number of input and output channels of the 2 convolutional layers of the encoder (feedforward))

loss = MSE

- **Setup 2:**

It is the same as above with the difference that we now have: factor = 90 and n_preds = 12

As we said above, the network training will be done in 2 stages using the sliding windows mechanism. The second stage for both setups will involve a 24-hour forecast (sliding window).

The size for the GEFCOM dataset is 17538 hours: from 1/1/12 1:00 until 31/12/13 18:00. We divide the dataset into 3 parts: i.e. training, validation, and test set. The test set is the last month and the validation set is the month before last. It is generally more appropriate to use the last month for the test set, as otherwise the model will be able to derive important information about the characteristics of the month used for the test data. We apply the same procedure for the AEMO dataset.

As you can see, we have tried to keep the network size small as the training data is limited. We are primarily interested in experimenting with ways for addressing such problems. The scope of this thesis is more about experimentation and not necessarily about building an end product.

13.2 Tables of scores

The table below shows the summarized results we obtained from training the 2 different setups in 2 stages.

- stage 1:
 - 4 or 12 hours of predictions (at the decoder's output, not using sliding windows), depending on the setup
- stage 2:
 - 24 hours of predictions using sliding windows during the training (very large gradient tapes at this stage because of the sliding windows training for 24h of predictions).

In all the results below, the training termination criterion is based on the validation set. Specifically, we keep the model that got the best score in the validation set after a specified number of epochs.

In the tables of this paragraph, you can find the aggregated data. For more detailed measurements, you can refer to the appendix where we have recorded all the results we have collected. Here, for the sake of brevity, we have included only those elements that are useful for our comments and conclusions.

- **GEFCOM (24h)**

Method\ MAE loss	Train	Validation	Test
TFT	-	-	0.11494
setup_1 (training with sliding windows)	0.12172	0.14982	0.14217
setup_2 (training with sliding windows)	0.10010	0.15370	0.15293
Vanilla Informer (without spt attention custom mask, sliding windows etc.)	-	0.19955	0.19411

Table 1. Aggregated results for 24 hours of predictions (MAE) (GEFCOM)

Method\ MSE loss	Train	Validation	Test
TFT	-	-	0.02758
setup_1 (training with	0.02654	0.03960	0.03969

sliding windows)			
setup_2 (training with sliding windows)	0.01805	0.03956	0.04285
Vanilla Informer (without spt attention custom mask, sliding windows etc.)	-	0.06287	0.05770

Table 2. Aggregated results for 24 hours of predictions (MSE) (GEFCOM)

- **AEMO (24h)**

Method\ MAE loss	Train	Validation	Test
TFT	-	-	0.1060
setup_1 (training with sliding windows)	0.0604	0.1857	0.2047

Table 3. Setup 1, AEMO, stage 1+2

Method\ MSE loss	Train	Validation	Test
TFT	-	-	0.0216
setup_1 (training with sliding windows)	0.01848	0.05981	0.0864

Table 4. Setup1, AEMO, stage 1+2

Also important for understanding the two-stage training we are using are the following tables in the sense that, although they do not provide any proof of a mathematical nature, they can however give an indication that two-stage training using sliding windows may have some benefits. Please refer to paragraph 14. for more information.

24 hours of predictions for the best model with the setup 1 , using only stage 1 training (without training using sliding windows)	
MSE (validation)	0.05356
MSE (test)	0.05174
MAE (validation)	0.1670
MAE (test)	0.1444

Table 5. (Setup 1, Gefcom, stage 1 for 24h of predictions)

24 hours of predictions for the best model with the setup 2 , using only stage 1 (without training using sliding windows)	
MSE (validation)	0.06163
MSE (test)	0.05501

Table 6. (Setup 2, Gefcom, stage 1 for 24h of predictions)

14. Conclusions

The results presented in the previous paragraph can give us a general picture of the nature of our network. We cannot prove anything in a mathematical sense and this is not our intention. We are simply interested in getting some directions to see how we can possibly improve our model. The conclusions we can draw from the results are therefore as follows:

- Training with the sliding windows method is likely to benefit the model to some extent. We observe that for setup 1, after the training stage utilizing sliding windows (stage 2) for 24 hours of predictions, we get approximately the same or even a slightly better score than what setup 2 achieves. (see Table 1, Table 2) It is noted that regarding setup 1, in the first stage of network training (without sliding windows), the decoder has been trained for only 4 predictions, while setup 2, also in the first stage of its training, has been trained with the decoder producing 12 predictions. This means that the second stage of training in setup 1 will be more heavily based (compared to setup 2) on previous predictions of the network, because the rolling windows will need to be 'expanded' more. This translates into more extensive gradient tapes compared to setup 2 and therefore more and, potentially, 'richer' paths for the optimizer to follow (hypothetically). This is therefore a possible indication (obviously not "proof") that the sliding windows training stage may help the model achieve somewhat better results.
- More specifically, regarding the indications for the (potential) effectiveness of stage 2 of training (sliding windows): We compared the network performance for 24 hours of predictions when: a) the network was trained only in stage 1, and b) the network was trained in stage 1 followed by stage 2 (2-stage training).
To produce the 24-hour forecasts (inference), we also used the sliding windows method. Therefore, for the network to expand the number of predictions it has been trained to produce in a single forward pass (e.g. for setup 1 it is 4 hours), it will have to rely on its previous predictions according to what was mentioned in the previous paragraphs.
Specifically, regarding setup 1, and for the GEFCOM dataset, we observed that the MSE loss (for a 24-hour forecast) went from 0.05174 in stage 1 (one forward pass for all predictions), to 0.03969 after training with the sliding windows method (stage 2) (multiple forward passes for obtaining the predictions both in training and in inference). The respective scores for setup 2 were 0.05501 -> 0.04285 (see Table 5, Table 6). In any case, this data is an indication and not of course proof (we emphasize this), of a small potential benefit of training with the sliding windows method (2 stage training etc.). In any case, it seems that if there is any benefit, it is probably relatively limited. The spatio-temporal attention combined with the modified mask in

the decoder seems to work better than the vanilla version of Informer, but not as well as the TFT architecture. However, the scores are somehow close.

- The TFT architecture is optimized to address the problem of overfitting by implementing an interpretable variant of attention, and the so-called variable selection layers. The network we proposed seems to be affected by the problem of overfitting to some extent.
- We observed that on the training data (train set) the network continues to reduce the error. Even after several epochs, the network did not reach a plateau in terms of the error. This means that it is overfitting, which is of course a problem, but it also means that it probably has the required capacity for the learning we are interested in. There are therefore some indications that lead us to believe that if the network is trained with more data, it is likely to approach the scores of the TFT architecture to some extent.

From the above, it appears that the main problem of the network is over-fitting to the training data. This problem is possibly (we cannot prove this of course) related to the fact that the dataset we use is relatively small. If we consider the non-overlapping windows of the dataset, that is, the set of sequences of length `context_len + pred_len` (= 18 + 4 e.g. for setup 1) that do not have any common elements, we will see that their number is only about 800.

One idea would be to use a layout with residual layers and skip connections based on the work by (Yusuke Tashiro 2021). Or we could abandon the use of full spatio-temporal attention and experiment with solutions for calculating temporal and spatial correlations in 2 stages as is done in the works by (Yusuke Tashiro 2021) and (Bryan Lim 2021). In general, there is no 'guarantee' for the effectiveness of one method over another, however, it is true that the experimentations we described above may push the network onto a path of over-fitting.

One idea to address this specific problem would be to use a GAN network to construct more artificial data and thus enrich the set of training samples.

Another idea is to use some architecture to denoise a noisy signal. We could, for example, add a little random noise and then denoise the signal. The denoised signal could then be, perhaps, added to the training examples.

Important note:

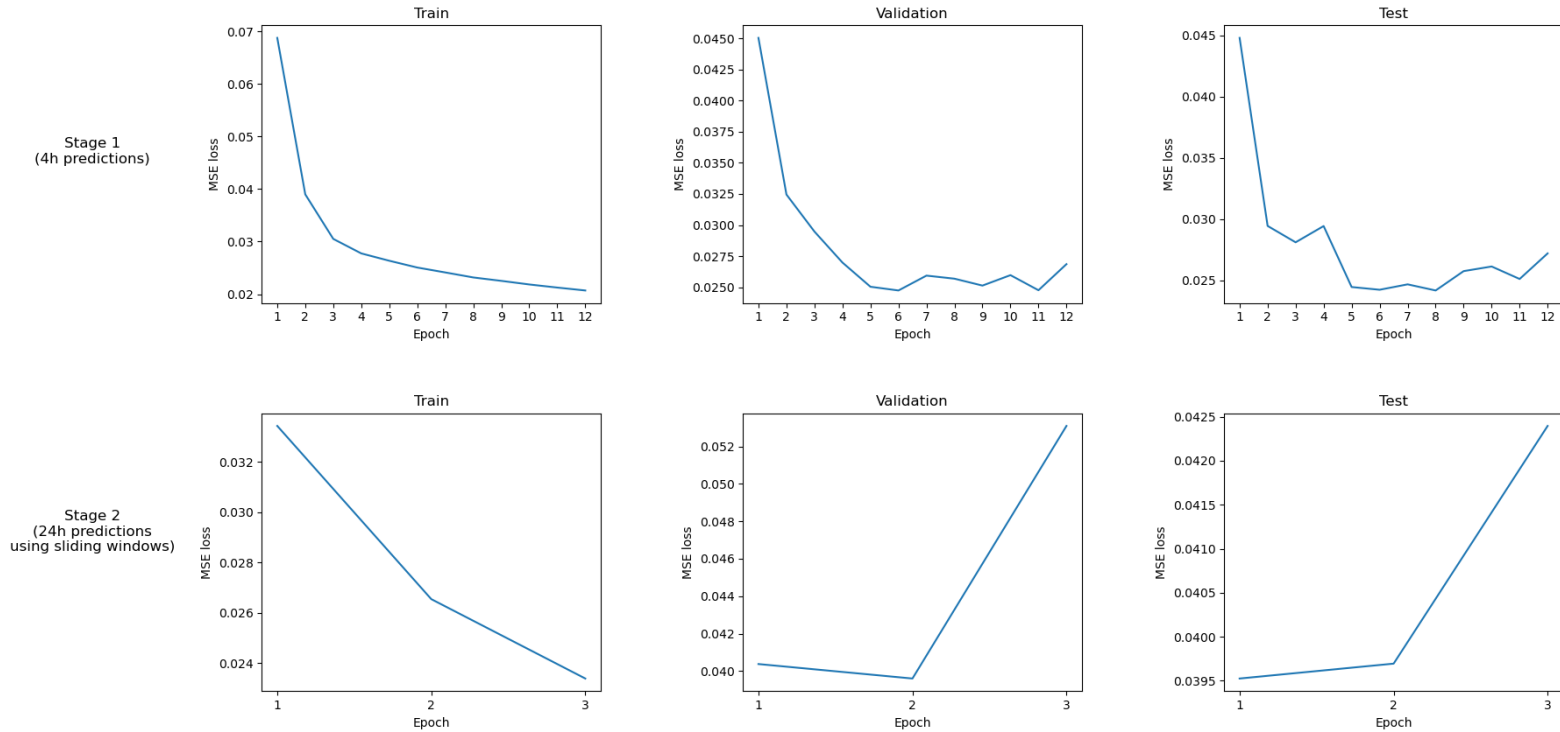
As a general comment, we would say that although there are some indications of some, possibly small, benefit of this particular proposed method, we cannot draw any firm conclusions at this stage as this would require more extensive experimentation. Any indications are not of course formal proofs. The TFT method continues to be advantageous and we believe that it is useful to look for alternative methods, besides TFT, which may show clearer advantages or provide some directions and ideas for future research. Our goal in this work is not, of course, to build a 'product' but to experiment with some of the available tools and evaluate their applicability to the specific problem we are studying. Therefore, regarding the effectiveness of the specific method we described above, we would say that it is not easy to draw any certain conclusion, but we have gained some insight into the possibilities and problems of ST attention, etc.

Therefore, we have examined one case and then we will continue our research by examining another method based on the so-called diffusion, avoiding the complexity of ST attention with the help of

two attention stages: a temporal and a feature attention layer. We will examine this in more detail below. Below we provide some diagrams with the error per epoch as well as some examples from our network's predictions.

Picture 21 Training for setup 1 (GEFCOM)

Setup 1 (context: 18h, label: 4h, decoder out: 4h, factor: 50)

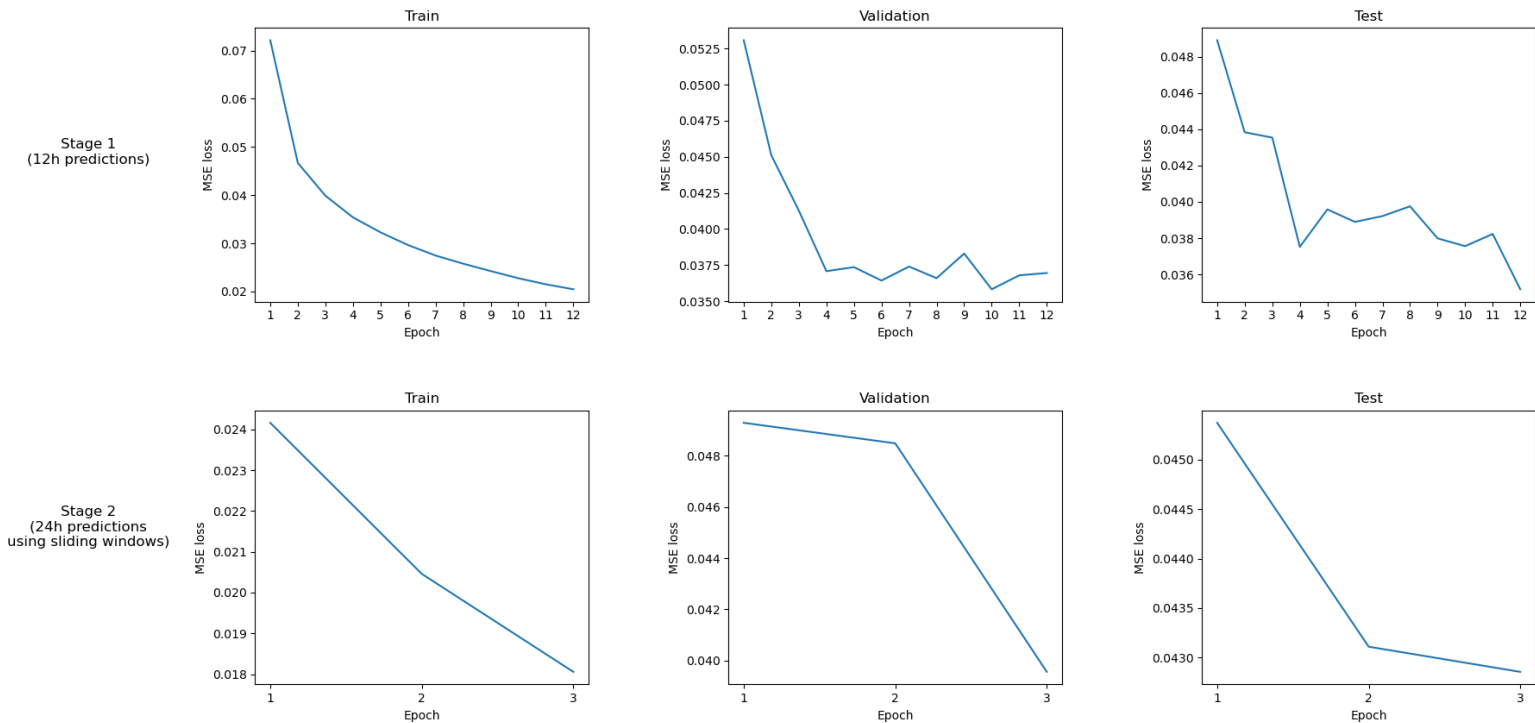


In Picture 21 we can observe the emergence of the overfitting problem after the epoch_n=6.

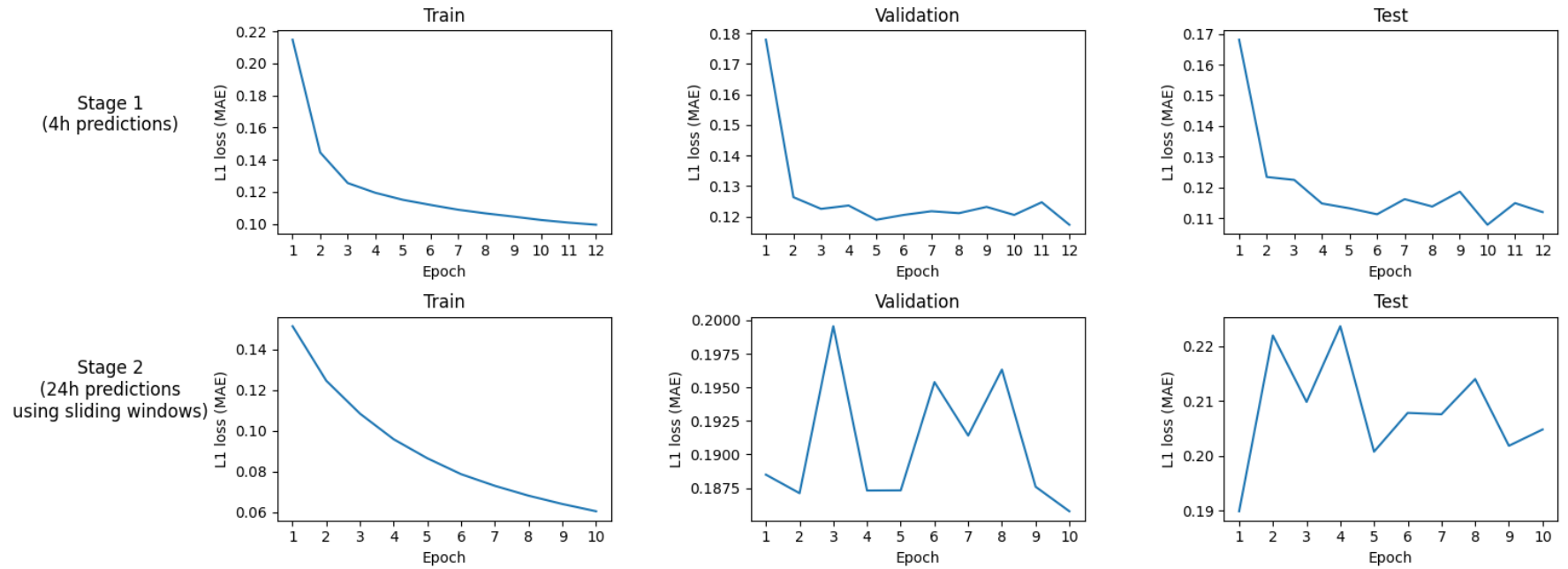
Notice that the error in the training set continues to decrease at a fairly good rate and generally behaves quite well in the sense that we do not have any particular 'turbulence'. Setup 2 has similar behavior as we see in Picture 22. This is an indication about the problem of overfitting. In the next paragraph we will examine some more details on this issue.

Picture 22 Training for setup 2 (GEFCOM)

Setup 2 (context: 18h, label: 4h, decoder out: 12h, factor: 90)



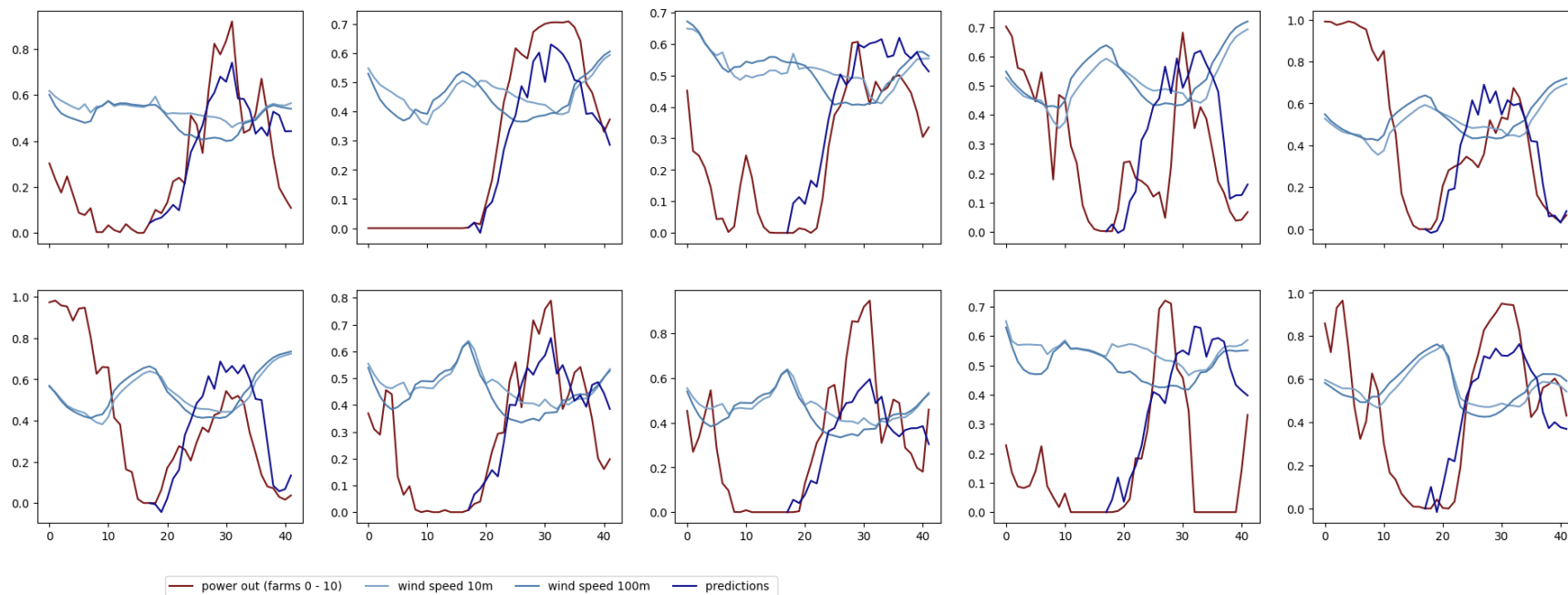
Setup 1 (context: 18h, label: 4h, decoder out: 4h, factor: 50)



Picture 23 MAE Loss for the AEMO dataset.

Similarly, the results for the AEMO dataset seem to lead to similar conclusions as for the GEFCOM dataset.

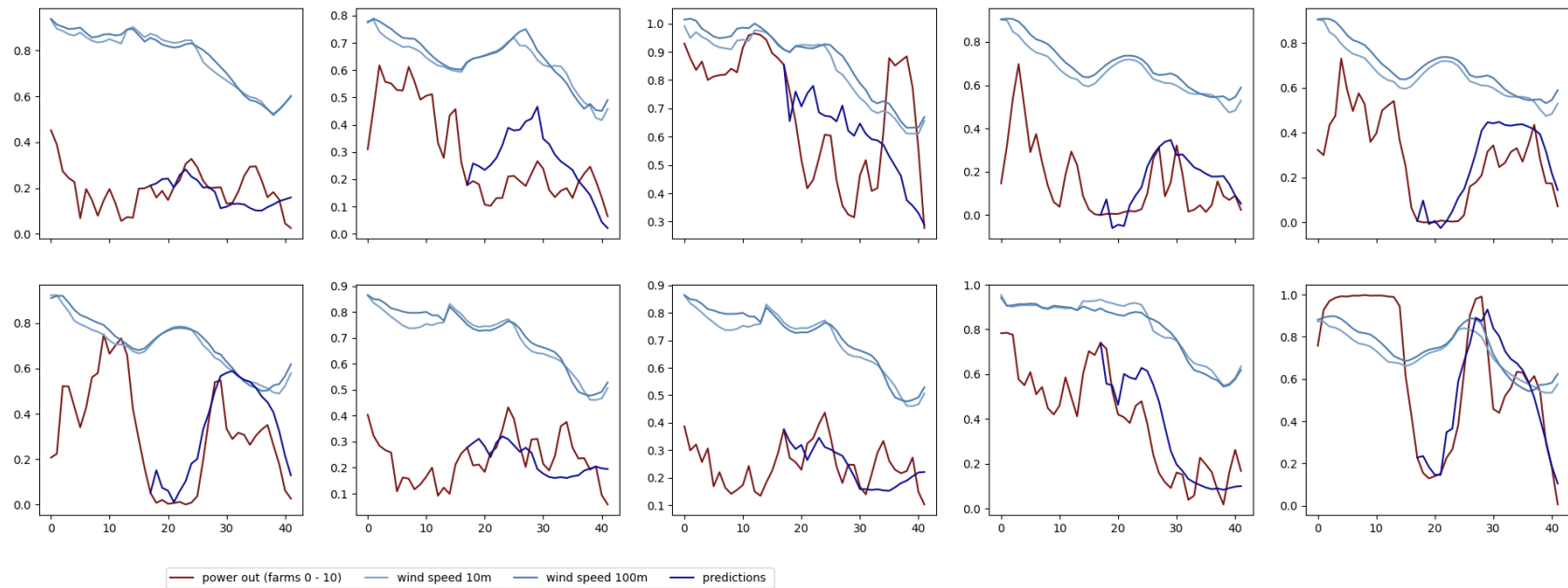
Picture 24 Example of forecasts for 24 hours (test set, best model (setup1 with sliding windows)) (GEFCOM dataset)



In Picture 24 we see the forecasts as well as the context (known data for the previous hours) for each of the 10 wind farms. Each plot corresponds to a different park. The red line represents the actual values from the data set. The blue line represents the forecasts (24 hours). We have 18 hours of context and 24 hours of forecasts (42 hours total) displayed on the charts. The remaining lines are related to the wind speed at 2 different altitudes. Note that the wind speeds are not one of the features of the dataset (although the wind speeds can be calculated from the respective vectors available in the datasets). Here we simply calculate the magnitude of the vector determined by the wind components U, V, to avoid piling up many lines on the graphs. Note that for the network not only the measure but also the direction is important, something which for the sake of simplifying the graphs is not shown in the picture. At least visually, the network seems to follow the target to a relatively satisfactory degree. All features plotted have been normalized to the range 0-1.

In Picture 25 we see another case of predictions of our (best) network (setup1 with 24 h sliding windows) for a different batch from the test set.

Picture 25 Example of forecasts for 24 hours (another example) (GEFCOM)



15. Comments on the problem of overfitting

As we saw in the conclusions paragraph, it seems that the network suffers from the problem of overfitting. This is a relatively common problem for which we have several tools at our disposal. An effective means to deal with overfitting is the use of dropout layers, regularization layers, the use of ensembling techniques, training with cross validation, etc. Another idea is to use larger datasets. For the case of the type of datasets we are interested in, we do not really have an abundance of records available, so we will need to resort to constructing synthetic data to enrich the datasets we already have.

In general, networks based on the Transformer architecture, like Informer, are to a significant extent 'optimized' in the sense that several measures (dropout, regularization layers, etc.) have already been implemented to deal with such problems. Therefore our assessment is that it makes sense to attempt to construct synthetic data to enrich the dataset to see if this can help the network to avoid overfitting to this extent. In general, several methods have been proposed for the construction of synthetic data, and the most popular are the so-called GANs and the models based on the so-called Diffusion.

Part D: Second proposed method

16. Diffusion.

In the previous part of this thesis, we studied the problem of forecasting the electricity production from wind farms using rich spatio-temporal correlations. We have seen that the main problem of this method is the increased complexity of the attention layer, which is required to calculate a very large number of scores. This kind of complexity is likely to be one of the causes for the main problem we examined in the previous part: overfitting.

In this part of the thesis, we will study an architecture presented in the work by (Tashiro 2021). The researchers propose the use of a network based on the so-called ‘diffusion’ technique. This technique was proposed by (Ho 2020) for the synthetic data construction problem. In recent years, models based on the diffusion technique have shown significant results in the construction of synthetic images and in many cases achieve better results than corresponding GAN networks.

16.1 Introduction

To facilitate the presentation of the basic concepts concerning the diffusion technique we will use the example of the construction of synthetic data for images. After that, we will examine in more detail the issues that have to do with the specific problem we are considering, which concerns time series forecasting.

The diffusion method has in many cases presented results comparable to or better than networks based on GANs. The biggest advantage of this method is that the training of the network is much more stable compared to GANs and the error used is simpler and provides us with a better understanding about the course of the training.

The basic idea is to train a network to predict noise in a noisy signal. Knowing the noise, we can denoise the original signal. The production of synthetic data begins by giving the network a signal that consists of 100% noise. Then, the trained network will pass the signal through successive stages of denoising, gradually adding a larger ratio of (predicted) signal to noise. Theoretically, at the end of the process we will have synthetic data that is not in the original dataset but similar to the original ones. Building on previous publications, which had identified a connection between thermodynamic diffusion and deep neural networks, (Ho 2020) proposed in 2020 the so-called Denoising Diffusion Probabilistic Models. These types of models have found widespread application in many popular artificial intelligence software products.

16.2 Adding the noise

Regarding the addition of noise to the original signal, we note that the noise should be added in such a way that the mean and variance do not change. Usually the first step is to normalize the signal so that it has a mean of 0 and a variance of 1. We can then add the noise to the signal at some signal-to-noise ratio in the following way:

$$x_t = \sqrt{a_t}x_0 + \sqrt{1 - a_t}\varepsilon$$

Equation 10. Forward diffusion. Addition of the noise. (Foster 2023)

Where x_t represents the noisy signal, x_0 the original signal while ε represents the noise (usually, a sample from the Gaussian distribution with a mean of 0 and a variance of 1). As for a_t , it determines the variation, (and therefore the intensity) of the noise to be added to the signal depending on the value of t . For example, most commonly (there are exceptions, depending on the "program" or "diffusion schedule", as we will see below) $a_0 = 1$ and as t increases (up to the value T), the more the noise increases in x_t . If x_0 and ε have both a mean of 0 and a variance of 1, then, x_t will have the same mean and variance as the original signal.

Usually a 'schedule' is used to determine the values of a_t , which controls the variation of the noise added. The purpose of this program is to facilitate the network, giving it the opportunity to train on signals that do not contain excessively high levels of noise. For example, in a linear program, noise will be added at a constant rate. Later work has shown that it is better to add noise more gradually, giving more emphasis to the signal and making the network easier to train for the first few epochs. For example, a popular diffusion schedule is the following (where T is the maximum value that the parameter t can take):

$$a_t = \cos^2\left(\frac{t}{T} \frac{\pi}{2}\right)$$

and therefore we have:

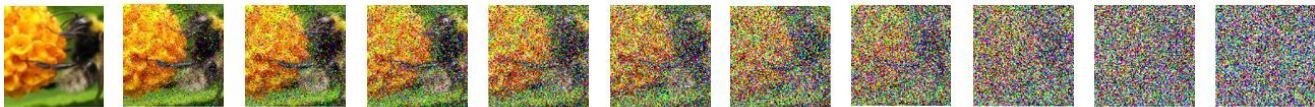
$$x_t = \cos\left(\frac{t}{T} \frac{\pi}{2}\right)x_0 + \sin\left(\frac{t}{T} \frac{\pi}{2}\right)\varepsilon$$

Equation 11. Cosine diffusion schedule. (Foster 2023)

There are many variations of the above program, here we just mention the basic idea.

In Picture 26 we can see the difference between different noise programs applied in an image:

Linear
schedule:



Sinusoidal
schedule:

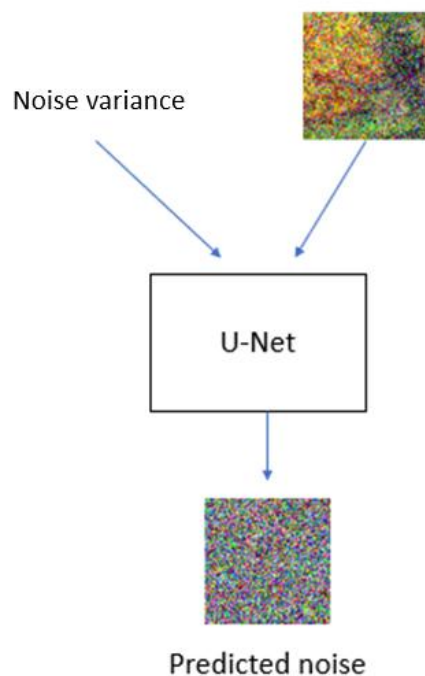


Picture 26 Linear and Sinusoidal forward diffusion schedule

When training the network, we construct batches by taking our noisy samples according to the forward diffusion schedule, giving Equation 11 random values for t from the uniform distribution.

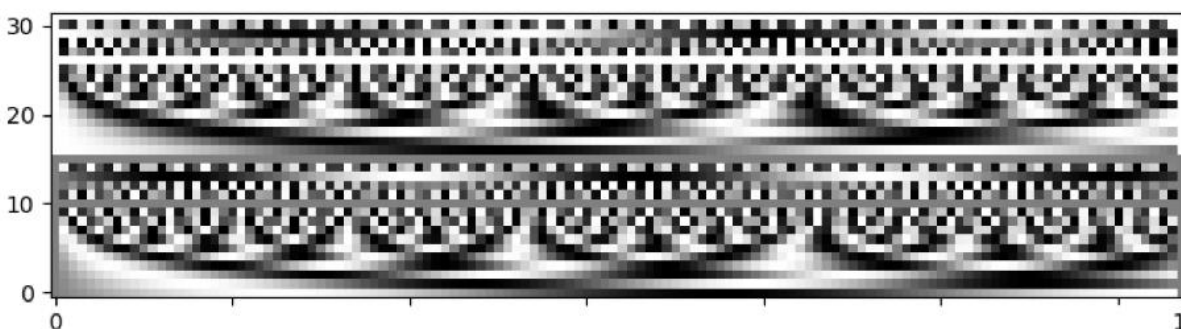
16.3 Sinusoidal embedding

Regarding the training of the network predicting the noise, we should note that at its inputs, in addition to the noisy signal, it should also receive information about the variation of the noise that the noisy signal includes. The image shows this process for the denoising of images. In this case, the U-Net architecture (among other methods) can be used (see (Foster 2023)):



Picture 27. The basic network in diffusion-based architectures learns to predict noise.

A question arises as to how to incorporate information about the amount of noise in the signal. One solution is to use the so-called sinusoidal encoding. This is a variant of positional encoding that supports the encoding of real numbers (instead of integers). Therefore, we can get encodings for the range 0 – 1 representing the noise intensity. The image below visually shows the sinusoidal encoding for a vector dimension of size 32:



Picture 28. Sinusoidal encoding

In the case of images, for example, one solution for incorporating noise intensity information into the noisy signal is to take the sinusoidal encoding and pass it through some convolutional up-sampling layers to produce a tensor that has the same dimensions as the original image (see (Foster 2023)). Then, one solution is to simply concatenate the channels of the 2 images. Then this new image with a larger number of channels can be passed through the U-Net or whatever network we are using.

16.4 Reverse diffusion

We now come to the construction of synthetic data after having trained the network that predicts noise in a noisy signal. To produce new synthetic data, we must start by giving the network we trained a signal that is completely random (100% noise). If we make a single forward pass to this network, the result we will get is not expected to be satisfactory. To improve the result, we can denoise the original signal according to the predictions for the noise, and construct a new noisy signal with a higher signal-to-noise ratio. We apply this procedure several times, each time essentially denoising the image and then re-applying noise to it using the predictions (predicted noise). Each step of the loop adds a smaller percentage of noise thus increasing the signal to noise ratio. Thus, the network gradually constructs the synthetic data having as its initial input a signal consisting of 100% noise. Essentially, at each step the network will reduce the noise a little until we get to a sufficiently low noise percentage.

The reverse diffusion process therefore denoises the signal gradually. In each iteration, a denoising step is performed followed by a re-noising step, but adding a smaller amount of noise than in the previous iteration. The source of this added noise may vary depending on the implementation. For example, in the book by (Foster 2023) the noise that will be added at each step is derived from the latest predictions (the network predicts the noise part of the noisy signal). In contrast, in the work by (y-tashi 2021) the

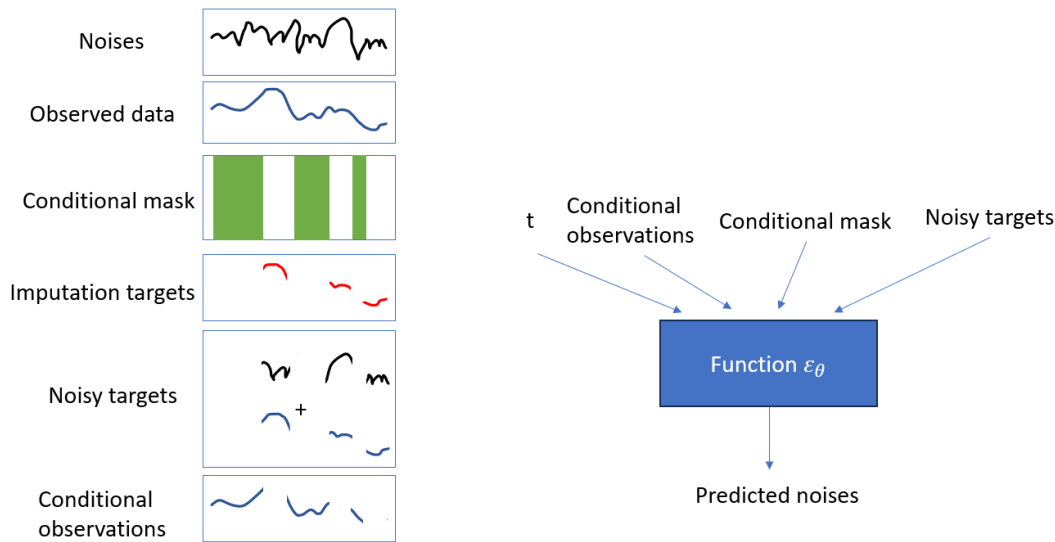
noise at each step of the reverse diffusion is taken from a random noise sample each time. This is an element that should be evaluated experimentally to decide the suitability of one method or another.

17. Using diffusion to construct synthetic data for time series

We previously described the diffusion process using the example of synthetic image construction because this makes it much easier to present the basic concepts. Let us now look at some techniques related to how we could use the diffusion process to construct synthetic data concerning time series, which is the main object of study of this thesis.

17.1 CSDI: Conditional Score-based Diffusion Model for Probabilistic Time Series Imputation

An interesting recent work that showed the effectiveness of using the diffusion technique for constructing synthetic data for time series is: (Tashiro 2021). In this work, researchers attempt to fill in



Picture 29. The basic idea behind the CSDI architecture. (Tashiro 2021)

time series that contain random gaps (i.e., missing data). This process is usually called imputation.

The researchers propose introducing noise at the missing positions of the time series (or if we are talking about the training stage, the positions of the time series that have been selected as targets for denoising). This noise is then gradually transformed during the process of reverse diffusion into consistent synthetic data. In this work, 2 encoders of the type used in the transformer architecture (i.e. they include a self-attention layer) are used. One to process temporal correlations (temporal transformer

layer) and one to process correlations between different features within a time frame (columns, if one row is considered to correspond to one timestamp) (feature transformer layer). The researchers call their method CSDI: Conditional Score-based Diffusion Model for Probabilistic Time Series Imputation. The basic idea is presented in Picture 29 (t (diffusion times) refers to the 'intensity' of the noise added to the signal):

The researchers feed their network with two different types of data, the so-called conditional observations, which are given to the network as is (i.e. free of noise), and the so-called imputation targets, to which some noise will be added depending on the t that will be chosen, as shown in the image.

We can use the CSDI architecture not only for the problem of imputation but also for forecasting. This can be done by considering all the so-called 'conditional observations' as the context of our time series, that is, as known data related to the past (known) timestamps. As for the so-called 'imputation targets', these will correspond to the last n hours for which we want our network to construct its forecasts.

17.2 two stage spatiotemporal attention

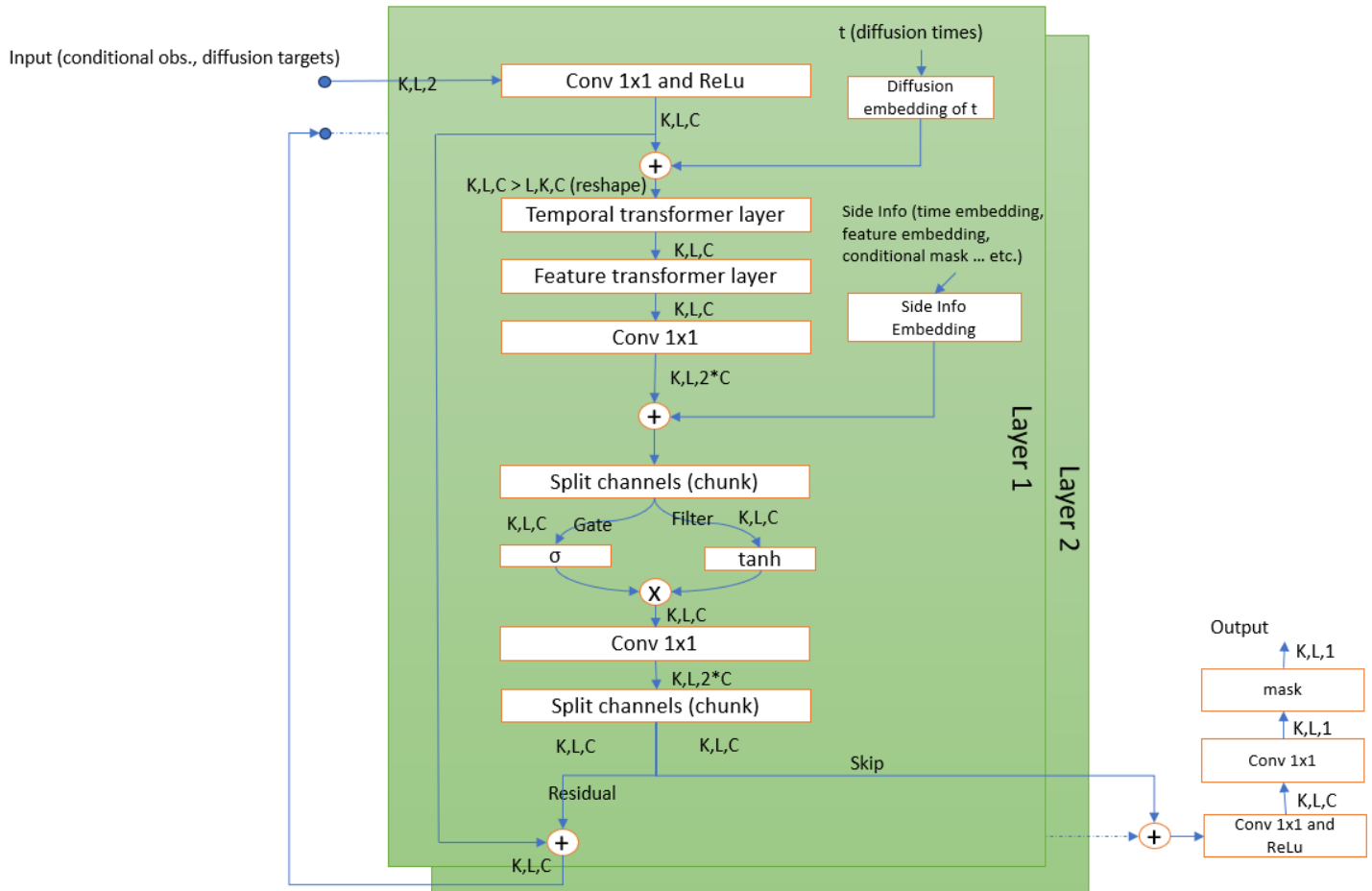
In the study of the previous proposed method, which is based on spatio-temporal attention, we studied, among other things, the work by (J Grigsby 2021) and discussed the problems related to the need to calculate a, potentially, very large number of attention scores.

In addition to this method, other alternative approaches have been proposed which are based on a two-stage processing: one stage that will process the temporal correlations (time axis) and one that will process the spatial ones (correlations between the features of the dataset).

An example of this technique can be found in the interesting work by (Tashiro 2021) where the CSDI architecture is proposed. The appendix provided with this publication (see (Yusuke Tashiro 2021)) provides more technical description about the method followed. An issue that arises with a two-stage approach has to do with ensuring the flow of information between the different attention layers (spatial and temporal attention). In the implementation of (Tashiro 2021) we have two different encoders with a self-attention layer. The researchers named the corresponding layers "temporal transformer layer" and "feature transformer layer" although, technically we have encoder layers and not an entire transformer network. By studying the implementation available on github (y-tashi 2021) we see that we have 2 consecutive encoder layers with attention (components of the transformer architecture). The queries to be used for each layer are determined by rearranging the dimensions of the layer's input (using the reshape function from PyTorch). The so-called "temporal transformer layer" passes its output directly to the "feature transformer layer". As mentioned earlier, in a two-stage approach we attempt to make distinct a process that may, theoretically, exhibit correlations between one stage and the other. Therefore, we must establish some way for the network to "pass" information from the spatial attention processing stage to the temporal attention processing stage and vice versa. In the work by (J Grigsby 2021) we have, theoretically, in one attention stage all the forms of correlations that exist (spatial, temporal or other) at the cost of the increased complexity of the model in its training: The so-called gradient tapes will be particularly 'extensive'.

We clarify that when we talk about gradient tapes, we are essentially referring to the computational graph that will be constructed by a software package such as TensorFlow, PyTorch, etc. In the context of these software packages, the term gradient tape refers to the data structure used to store the necessary records related to the automatic differentiation algorithms.

The solution proposed in the work by (Tashiro 2021) is based on the use of skip connections, residual layers, but also on the use of an idea that comes from the implementation of the LSTM: the so-called



Picture 30. The CSDI architecture. Some details are omitted for simplicity. You may refer to the original publication for a more technical understanding. Image based on: (Tashiro 2021)

'gated activation unit', which undertakes to separate its input into 2 elements: the residual, and skip as they are called in the implementation code, and which will be fed to the corresponding subsequent elements of the network architecture.

17.3 Other details about the CSDI network

In order to be able to use the CSDI network (Tashiro 2021) for the datasets that interest us (GEFCOM, AEMO), we will need to pass the information about future weather forecasts to the network, and also modify the embeddings, encodings, etc. accordingly.

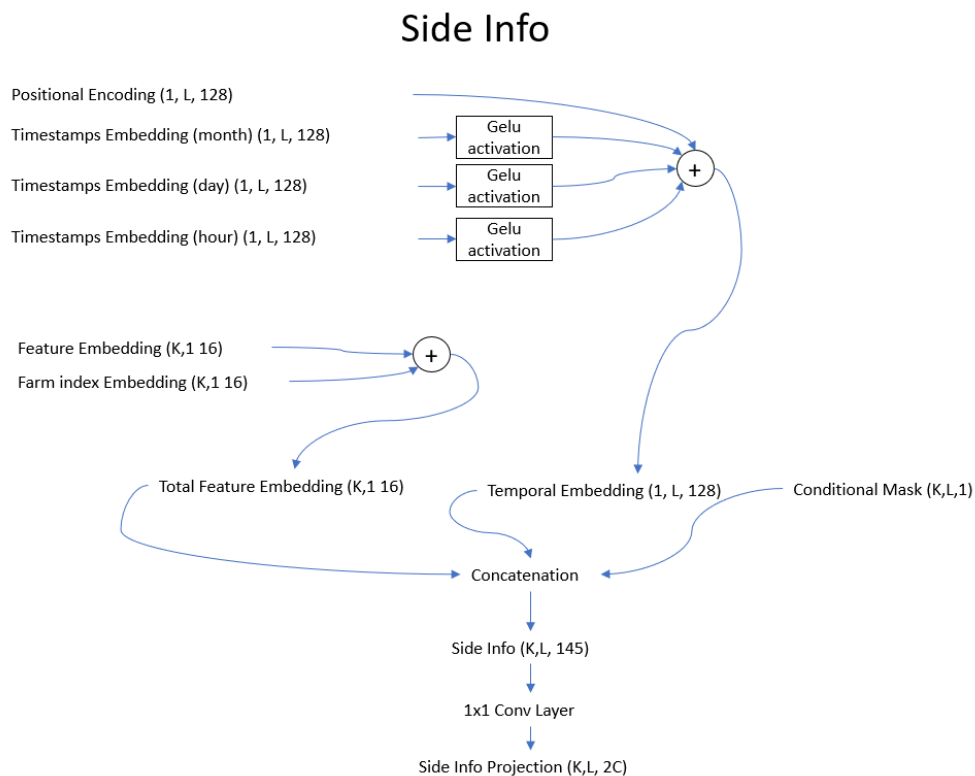
Regarding the integration of information related to the future weather forecast items, we can use the mask modification technique (in the encoder this time) as described in the analysis of the first method. (see paragraph: 11.2.4). Besides, we are not interested in producing synthetic data for future weather forecasts. All we are interested in is predicting the energy production from all the parks.

Essentially, these are adjustments that need to be made to better fit the CSDI model to the specific paradigm we are interested in. The essence of the model will not change of course; we just need to intervene in the embeddings and the masks.

To clarify the adjustments that will need to be made to the CSDI network so that it can process the time series of interest to us, we will need to mention some additional details regarding the structure of this particular network.

First, let's make a reference to the way embeddings and encodings are constructed. In the specific datasets that are of concern to us, we have a multi-level internal organization. Specifically, each element of the dataset consists, as we know, of 50 features: 10 farms by 5 elements (energy production plus 4 different weather forecasts) for each timestamp.

Therefore, one axis of organization of the data is temporal and concerns timestamps. Each of the 50 features is related to a specific (the same) timestamp. We also have another axis of organization which



Picture 31 Calculating the so-called 'side info' for CSDI.

concerns the identifier of each park, etc. That is, there are features related to park 1, park 2, etc.

The CSDI network uses an element called ‘side Info’ to pass to the network the embeddings and encodings that concern the ‘labeling’ (noisy targets mark, conditional observations mark etc.) of the internal organization of the features.

Picture 31 shows the calculations related to the ‘side info’ for the specific datasets we are concerned with. Where K is the number of features, L the number of time points, and C is the number of channels that the encoder layer outputs.

The ‘side info’ will then be added to the output of the 1×1 convolutional layer which follows immediately after the Feature Transformer Layer. This is why the shape of the side info is of dimensions $K, L, 2C$.

Regarding the mask that will highlight the elements that are part of the context and the elements to which noise has been added, (the diffusion targets as they are called) zeros are used for the context positions and units for the diffusion target positions.

Again, CSDI uses the concatenation technique followed by a 1×1 convolutional layer (y-tashi 2021) (Tashiro 2021). In other words, we have a concatenation between the so-called ‘conditional observations’ (the historical data of previous hours (context)) and the noisy target. The result is then passed to a convolutional layer.

Therefore, we can modify the mask using the same logic we described in the first proposed method.

17.3.1 Incorporation of the Variable Selection Layer from the TFT architecture

Another idea would be to use the Variable selection layer of the Temporal Fusion Transformer architecture at some point in the CSDI network. The Variable Selection Layer (VSL) is an important element of the TFT architecture which, as we have seen, has demonstrated remarkable results. Therefore, we believe it is a logical idea to use this layer in other architectures in the sense that the computation being performed is of a more generic nature (i.e. not strictly domain specific).

In general, the problem of selecting the appropriate features etc. is an important field of study in machine learning. Many different methods have been proposed for this problem, such as PCA, LDA, methods based on decision trees, Shapley values etc. In general, it is important to have a filter in the network that removes those elements of the input that are not expected to contribute to the quality of the predictions, or that could even create a problem. In this way, the network can focus, in a way, on the important elements, avoiding the processing of a signal that could be characterized as noise.

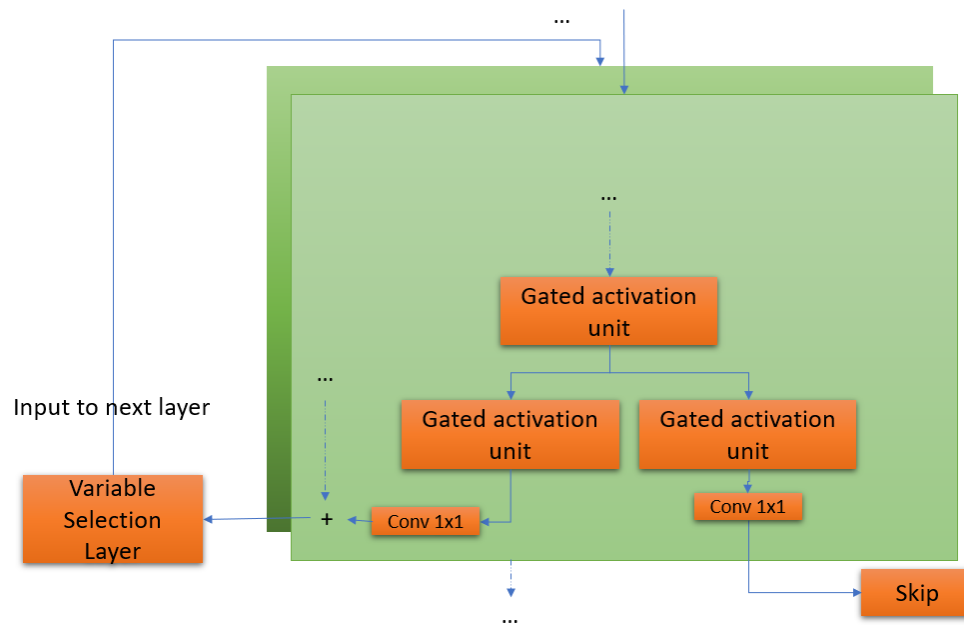
For example, one idea would be to interpolate the TFT’s VSL after calculating the residual of each layer of the CSDI network. The architecture of the VSL is shown in Picture 20.

Note: As we can see in this image, the VSL of the TFT network ‘compresses’ all the features into one. For example, in the case of the datasets we are examining (GEFCOM, AEMO) we have, as we have seen, 50 features. The CSDI network in the specific implementation we use processes time series that include all 50 features in all layers. In TFT we generally have a VSLayer followed by interpretable attention etc., but CSDI is essentially based on residuals. Therefore, one solution would be to skip the final summation of all the features and thus obtain an output from the VSL of the same dimensions as the input (50 features

instead of 1). This is the idea that we examined in our experiments and we note that exactly how the integration of this particular layer will have to be done is a subject of experimentation and is pertaining to a variety of decisions regarding the technical details of the internal operation of the VSL, the gated activation unit, the activation function, the position of the layer within the CSDI architecture, etc. Here we simply want to record this idea which in general seems logical, but the specific details of the implementation require some experimentation. So, in the scenario we examined we will not compress the 50 features into 1, but we hope that the layer will somehow 'enhance' the important features and reduce the influence of the unimportant ones (by giving smaller or larger values respectively). It is also noted that for our experiments, in each residual layer of the CSDI the VSL will be interpolated in the form we described above. However, the issue remains open and one can experiment more extensively by 'compressing' the 50 features into fewer or trying different positions for the integration of the layer, etc.

Also, another element that requires some experimentation is the usage (or the omission) of the LSTM layer that follows the VSL. From our experiments we conclude that the LSTM layer (encoder-decoder) does indeed contribute somewhat to the prediction results. Here again there are many different solutions we can use. For our experiments we chose to combine the features in a similar way to what we see in the multi head attention mechanism (concatenation etc.) followed by a stage of re-separation of the 50 features so that the output is passed to the next residual layer of CSDI (which is based on the existence of multiple features due to the usage of 2 encoders etc.). Again, we emphasize that in all these issues many design decisions are involved which require extensive experimentation etc. Here we tested

Picture 32. Possible integration of VSL into the CSDI network.



some cases and our main goal is to capture the basic idea.

17.4 Results of electricity production forecasts from wind farms using the CSDI network.

Before presenting the results, we will mention the parameters of the network used.

We had the following parameters:

Epochs: 50

Batch size: 16

Learning rate: 0.001 (optimizer: Adam)

Layers: 4

Channels: 64

Number of attention heads: 8

Diffusion schedule: quad

Time embedding: 128,

Feature embedding: 16

reverse diffusion steps: 50

Context: 18 h

Predictions: 24 h

Regarding the network loss used for training, the average of the sum of squares of the residuals was used for noise predictions.

We compare our method to the Temporal fusion transformer (TFT) network which is one of the leading networks for the problem of time series forecasting. (Bryan Lim 2021).

- First, we will present the results we obtained *without* using the Variable selection layer (VSL) of the Temporal Fusion Transformer (TFT). (see 17.3.1).

The results we obtained for the wind farm energy production forecasts, using the GEFCOM and AEMO datasets, are shown in the tables below:

- GEFCOM (the values apply to the test set)

	MAE	MSE
TFT	0.11494	0.02758

CSDI	0.1200	0.0300
------	--------	--------

Table 7. GEFCOM. 18 h of context, 24 h of forecasts. Test set

- AEMO (test set)

	MAE	MSE
TFT	0.1060	0.0216
CSDI	0.1650	0.0523

Table 8. AEMO. 18 h of context, 24 h of forecasts. Test set

- We also tried running the same experiments for more hours in the context (168 hours).

The results are shown in the tables below (we do not mention TFT because we do not have scores for more hours in the context):

- GEFCOM (test set)

	MAE	MSE
CSDI	0.1110	0.0254

Table 9. GEFCOM. 168 h of context, 24 h of forecasts

- AEMO (test set)

	MAE	MSE
CSDI	0.1440	0.0419

Table 10. AEMO. 168 h of context, 24 h of forecasts

We mentioned earlier that we need to take into account the internal structure and organization of each row of the dataset. These are very small modifications that do not fundamentally change the network but apparently, they need to be made. As we see, these specific micro-adjustments may, in some cases, prove to be somewhat beneficial. Let's see for example the results without these small adjustments to the embedding etc.

- GEFCOM. Without modified embedding layer and without modified mask. (values refer to test set)

	MAE	MSE
CSDI	0.186	0.0638

Table 11. GEFCOM. 18 h of context, 24 h of forecasts. Without modified embedding layer and without modified mask.

- AEMO. Without modified embedding layer and without modified mask. (test set)

	MAE	MSE
CSDI	0.202	0.0696

Table 12. AEMO. 18 h of context, 24 h of forecasts. Without modified embedding layer and without modified mask.

- GEFCOM. Without modified embedding layer and with modified mask. (values refer to test set)

	MAE	MSE
CSDI	0.122	0.0306

- AEMO. Without modified embedding layer and with modified mask. (numbers refer to test set)

	MAE	MSE
CSDI	0.186	0.0637

- Next, let's look at the results we obtained using the network variant described in the paragraph 17.3.1

The following tables relate to the use of SoftMax:

- GEFCOM (Context 18h, preds:24h)

	MAE	MSE
CSDI με VSL και LSTM	0.134	0.0345

- AEMO (Context 18h, preds:24h)

	MAE	MSE
CSDI με VSL και LSTM	0.201	0.0682

We also tried using Tanh as an activation function. The results are shown below.:

- GEFCOM (Context 18h, preds:24h)

	MAE	MSE
CSDI με VSL και LSTM	0.274	0.136

- AEMO (Context 18h, preds:24h)

	MAE	MSE
CSDI με VSL και LSTM	0.179	0.0558

17.5 Conclusions.

The results of this method seem to be somewhat more encouraging. The network is generally more 'lightweight' than the network of the previous method. The results are comparable to the TFT method, which is one of the leading methods for the problem we are examining. It seems that the network benefits to some extent from the use of a larger context. As a comment for future study, we could mention the use of separate encoder layers that specifically process future weather forecasts, since it is likely that the network's predictions are mainly influenced by these elements.

Regarding the usage of the VSL layer drawing from the TFT architecture, our implementation did not give better results compared to not using such a layer. Here we would like to make the following assessment: Although the initial results did not yield better results as one might expect, we estimate that it is possible that this specific method, namely the inclusion of some kind of variable selection layer in the CSDI architecture, can indeed give better results granted more experimentation. This is an intuitive assessment, of course, driven by the practical importance of the feature selection for many networks.

18. Aggregate results for all methods

At this point we present a table with the aggregate results for all our experiments and for the best models and methods, in order to facilitate the reader in evaluating the results overall. For a detailed commentary on each method, you can refer to the corresponding chapters.

- GEFCON, 18 h of context 24 h of forecasts. (test set)

method / metric	MAE	MSE
TFT	0.11494	0.02758
CSDI	0.1200	0.0300
CSDI with VSL and LSTM	0.1340	0.0345
Informer with ST attn and sliding windows	0.14217	0.03969

Table 13. GEFCON. 18 h of context, 24 h of predictions. Aggregate results for all methods.

- AEMO, 18 h of context 24 h of forecasts. (test set)

method / metric	MAE	MSE
TFT	0.1060	0.0216
CSDI	0.1650	0.0523
CSDI with VSL and LSTM	0.201	0.0682
Informer with ST attn and sliding windows	0.2047	0.0864

Table 14. AEMO. 18 h of context, 24 h of forecasts. Aggregate results for all methods.

Part E - Other thoughts - Suggestions for future study

19. Suggestions for future study

This study led us to use networks based on the Informer architecture using the so-called spatio-temporal attention. We also saw that we could potentially use the sliding windows training method to obtain somewhat more accurate predictions. However, we also identified the issue of overfitting.

Therefore, regarding the first of the two methods we proposed, we could continue our study in a second, future stage trying to address this specific overfitting problem. Below we will present some suggestions for possible directions for the future research on the subject.

19.1 Comments on synthetic data construction

In the following paragraphs we will present some thoughts and ideas regarding the construction of synthetic data as a way to address the problem of overfitting which we identified in previous paragraphs with regards to the first of the two methods proposed (spatio-temporal attention, informer, sliding windows, etc.). It should be noted that the problem of overfitting can be approached from many fronts. Here we are called upon to make decisions about the methods we will use. Due to the great interest in the construction of synthetic data, we will present some thoughts and ideas in this direction.

We will first describe some of the methods that have been proposed in the scientific literature and discuss their problems. Then we will present some ideas for constructing synthetic data using diffusion-based models.

So let's look at some of the options we have available in the following paragraphs.

19.2 Overview of available techniques for producing synthetic data

19.2.1 GANs

GANs (Generative Adversarial Networks) became particularly popular after the publication by (Goodfellow 2014) in 2014. The basic idea is to use a contraption consisting of 2 elements: The generator network and the discriminator network (or critic as it is sometimes called). The generator undertakes the task of the construction of synthetic data and the discriminator decides whether its input data is synthetic (i.e. has been derived from the generator network) or is 'real' data derived from the original dataset.

These 2 networks are trained in a 'competitive' way. Specifically, the generator network tries to adapt so that the discriminator cannot recognize its synthetic data as such, and the discriminator network is trained to recognize the synthetic data produced by the generator as different from the 'real' data taken from the dataset. Of course, during the training of the generator we will need to 'freeze' the weights of the discriminator network. Therefore, at each step of the training we have several backpropagations in the discriminator network and then the generator is trained whose error draws information from the discriminator network.

This is an interesting idea that has yielded many highly successful models for constructing synthetic images and other data.

19.2.1.1 The problems of the GANs

There are some problems with the vanilla version of GAN networks described above. Specifically, in the book by (Foster 2023) we find the following issues:

- uninformative loss

We usually expect the error to take a certain value (e.g. to tend to zero), and we can compare between batches, successive epochs, etc. This gives us an indication of whether the network is progressing in the right direction. Because training a GAN is a dynamic process, we cannot do the same for their loss. At each step, the generator is compared to the current discriminator, which is constantly improving. Therefore, it is difficult to judge how the network has done simply by studying the values of the error.

- Sensitivity to hyper-parameter tuning

GANs are generally quite sensitive to changes in their hyper-parameter values. Their training is also often unstable in the sense that it exhibits significant fluctuations.

- Mode collapse

This is the case where the generator finds a case for which the discriminator considers with great certainty that it is 'real' and not synthetic data and then the generator continuously produces the same or similar outputs since its goal is just to 'fool' the discriminator.

- The discriminator learns much faster than the generator

This is perhaps the biggest problem with GANs, at least in their vanilla form. If the discriminator learns the problem too quickly, the generator will not have a chance to improve since it will not know in which direction to move the weights of the network, since anything it produces will be rejected right away by the discriminator.

19.2.1.2 Wasserstein GAN with Gradient Penalty (WGAN-GP)

An improvement of the vanilla GAN which can partially address the above problems was proposed in (Gulrajani 2017) and is essentially a continuation of the work of (Arjovsky 2017). The main contribution of this work pertains to addressing the problem of excessively fast discriminator learning and the relatively 'unstable' training.

The basic idea is to somehow restrain the learning speed of the discriminator based on the norm of the gradients.

It is noted that in the original publication of (Arjovsky 2017) the problem of limiting the learning rate of the discriminator was addressed through gradient clipping. This idea was proven to theoretically ensure that the discriminator error function will be Lipschitz continuous. The slope of these functions (Lipschitz continuous) does not exceed a certain limit at any point. However, addressing this problem using gradient clipping is not particularly satisfactory and for this reason, an improvement using the gradient penalty method was proposed in the work by (Gulrajani 2017).

The idea is to use some penalty that will limit any excessively sharp decrease in the discriminator error. Unfortunately, it is not simple to calculate the value for this penalty. The problem is that we cannot easily get an estimate for the true slope of the discriminator error function at all points. Therefore, we will necessarily have to resort to some estimation using only a few points in the gradient space. The solution proposed by (Gulrajani 2017) to obtain an estimate for the gradient values is to pass through the network a batch consisting of data that has resulted from random interpolations between the real and synthetic data.

19.2.2 Real NVP

Another very interesting technique for constructing synthetic data published in 2016 proposes the use of a technique called real NVP (real-valued non-volume preserving transformation) (Dinh 2016). The network of this architecture learns an invertible mapping between a data distribution and a latent distribution (usually a Gaussian). You can find indicative implementations of this method in (Foster 2023) and (Mandolini Giorgio Maria 2020).

Here we will present the basic idea proposed in the work of (Dinh 2016) .

Essentially, the real nvp method attempts to construct a neural network that transforms the geometry of the training data space into the geometry of the Gaussian (normal) distribution space and vice versa. Thus, we can theoretically take samples from the normal distribution (possibly multidimensional) and using the transformation that the network learned, transform them into samples from the distribution of the data we are interested in. In a way, therefore, the network could be said to curve and transform the space and the geometry of the Gaussian distribution into the data space that we are interested in constructing synthetically.

The basic idea is to use the 'change of variable' formula:

$$p_X(x) = p_Z(f(x)) \left| \det \left(\frac{\partial f(x)}{\partial f(x^T)} \right) \right|, x \in X$$

Where X represents the space of the distribution of observations (prior probability distribution) p_X and Z represents the space of the latent (typically Gaussian) distribution p_Z .

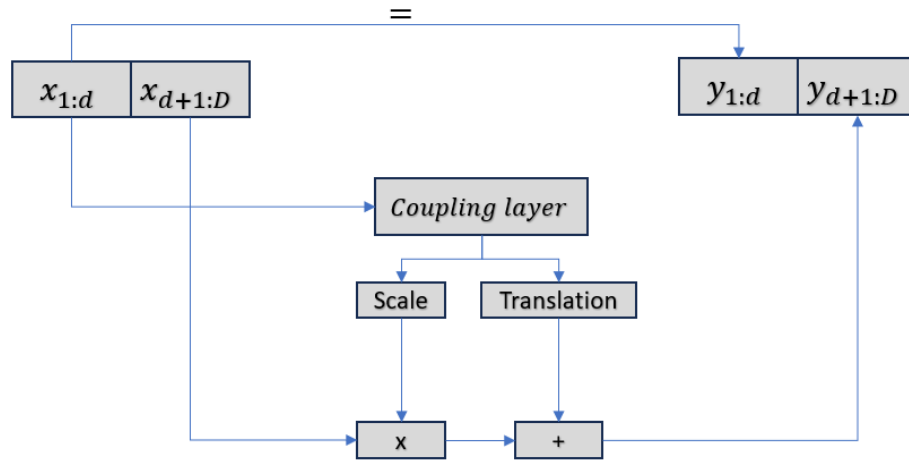
It is also noted that the function f is invertible ($g = f^{-1}$).

The idea is to use a neural network to learn this transformation (f). The problem with the above calculation is that it is not computationally easy to find the determinant of the Jacobian ($\det\left(\frac{\partial f(x)}{\partial f(x^T)}\right)$). This particular calculation has complexity of $\mathcal{O}(n^3)$. The researchers of the paper (Dinh 2016) therefore proposed a smart architecture for their neural network which guarantees that the calculation of the determinant will always be very easy. Specifically, the researchers take advantage of the fact that it is easy to calculate the determinant of a triangular matrix (it is calculated as the product of the elements of the diagonal). Thus, they proposed a layer which they call the coupling layer and which guarantees that the transformation f will be such that the matrix $\frac{\partial f(x)}{\partial f(x^T)}$ is always triangular.

Specifically:

Let y be the output of the coupling layer and x be its input. (x is an element of the dataset and y is a point in the latent distribution (e.g. Gaussian)). Also, x, y have D dimensions.

The function of the coupling layer is shown in the image below:



Picture 33 Coupling layer of a real nvp network (image based on: (Dinh 2016), (Foster 2023))

As can be seen, part of the D dimensions ($1:d$) will pass to the layer output without any processing. This clever trick results in a transformation that is both reversible (all we have to do is undo the scaling and translation that the input underwent) and also produces a triangular Jacobian.

The coupling layers should of course be placed in a sequential arrangement where, at each step, the part of the input that passed to the output without processing, passes through the processing stage in the next layer. This is implemented by using a mask which essentially alternates the processing of the parts $x_{1:d}$ and $x_{d+1:D}$ as shown in Picture 33.

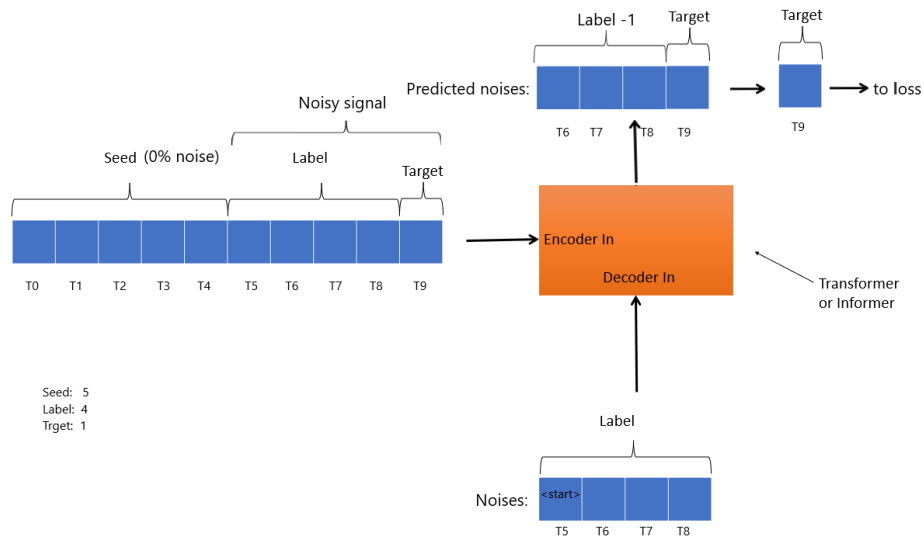
In general, there are many variations of the real nvp architecture which address a variety of different problems. Thus, at this point we will refer the reader to the literature for a more technical discussion. You can, for example, find some applications of this method for time series data in the interesting papers by (S. Wang 2022) and (Shibo Feng 2023).

19.2.3 Diffusion

Regarding the diffusion technique, we have already made an introduction. Here we will refer the reader to chapter 16.2 for a more detailed description of this specific technique.

19.3. Other suggestions and ideas for the architecture of a noise reduction network.

For a possible architecture of a network that performs noise predictions for timeseries, we can rely on Transformers and the Informer architectures, while also taking some ideas from the interesting work of



Picture 34 Possible architecture for the denoising network using the diffusion technique.

(Tashiro 2021).

Specifically, one idea is to use a Transformer or Informer network where at the input of the encoder we will have the noisy time series and at the input of the decoder the noises that have been added to the signal. A possible architecture that we would like to try in the future is shown in Picture 34. It is noted that the use of diffusion techniques in networks based on the Transformer architecture has been used in works such as: (Tashiro 2021) and (Ping Chang 2023) to name a few.

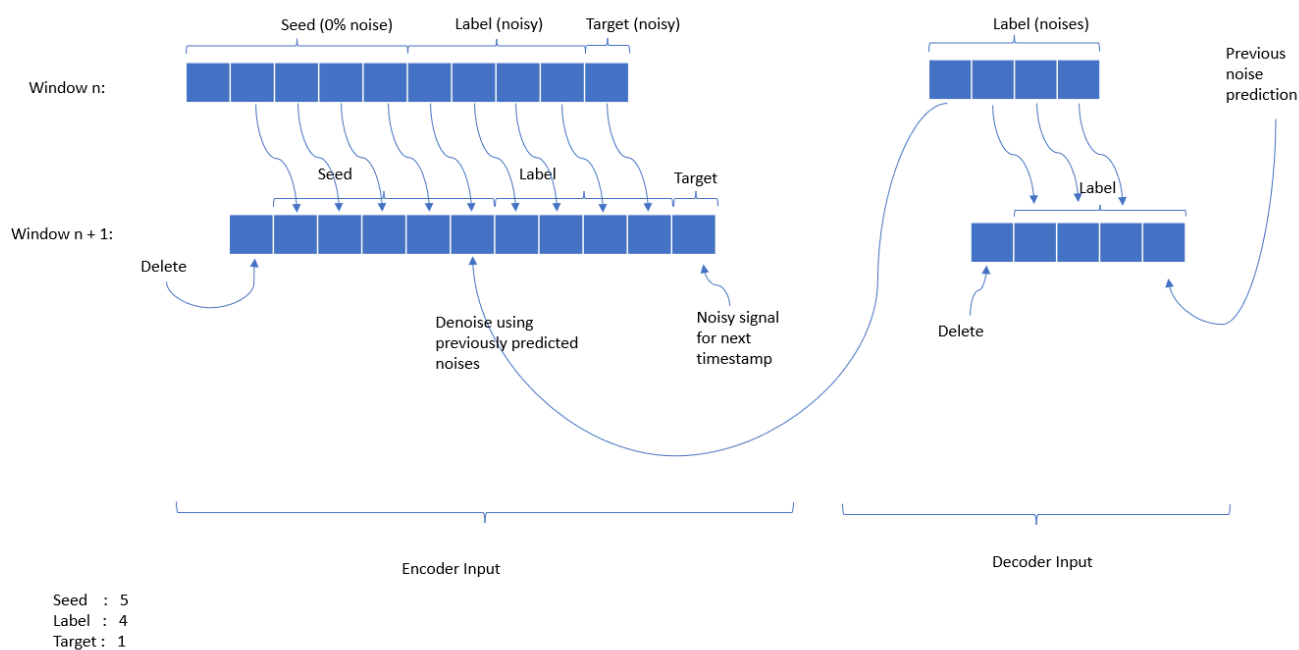
The idea is to use a small initial part of the sequence that is free of noise so that the network can form a ‘picture’ of the features of the sequence. This idea is, perhaps, similar (at least in general terms) to the idea presented in the work of (Tashiro 2021). We could also use an element that we can call e.g. label or preamble and which concerns the part of the sequence for which we have both the noisy signal and the noises at our disposal. Finally, the target element concerns the part of the sequence that we want to denoise and for which the noises are not known.

As we can see in Picture 34, in the encoder’s input sequence we have: Initially a chunk of size `seed_len` which includes consecutive tokens of the sequence without noise. This part is followed by a chunk of length `label_len` which includes the next tokens of the sequence, this time after adding some amount of noise. Then we have a chunk of length `target = 1` token, which includes the noisy signal for the time we want to denoise.

The decoder sequence includes a sequence of tokens of size `label_len` which includes the noises for the timestamps up to the target.

The above, of course, concern ideas for which we cannot prove whether they are effective or not. Essentially, we have listed some suggestions for a future study that we intend to implement in the near future.

19.3.1. Other ideas-suggestions: Training with sliding windows



Picture 35 Suggested possible design for sliding windows in combination with the denoiser.

Another idea would be to use the sliding windows training method that we proposed in the previous paragraph. The idea again is to use previous predictions of the network to construct the next predictions

and to incorporate this whole process into the training of the network. The process that we propose and intend to study in the future is shown in Picture 35. Essentially, the idea is to use the previous noise predictions to extend a seed portion of the decoder input using previous noise predictions to denoise the first token of the label, which in the next window will become the last token of the seed. We also use the previous noise predictions to extend the decoder input by adding the previous prediction to the end of the input sequence of the previous window and deleting the first token of the decoder input sequence, thus obtaining the next window.

19.3.2. Sinusoidal encoding for time series data

In a previous paragraph, we mentioned the sinusoidal encoding and how it can be used in a network based on the U-Net architecture. In the case of time series that will be processed by networks based on the transformer architecture, we could possibly add a sinusoidal encoding to each token in the same way that we add the positional encoding. In a way, sinusoidal encoding, like positional encoding, is like a watermark that is added to each token. Another idea would be to experiment with the usage of some additional embedding layer.

20. Epilogue

In this thesis, we studied the problem of predicting the energy output of RES parks and specifically from wind farms. We studied the respective literature on the subject and presented our own comments, methods and ideas. The basic idea we proposed concerns the use of the so-called spatio-temporal attention (J Grigsby 2021) and the Informer architecture (Zhou 2021) to construct rich spatial and temporal correlations between different parks which are assumed to be located at a geographical proximity and therefore the predictions for one may influence the predictions for another.

We described the pre-processing of the the dataset and the corresponding encoding and embedding layers. We also proposed a two-stage training method using sliding windows, with the aim of reducing the effect of overfitting and also for, hopefully, constructing more reliable predictions. This sliding windows mechanism seemed to be of significant benefit, allowing the network to produce predictions that appear (we emphasize that we cannot present any proof here) to be close but still slightly worse than the Temporal Fusion Transformer architecture, which is considered one of the best architectures for time series processing.

We also proposed a modification to the Informer decoder mask to allow queries to access future weather forecasts.

Finally, we identified the problem of overfitting in our network and proposed some ideas for addressing it.

After presenting our arguments, we hypothesized that a possible solution to address the problem may involve the construction of synthetic data with the aim of enriching the dataset. We examined several methods for constructing synthetic data based on GANs and the so-called Diffusion, and we discussed their advantages and disadvantages.

As a second alternative method, we examined the use of the CSDI network to construct our predictions using the Diffusion technique. The results from this second method were quite satisfactory and this particular technique seems to be generally better than the first method which uses the so-called spatio-temporal attention etc. The network also has the advantage of being trained in a shorter period of time.

Finally, we presented some ideas-thoughts for the alternative construction of a network for the production of synthetic data based on the diffusion technique. We addressed the problem of generating noise that follows the same distribution as the features of the dataset and then proposed a possible architecture for the network.

As a suggestion for a future study, we could propose, in addition to experimenting with architectures for the construction of synthetic data, the examination of ensemble methods (usage of multiple models) with the aim of achieving better quality in the predictions and, most importantly, less overfitting to the training dataset.

Bibliography - references

- Arjovsky, Martin, Soumith Chintala, Léon Bottou. 2017. "Wasserstein generative adversarial networks." *International conference on machine learning*. PMLR.
- Bryan Lim, Sercan O. Arik, Nicolas Loeff, Tomas Pfister. 2021. "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting." *International Journal of Forecasting* 37 (4).
- Chenhan Zhang, James J. Q. Yu, Yi Liu. 2019. "Spatial-Temporal Graph Attention Networks: A Deep Learning Approach for Traffic Forecasting." *IEEE Access SPECIAL SECTION ON ARTIFICIAL INTELLIGENCE (AI)-EMPOWERED INTELLIGENT TRANSPORTATION SYSTEMS*.
- Chevalier, Guillaume. 2023. "Wikimedia Commons." *LSTM_Cell*.
https://commons.wikimedia.org/wiki/File:LSTM_Cell.svg.
- Dinh, L., Sohl-Dickstein, J., & Bengio, S. 2016. "Density estimation using real nvp." *arXiv preprint arXiv:1605.08803*.
- Eurostat. 2023. *Renewable energy statistics*. Accessed 08 2023. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Renewable_energy_statistics.
- Fabio Lima, Tsang Ing Ren, Alexandre Costa. 2022. "Wind Power Forecast Based on Transformers and Clustering of Wind Farms with Temporal and Spatial Interdependence." *International Joint Conference on Neural Networks* (International Joint Conference on Neural Networks ()).
- Fleuret, Francois. n.d. *Deep Learning Course*. <https://fleuret.org/dlc/>.
- Foster, David. 2023. *Generative Deep Learning: Teaching Machines To Paint, Write, Compose, and Play*. 2nd edition. O'Reilly.
- Glassner, Andrew. 2023. "*Deep Learning - A Visual Approach*" by Andrew Glassner. *The official book repository*. <https://github.com/blueberry-music/Deep-Learning-A-Visual-Approach>.
- . 2021. *Deep Learning: A Visual Approach*. no starch press.
- Goodfellow, Ian, et al. 2014. "Generative adversarial nets." *Advances in neural information processing systems*.
- Gulrajani, Ishaan, et al. 2017. "Improved training of wasserstein gans." *Advances in neural information processing systems*.
- Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. "Denoising diffusion probabilistic models." *Advances in neural information processing systems*.
- Huijuan Wua, Keqilao Meng, Daoerji Fan, Zhanqiang Zhang, Qing Liu. 2022. "Multistep short-term wind speed forecasting using transformer." *Energy*.

- Ian Goodfellow, Yoshua Bengio, Aaron Courville. 2016. *Deep Learning (Adaptive Computation and Machine Learning series)*. Illustrated edition. The MIT Press.
- Inkit Padhi, Yair Schiff, Igor Melnyk, Mattia Rigotti, Youssef Mroueh, Pierre Dognin. 2021. "TABULAR TRANSFORMERS FOR MODELING MULTIVARIATE TIME SERIES." *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- Iz Beltagy, Matthew E. Peters, Arman Cohan. 2020. "Longformer: The Long-Document Transformer arXiv:2004.05150."
- J Grigsby, Z Wang, N Nguyen, Y Qi. 2021. "Long-Range Transformers for Dynamic Spatiotemporal Forecasting." *arXiv preprint arXiv:2109.12218*.
- J. Dowell, P. Pinson. n.d. "Very-short-term Probabilistic Wind Power Forecasts by Sparse Vector Autoregression." *IEEE Transactions on Smart Grid*.
- J. Schmidt, G. Lehecka, V. Gass, E. Schmid. 2013. "Where the wind blows: Assessing the effect of fixed and premium based feed-in tariffs on the spatial diversification of wind turbines." *Energy Economics* Volume 40.
- Jia, Yuening. 2023. "wikimedia commons." *The Transformer model architecture*.
<https://commons.wikimedia.org/wiki/File:The-Transformer-model-architecture.png>.
- Jiaan Zhang, Dong Liu, Zhijun Li, Xu Han, Hui Liu, Cun Dong, Junyan Wang. 2021. "Power prediction of a wind farm cluster based on." *Applied Energy*.
- K. CHOROMANSKI, V. LIKHOSHERSTOV, D. DOHAN, X. SONG, A. GANE, T. Sarlós, P. HAWKINS, J. DAVIS, A. MOHIUDDIN, L. KAISER, D. BELANGER, L. J. COLWELL, AND A. WELLER. 2021. "Rethinking attention with performers. ArXiv, abs/2009.14794."
- Keras. 2023. *sine_position_encoding layer*. Accessed October 2023.
https://keras.io/api/keras_nlp/modeling_layers/sine_position_encoding/.
- Lei Wang, Yigang He, Xiaoyan Liu, Lie Li, Kaixuan Shao. 2022. "M2TNet: Multi-modal multi-task Transformer network for ultra-short-term wind power multi-step forecasting." *Energy Reports*.
- M. ZAHEER, G. GURUGANESH, K. A. DUBEY, J. AINSLIE, C. ALBERTI, S. ONTAÑÓN, P. PHAM, A. RAVULA, Q. WANG, L. YANG, AND A. AHMED. 2020. "Big bird: Transformers for longer sequences." *Advances in neural information processing systems* (33).
- machinelearningmastery. 2023. *One-hot encoding*. Accessed 2023.
<https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/>.
- Mandolini Giorgio Maria, Sanna Daniele, Zannini Quirini Giorgio. 2020. "Density estimation using Real NVP." https://keras.io/examples/generative/real_nvp/.
- Minh-Thang Luong, Hieu Pham, Christopher D. Manning. 2015. "Effective Approaches to Attention-based Neural Machine Translation." *Computer Ence*.
- Nikita Kitaev, Łukasz Kaiser, Anselm Levskaya. 2020. "REFORMER: THE EFFICIENT TRANSFORMER." *ICLR 2020*.

- Ping Chang, Huayu Li, Stuart F. Quan, Shuyang Lu, Shu-Fen Wung, Janet Roveda, Ao Li. 2023. "TDSTF: Transformer-based Diffusion probabilistic model for Sparse Time series Forecasting." *arXiv:2301.06625 [cs.LG]*.
- PyTorch. 2023. *Embedding*. <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>.
- . 2023. *PyTorch Documentation*. <https://pytorch.org/docs/stable/>.
- Rothman, Denis. 2022. *Transformers for Natural Language Processing*. 2nd Edition. Packt Publishing.
- S. Wang, F. Zhou, Y. Sun, L. Ma, J. Zhang and Y. Zheng. 2022. "End-to-End Modeling of Hierarchical Time Series Using Autoregressive Transformer and Conditional Normalizing Flow-based Reconciliation." *IEEE International Conference on Data Mining Workshops (ICDMW), Orlando, FL, USA*.
- scikit-learn. 2023. *cosine-similarity*. Accessed 2023. <https://scikit-learn.org/stable/modules/metrics.html#cosine-similarity>.
- Shibo Feng, Chunyan Miao, Ke Xu, Jiaxiang Wu, Pengcheng Wu, Yang Zhang, Peilin Zhao. 2023. "Multi-scale Attention Flow for Probabilistic Time Series Forecasting." *IEEE Transactions on Knowledge and Data Engineering*. doi:10.1109/TKDE.2023.3319672.
- T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, and R. J. 2016. "Probabilistic energy forecasting: Global energy forecasting competition 2014 and beyond." *International Journal of Forecasting* 32.
- Tamura, Yasuto. 2023. *LSTM back propagation: following the flows of variables*. Accessed 2023. <https://data-science-blog.com/blog/2020/09/07/back-propagation-of-lstm/>.
- Tashiro, Y., Song, J., Song, Y., & Ermon, S. 2021. "CSDI: Conditional score-based diffusion models for probabilistic time series imputation." *Neural Information Processing Systems* 34.
- TensorFlow. n.d. "tf.GradientTape." https://www.tensorflow.org/api_docs/python/tf/GradientTape.
- Uday Kamath, Kenneth Graham, Wael Emara. 2022. *Transformers for Machine Learning. A Deep Dive*. 1st. Chapman & Hall.
- Vaswani, Ashish, et al. 2017. "Attention is all you need." *Advances in neural information processing systems*.
- Vijay Prakash Dwivedi, Xavier Bresson. 2020. *A Generalization of Transformer Networks to Graphs*. arXiv:2012.09699 [cs.LG] AAAI 2021 Workshop on Deep Learning on Graphs: Methods and Applications (DLG-AAAI 2021).
- Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, Ruslan Salakhutdinov. 2019. "Transformer Dissection: An Unified Understanding for Transformer's Attention via the Lens of Kernel." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

- y-tashi. 2021. *The github repository for the NeurIPS 2021 paper "CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation"*. <https://github.com/ermongroup/CSDI>.
- Yusuke Tashiro, Jiaming Song, Yang Song, Stefano Ermon. 2021. "CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation (Supplement)." https://proceedings.neurips.cc/paper_files/paper/2021/file/cfe8504bda37b575c70ee1a8276f3486-Supplemental.pdf.
- Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J. 2023. *Dive into Deep Learning*. Cambridge University Press. <https://D2L.ai>.
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. 2021. "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting." *Proceedings of the AAAI Conference on Artificial Intelligence* (35(12)).
- zhouhaoyi. 2020. *Informer github repository*. <https://github.com/zhouhaoyi/Informer2020>.

Appendix

Aggregate results - analytic result tables for the first method

➤ GEFCOM

Architecture: Informer with spatio-temporal attention and custom masking		
Parameters: Setup 1 (context: 18h, n_preds:4h , factor:50)		
Criterion: Best model based on the MSE loss in validation		
Dataset: Train		
Metric / Training stage	Stage 1 (4h)	Stage 2 (24h)
MAE	0.11781	0.12172
MSE	0.02507	0.02654
Dataset: Validation		
Metric / Training stage	Stage 1 (4h)	Stage 2 (24h)
MAE	0.11680	0.14982
MSE	0.02473	0.03960
Dataset: Test		
Metric / Training stage	Stage 1 (4h)	Stage 2 (24h)
MAE	0.11155	0.14217
MSE	0.02422	0.03969

Table of results 1. (Setup 1, Gefcom, stage 1,2)

Architecture: Informer with spatio-temporal attention and custom masking		
Parameters: Setup 2 (context: 18h, n_preds:12h , factor:90)		
Criterion: Best model based on MSE loss in validation		
Dataset: Train		
Metric / Training stage	Stage 1 (12h)	Stage 2 (24h)
MAE	0.11315	0.10010
MSE	0.02276	0.01805
Dataset: Validation		
Metric / Training stage	Stage 1 (12h)	Stage 2 (24h)
MAE	0.14379	0.15370
MSE	0.03582	0.03956
Dataset: Test		
Metric / Training stage	Stage 1 (12h)	Stage 2 (24h)
MAE	0.13696	0.15293
MSE	0.03755	0.04285

Table of results 2. (Setup 2, Gefcom, stage 1,2)

24 hours of predictions for the best model with setup 1 parameters, with stage 1 training only (without sliding window training)	
MSE (validation)	0.05356
MSE (test)	0.05174

Table of results 3. (Setup 1, Gefcom, stage 1 for 24h of predictions)

24 hours of predictions for the best model with setup 2 parameters, with stage 1 training only (without sliding windows training)	
MSE (validation)	0.06163
MSE (test)	0.05501

Table of results 4. (Setup 2, Gefcom, stage 1 for 24h of predictions)

Architecture: TFT	
Parameters: 12 h of predictions. (Parameters found after grid search, omitted for brevity)	
Criterion: Best model based on MSE loss in validation	
Dataset: Test	
Metric (avg. of all parks) / Training stage	
MAE	0.11389
MSE	0.02692

Table of results 5. TFT, Gefcom (12h of predictions)

Architecture: TFT	
Parameters: 24 h of predictions. (Parameters found after grid search, omitted for brevity)	
Criterion: Best model based on MSE loss in validation	
Dataset: Test	
Metric (avg. of all parks)	
MAE	0.11494
MSE	0.02758

Table of results 6. TFT, Gefcom (24h of predictions)

Architecture: Vanilla Informer with default trainer	
Parameters: Default values and 24 hour of predictions	
Criterion: Best model based on MSE loss in validation	

Dataset: Validation	
Metric	
MAE	0.19955
MSE	0.06287
Dataset: Test	
Metric	
MAE	0.19411
MSE	0.05770

Table of results 7. Vanilla Informer without S-T attention.

➤ **AEMO**

Method\ MAE loss	Train	Validation	Test
TFT	-	-	0.1060
setup_1 (training using sliding windows)	0.0604	0.1857	0.2047

Table of results 8. Setup 1, AEMO (stage 1 +2)

Method\ MSE loss	Train	Validation	Test
TFT	-	-	0.0216
setup_1 (training using sliding windows)	0.01848	0.05981	0.0864

Table of results 9. Setup 1, AEMO (stage 1 +2)

Aggregate results - analytical result tables for the second method (CSDI)

- GEFCOM (test set)

	MAE	MSE
TFT	0.11494	0.02758
CSDI	0.1200	0.0300

Table of results 10. GEFCOM. 18 h of context, 24 h of predictions

- AEMO (test set)

	MAE	MSE
TFT	0.1060	0.0216
CSDI	0.1650	0.0523

Table of results 11. AEMO. 18 h of context, 24 h of predictions

We also tried running the same experiments for more hours in the context (168 hours).

The results are shown in the tables below (we do not mention TFT because we do not have scores for more hours in the context):

- GEFCOM (test set)

	MAE	MSE
CSDI	0.1110	0.0254

Table of results 12. GEFCOM. 168 h of context, 24 h of predictions

- AEMO (test set)

	MAE	MSE
CSDI	0.1440	0.0419

Table of results 13. AEMO. 168 h of context, 24 h of predictions

Image index

Picture 1. Unrolled RNN. Picture based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)	10
Picture 2 The chain rule. Based on: (Tamura 2023)	12
Picture 3 The chain rule. A more complex example. Source: (Zhang 2023)	12
Picture 4 An LSTM cell. Source of this image (Chevalier 2023)	13
Picture 5. Sequence 2 sequence using an RNN. Based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)	15
Picture 6 Transformer. The basic architecture. Source: (Jia 2023)	16
Picture 7 The basic difference between an RNN and an Attention layer	17
Picture 8 Embedding addition (Example)	18

Picture 9. The calculation of the self-attention. Image based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023)	21
Picture 10 An intuitive basic understanding for the Attention Scores	22
Picture 11 Self-attention. (Based on: (Glassner, "Deep Learning - A Visual Approach" by Andrew Glassner. The official book repository. 2023))	23
Picture 12 The construction of the training samples	34
Picture 13 Spatio-temporal attention. Based on: (J Grigsby 2021).....	37
Picture 14 Longformer with a window of size $w=1$	38
Picture 15 The encoder's input	43
Picture 16 Decoder self-attention mask.....	53
Picture 17 Informer's architecture	56
Picture 18 Sliding window	57
Picture 19. Temporal Fusion Transformer's architecture (Bryan Lim 2021).....	59
Picture 20. Temporal Fusion Transformer (Variable selection layer). Based on: (Bryan Lim 2021).	60
Picture 21 Training for setup 1 (GEFCOM)	67
Picture 22 Training for setup 2 (GEFCOM)	68
Picture 23 MAE Loss for the AEMO dataset.....	69
Picture 24 Example of forecasts for 24 hours (test set, best model (setup1 with sliding windows)) (GEFCOM dataset).....	70
Picture 25 Example of forecasts for 24 hours (another example) (GEFCOM)	71
Picture 26 Linear and Sinusoidal forward diffusion schedule	75
Picture 27. The basic network in diffusion-based architectures learns to predict noise	75
Picture 28. Sinusoidal encoding	76
Picture 29. The basic idea behind the CSDI architecture. (Tashiro 2021).....	77
Picture 30. The CSDI architecture. Some details are omitted for simplicity. You may refer to the original publication for a more technical understanding. Image based on: (Tashiro 2021)	79
Picture 31 Calculating the so-called 'side info' for CSDI.....	80
Picture 32. Possible integration of VSL into the CSDI network.	82
Picture 33 Coupling layer of a real nvp network (image based on: (Dinh 2016), (Foster 2023)).....	91
Picture 34 Possible architecture for the denoising network using the diffusion technique.	92
Picture 35 Suggested possible design for sliding windows in combination with the denoiser.....	93

Tables Index

Table 1. Aggregated results for 24 hours of predictions (MAE) (GEFCOM)	62
Table 2. Aggregated results for 24 hours of predictions (MSE) (GEFCOM)	63
Table 3. Setup 1, AEMO, stage 1+2	63
Table 4. Setup1, AEMO, stage 1+2	63
Table 5. (Setup 1, Gefcom, stage 1 for 24h of predictions)	63
Table 6. (Setup 2, Gefcom, stage 1 for 24h of predictions)	64
Table 7. GEFCOM. 18 h of context, 24 h of forecasts. Test set.....	84
Table 8. AEMO. 18 h of context, 24 h of forecasts. Test set	84

Table 9. GEFCOM. 168 h of context, 24 h of forecasts	84
Table 10. AEMO. 168 h of context, 24 h of forecasts	84
Table 11. GEFCOM. 18 h of context, 24 h of forecasts. Without modified embedding layer and without modified mask.....	84
Table 12. AEMO. 18 h of context, 24 h of forecasts. Without modified embedding layer and without modified mask.....	85
Table 13. GEFCOM. 18 h of context, 24 h of predictions. Aggregate results for all methods.	87
Table 14. AEMO. 18 h of context, 24 h of forecasts. Aggregate results for all methods.....	87

Equations Index

Statement 1. The definition of the RNN.	10
Statement 2. Hidden state (RNN) (Zhang 2023).....	10
Statement 3. The problem with using very deep networks.....	13
Statement 4. The equations for the positional encoding of the vanilla Transformer.	19
Statement 5. Attention score	24
Statement 6. Attention score as inner product.....	24
Statement 7. Attention Scores (before the application of the SoftMax layer)	24
Statement 8. Calculating Attention. Attention can be thought of as a kind of weighted average.	24
Statement 9 Sparsity measurement	40
Equation 10. Forward diffusion. Addition of the noise. (Foster 2023)	74
Equation 11. Cosine diffusion schedule. (Foster 2023).....	74

Code snippets in PyTorch (for the first proposed method)

Code snippet 1. Positional encoding (zhouhaoyi 2020))

```

...
pe = torch.zeros(max_len, d_model).float()
pe.requires_grad = False

position = torch.arange(0, max_len).float().unsqueeze(1)
div_term = (torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model)).exp()

pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
...

```

Code snippet 2. Modified version of positional encoding for spatio-temporal attention timestamps

```
class PositionalEmbedding_per_timestamp(nn.Module):
    def __init__(self, d_model, max_len=7000, n_farms=10, tokens_per_timestamp=50):
        super(PositionalEmbedding_per_timestamp, self).__init__()
        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model).float()
        pe.requires_grad = False

        n_farms = n_farms
        n_weather_pred_cols = int((tokens_per_timestamp - n_farms)/n_farms)
        n_timestamps = math.ceil(max_len/(n_farms * (1+n_weather_pred_cols)))

        position =
        torch.tensor(np.arange(n_timestamps)).repeat_interleave(tokens_per_timestamp).repeat(n_timestamps).float().unsqueeze(1)
        position = position[:max_len]

        div_term = (torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model)).exp()

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return self.pe[:, :x.size(1)]

(βασίζεται στο (zhouhaoyi 2020))
```

Code snippet 3 Farm index embedding

```
class FarmIndexEmbedding(nn.Module):
    def __init__(self, d_model, n_farms, tokens_per_timestamp):
        super(FarmIndexEmbedding, self).__init__()
        self.tokens_per_timestamp = tokens_per_timestamp
        Embed = nn.Embedding
        self.farm_idx_embed = Embed(n_farms, d_model)
        n_weather_pred_cols = int((tokens_per_timestamp - n_farms)/n_farms)
        self.farm_id_stamp = torch.cat(
            (
                torch.arange(0, n_farms),
                torch.tensor(np.arange(n_farms)).repeat_interleave(n_weather_pred_cols)
            ), axis=0)

    def forward(self, x):
        x = x.long()
        n_timestamps = int(x.shape[-2]/self.farm_id_stamp.shape[-1])

        farm_indexes = self.farm_id_stamp.expand(n_timestamps, self.tokens_per_timestamp).reshape(-1,1)
        farm_indexes = farm_indexes.unsqueeze(0).expand([x.shape[0]] + list(farm_indexes.shape)).to(x.device)

        farm = self.farm_idx_embed(farm_indexes[:, :, 0])
```

```
return farm
```

Code snippet 4. The construction of the mask for the self-attention layer of the decoder.

```
mask = torch.einsum('ijkl,ij->ijkl',
                    torch.ones(n_queries, n_queries, args.n_feats, args.n_feats, dtype=torch.int),
                    torch.tril(torch.ones(n_queries, n_queries, dtype=torch.int)))

mask = mask.transpose(1, 2).reshape(n_queries*args.n_feats, n_queries*args.n_feats).float()
mask = mask.masked_fill(mask == 0, float('-inf'))
mask = mask.masked_fill(mask == 1, float(0.0))
```

Code snippet 5. Projection for the token embedding in our method((based on: (zhouhaoyi 2020)))

```
self.conv1 = nn.Conv1d(in_channels=(label_len + pred_len) * self.n_features, out_channels=self.pred_len +
                        self.label_len, kernel_size=1, bias=True)
self.conv2 = nn.Conv1d(in_channels=d_model, out_channels=self.n_farms, kernel_size=1, bias=True)
```

Code snippet 6. Modifying the ProbMask function (see. zhouhaoyi 2020))

In paragraph 11.2.4 we see the changes related to the decoder mask and concern the unimportant queries according to the estimate given by the sparsity measurement. Let's see how the scores are calculated in the attn.py file:

```
if self.mask_flag:
    attn_mask = My_ProbMask(B, H, L, Q, index, scores, device=V.device, n_farms=self.n_farms, n_cols=self.n_cols,
label_len=self.label_len, pred_len=self.pred_len)
    scores.masked_fill_(attn_mask.mask, -np.inf)

attn = torch.softmax(scores, dim=-1) # nn.Softmax(dim=-1)(scores)
```

We have implemented the My_ProbMask class which essentially contains some modifications of the original ProbMask as defined in the Zhou et al. code. More specifically:

```
class My_ProbMask():
    def __init__(self, B, H, L, index, scores, device="cpu", n_farms = 10, n_cols = 50, label_len=48, pred_len=24):
        _mask = torch.ones(L, scores.shape[-1], dtype=torch.bool).to(device).triu(1)
        _mask_ex = _mask[None, None, :].expand(B, H, L, scores.shape[-1])
        # If the query's position is not at the end of the n_cols columns, then do not apply the mask for all n_cols.
        # This is because every n_cols tokens (keys) are associated with a single timestamp (row of the dataset).
        index_timestamp_based = torch.ceil(index/50)*50 - 1
        index_timestamp_based = index_timestamp_based.type(dtype=torch.int)
        indicator = _mask_ex[torch.arange(B)[:], None, None,
                             torch.arange(H)[None, :, None,
                             index_timestamp_based, :].to(device)
        self._mask = indicator.view(scores.shape).to(device)

        mask2 = torch.empty([list(self._mask.shape[:-1]) + [(label_len + pred_len) * n_cols], dtype=torch.bool).fill_(False).to(device)
        mask2.reshape((-1, n_cols))[ :, n_farms:n_cols] = True
```

```

mask2.flatten()
self._mask = self._mask.masked_fill_(mask2, False)
@property
def mask(self):
    return self._mask

```

Code snippet 7. Modifications of the `_process_one_batch` function of the Informer (see (zhouhaoyi 2020))

```

# decoder input
dec_inp = torch.zeros([batch_y.shape[0], self.args.pred_len * n_cols, batch_y.shape[-1]]).float()
dec_inp.reshape((-1, n_cols))[:, n_farms:n_cols] = batch_y[:, self.args.label_len * n_cols:, :].reshape((-1, n_cols))[:,
n_farms:n_cols]

dec_inp = torch.cat([batch_y[:, :self.args.label_len * n_cols, :], dec_inp], dim=1).float().to(self.device)

```

The `batch_y` is the decoder input data but without the preamble (label). As we can see, in the part of the sequence that concerns the forecasts, we put zeros everywhere except for the positions concerning the weather forecasts where we have the actual data values.

